

PostgreSQL / Patroni / TimescaleDB — Operator Manual

What this technology does

PostgreSQL is the durable relational state layer. Patroni orchestrates one writable PostgreSQL leader plus replicas using etcd. TimescaleDB is an extension inside the `lorawan_telemetry` database; it is **not** a separate server.

Current commissioned baseline

- ULC-01/02/03 each host one Patroni/PostgreSQL member.
- Current commissioned PostgreSQL build: 18.6.
- Current TimescaleDB extension: 2.29.2.
- One writable leader, two streaming replicas in normal state.
- `chirpstack` database stores ChirpStack state.
- `lorawan_telemetry` stores telemetry, measurements, Fabric outbox, and gateway-evidence metadata.
- Normal clients use PgBouncer/HAProxy rather than pinning the current leader.

The normal client path

```
application
-> pgbouncer.internal.lorawan.com:6432
-> local PgBouncer
-> local HAProxy primary route :15432
-> current Patroni leader :5432
```

Do not bypass this path in an application configuration just because you know today's leader IP.

Step 1 — Establish the authoritative Patroni topology

On any database host:

```
sudo docker exec -e LC_ALL=C spilo patronictl -c /run/postgres.yml list
```

PASS means: exactly one member is Leader and the other intended members are running replicas without a critical lag/error state.

Do not use an old document to decide which node should be leader. The live Patroni result wins.

Step 2 — Confirm local PostgreSQL role when needed

On a specific member, use the approved local/admin connection method and run:

```
SELECT inet_server_addr(), pg_is_in_recovery();
```

Interpretation:

- `pg_is_in_recovery() = false` -> writable primary/leader candidate;
- `true` -> replica.

Never run a write/cleanup script on a node until you know whether it is the current authoritative leader and whether the script should use the normal HA route instead.

Step 3 — Verify the normal application route

Use a protected, least-privilege test role/DSN already provisioned on the host. Do **not** paste a password into the command history.

The important SQL is:

```
SELECT now() AT TIME ZONE 'UTC' AS utc_time,  
       inet_server_addr(),  
       pg_is_in_recovery();
```

PASS means: the connection through PgBouncer/HAProxy succeeds with TLS verification and lands on `pg_is_in_recovery() = false` for a primary-route client.

If direct member access works but the normal route fails, troubleshoot PgBouncer/HAProxy before changing Patroni.

Step 4 — Verify both logical databases exist

Through the approved admin/read-only path:

```
SELECT datname  
FROM pg_database  
WHERE datname IN ('chirpstack', 'lorawan_telemetry')  
ORDER BY datname;
```

Both databases must exist.

Step 5 — Verify TimescaleDB extension

Connect to `lorawan_telemetry` and run:

```
SELECT extname, extversion  
FROM pg_extension  
WHERE extname = 'timescaledb';
```

PASS means: one `timescaledb` row exists with the commissioned version or an intentionally documented upgraded version.

After any Patroni promotion, this must still work on the promoted member.

Step 6 — Verify required hypertables

```
SELECT hypertable_schema, hypertable_name
FROM timescaledb_information.hypertables
WHERE hypertable_schema = 'telemetry'
ORDER BY 1,2;
```

Required time-series objects include:

```
telemetry.uplinks
telemetry.measurements
```

telemetry.fabric_outbox is intentionally an ordinary transactional table, not a hypertable.

Step 7 — Verify fresh telemetry read path

Use a read-only role:

```
SELECT time, dev_eui, metric_name, metric_value, metric_text, metric_bool, unit, quality
FROM telemetry.measurements
ORDER BY time DESC
LIMIT 20;
```

Do not invent a sensor-specific column when the normalized measurements table is the current schema.

Step 8 — Inspect Fabric outbox separately from adapter health

```
SELECT status, count(*)
FROM telemetry.fabric_outbox
GROUP BY status
ORDER BY status;
```

Important rule: pending with attempts=0 is **not automatically a Fabric outage**. Before blaming the adapter, verify the row has immutable finalized_payload and, for the v2 path, verifier-owned status='verified' eligibility.

Use [15-hyperledger-fabric-adapter.md](#) for the exact claim/reconciliation rules.

Step 9 — Check replication/health before maintenance

Use `patronictl list` and the database runbook to inspect replication/lag. If a replica is behind, do not start an unrelated switchover or upgrade until its state is understood.

A PostgreSQL process being running does not mean it is safe for promotion.

Safe schema migration procedure

1. prove etcd 3/3 and one Patroni leader;
2. create/verify the required logical backup or rollback point;
3. use the normal primary-routed connection;
4. inspect migration prerequisites/current schema version;
5. run exactly one reviewed migration path;

6. verify transaction success and expected schema objects;
7. verify replicas remain healthy;
8. verify one representative application operation;
9. retain migration evidence/rollback notes.

Do not manually edit schema on replicas.

Safe PostgreSQL/Timescale upgrade rule

Database major/extension upgrades require the dedicated runbook. All promotion-eligible members must have compatible PostgreSQL and TimescaleDB builds before a node can safely become primary.

Do not point an older/newer PostgreSQL major at an existing data directory because a container happens to start.

Backup/recovery rules

- HA replication is availability, not backup.
- Logical deletion replicates too.
- Take/verify backups before destructive testing/migrations.
- Restore into a controlled recovery target first unless the disaster procedure explicitly calls for in-place recovery.
- Re-establish one authoritative leader and healthy replicas before reopening all application writers.

Troubleshooting order

Symptom	First check
No Patroni leader	etcd quorum + Patroni state
Leader healthy, app cannot connect	PgBouncer/HAProxy/TLS/auth
One replica lagging	WAL/network/disk/member logs
Timescale query fails after promotion	extension/version/preload on promoted member
Duplicate telemetry	Node-RED/event idempotency + DB constraints
Outbox stuck pending/0	finalized payload/verifier eligibility before Fabric

Completion checklist

- etcd healthy;
- exactly one Patroni leader;
- expected replicas healthy;
- normal PgBouncer/HAProxy primary route succeeds;
- chirpstack and lorawan_telemetry exist;
- TimescaleDB loads;
- required hypertables exist;
- representative telemetry read succeeds;

- outbox can be inspected without conflating eligibility with worker failure;
- current backup boundary is known.

Detailed runbooks

- [../server/cloud-production/06-spilo-patroni-postgresql-cluster.md](#)
- [../server/cloud-production/07-haproxy-and-pgbouncer.md](#)
- [../server/cloud-production/13-backup-restore-and-disaster-recovery.md](#)
- [../server/integrations/timescaledb/00-README.md](#)

Supporting runbook: 06-spilo-patroni-postgresql-cluster.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

6. Spilo, Patroni, and PostgreSQL Cluster

Status: ACTIVE - THREE-MEMBER POSTGRESQL/PATRONI HA CLUSTER ESTABLISHED; TIMESCALEDB 2.29.2 LIVE FUNCTIONALITY PASS; PERMANENT DATABASES + OWNERSHIP + RUNTIME CREDENTIAL ACTIVATION PASS; DIRECT VERIFY-FULL RUNTIME AUTH PASS; TELEMETRY OBJECT SCHEMA + ACL GATE PASS; HBA HARDENING PREFLIGHT PASS; ULC-02 PERSISTENT-ONLY REPAIR PASS; ULC-02 LIVE HARDENED 20-RULE HBA CANARY PASS; ULC-02 HARDENED HBA PERSISTENCE PASS; CROSS-HOST FUTURE-PRIMARY REPLICATION AUTH PASS; ULC-03 LIVE HARDENED 20-RULE HBA CANARY PASS; ULC-03 HARDENED HBA PERSISTENCE PASS; ULC-03 CROSS-HOST FUTURE-PRIMARY REPLICATION AUTH PASS; ULC-01 LEADER LIVE HARDENED 20-RULE HBA CANARY PASS; FRESH CROSS-HOST REPLICATION AUTH TO ULC-01 PASS; ULC-01 HARDENED HBA PERSISTENCE PASS; ALL THREE MEMBERS HARDENED LIVE + PERSISTENT; FINAL THREE-NODE HBA PARITY/TLS/NEGATIVE GATE PASS; FINAL POST-HARDENING APPLICATION-ROLE AUTH PASS; POC LOGICAL BACKUP BOUNDARY COMPLETE; CONTROLLED ULC-01 -> ULC-02 PATRONI SWITCHOVER PASS; ULC-02 PROMOTED-PRIMARY VALIDATION PASS; POST-PROMOTION APPLICATION AUTH PASS; PHASE 6 DATABASE-LAYER SWITCHOVER VALIDATION CLOSED; PHASE 7 HAPROXY/PGBouncer PREFLIGHT NEXT.

The reviewed patched Spilo image is published privately in GHCR and all three nodes verified the same immutable digest. Replacement cluster-role secrets are consistent. PostgreSQL certificates are installed and verified on all three members. The native storage migration is complete: `/srv/spilo/pgdata` is mode `0700` numeric `101:103` on all three nodes, and the protected environment files use `PGRROOT=/home/postgres/pgdata/pgroot` plus `PGDATA=/home/postgres/pgdata/pgroot/data`. Spilo has now been started on `ulc-01` only; the first PostgreSQL/Patroni bootstrap outcome is not yet validated. Deterministic Patroni member identity, the exact `ETCD3_HOSTS` parser path, the three-node pre-bootstrap checks, and the `ALLOW_NOSSL=` empty-string correction are proven. Exact-image source inspection shows the generated template defaults to wildcard listeners: `restapi.listen: '{{APIPORT}}'` and `postgresql.listen: '{{PGPORT}}'`. The supported override mechanism is now proven from the same immutable image: `SPILO_CONFIGURATION` is YAML-decoded into `user_config`, then merged as `deep_update(user_config_copy, config)`. `deep_update()` recursively merges dictionaries and returns the user value for scalar/list conflicts, so a user-supplied `restapi.listen` or `postgresql.listen` overrides the template default while unspecified settings remain from the generated configuration. The corrected isolated exact-image merge/render probe is now **PASS** on `ulc-03`: `instance_data.id` rendered as `ulc-03`; `restapi.listen` and `restapi.connect_address` both rendered as `10.104.0.8:8008`; `postgresql.listen` and `postgresql.connect_address` both rendered as `10.104.0.8:5432`; and `etcd3.hosts` remained the intended three east-west endpoints. The template context also proves the Patroni member name lives at `postgresql.name: '{{instance_data.id}}'`, not at top-level name, explaining the earlier diagnostic `KeyError`. The temporary Compose dotenv test is now **PASS**: outer single quotes

preserve the compact JSON string exactly, `docker compose config --quiet` succeeds, and the Compose-decoded value contains the intended `10.104.0.8:8008` and `10.104.0.8:5432` listeners. The listener correction is now applied and live-validated on all three protected env files: `ulc-01` decodes to `10.104.0.2:8008` and `10.104.0.2:5432`, `ulc-02` to `10.104.0.4:8008` and `10.104.0.4:5432`, and `ulc-03` to `10.104.0.8:8008` and `10.104.0.8:5432`; each node returned Compose syntax: PASS and LIVE LISTENER OVERRIDE: PASS. The combined private-listener plus conservative 2-GiB configuration is now live-validated on all three nodes. The final last-state check is also **PASS on ulc-01, ulc-02, and ulc-03**: only etcd is running, all three etcd endpoints are healthy, `10.104.0.4` is leader at this checkpoint, there are no learners or endpoint errors, ports `5432/8008` are free, `/srv/spilo/pgdata` is mode `0700` numeric `101:103` and empty, and Compose syntax passes. PostgreSQL/Patroni has still not been started. The next controlled action is to start Spilo on `ulc-01` only and verify the first primary before either replica is allowed to start. Exact-image inspection now proves the default placeholder code reads the cgroup/host memory limit, sets `local postgresql.parameters.shared_buffers` to one quarter of detected memory for the local provider, and sets `max_connections` to at least 100. The bootstrap DCS template separately emits `bootstrap.dcs.postgresql.parameters.max_connections`, while the local PostgreSQL section emits `postgresql.parameters.shared_buffers`; the inspected default template does not explicitly carry `work_mem` or `maintenance_work_mem`. Because the already-proven `SPILO_CONFIGURATION` deep merge preserves user-supplied extra keys, the same conservative values were render-tested in **both** `bootstrap.dcs.postgresql.parameters` and `local postgresql.parameters`. The exact-image probe showed the unmodified image would have used `shared_buffers = 491MB` and `bootstrap max_connections = 100` on this node; after the override, both parameter maps contained `max_connections = 40`, `shared_buffers = 128MB`, `work_mem = 2MB`, and `maintenance_work_mem = 32MB`, while the private listener values remained unchanged. The probe finished with 2-GIB BOOTSTRAP TUNING RENDER: PASS. The larger combined JSON also passed an isolated Docker Compose dotenv test on `ulc-03`: Compose syntax passed, the decoded private listeners remained `10.104.0.8:8008` and `10.104.0.8:5432`, and both the local and bootstrap parameter maps decoded to `max_connections = 40`, `shared_buffers = 128MB`, `work_mem = 2MB`, and `maintenance_work_mem = 32MB`, finishing with 2-GIB `SPILO_CONFIGURATION` dotenv: PASS. The live per-node replacement is now **PASS on all three nodes**. Each protected env file remains mode `0600` `0:0`, contains exactly one `SPILO_CONFIGURATION`, passes Compose syntax validation, keeps the node-specific `10.104.0.x` listeners, preserves `ALLOW_NOSSL=` as an empty string, and decodes both local and bootstrap parameter maps to `max_connections=40`, `shared_buffers=128MB`, `work_mem=2MB`, and `maintenance_work_mem=32MB`. The last-state gate is **PASS on all three nodes**. First bootstrap was attempted on `ulc-01` only and **FAILED before PostgreSQL initialization completed**. Patroni reached etcd, reported Lock owner: None; I am `ulc-01`, and attempted to bootstrap a new cluster, but `initdb` failed repeatedly with invalid locale name `"en_US.UTF-8"`. Patroni removed the initialize key after each failed attempt and renamed the failed data directory on the first attempt. No PostgreSQL or Patroni REST listener came up, `pg_isready` returned no response, and the current `/srv/spilo/pgdata/pgroot/data` path is absent after the failed initialization. This is a locale/bootstrap-input failure, not an etcd, TLS, listener-override, or memory-tuning failure. The failed retry loop is now stopped on `ulc-01`; Compose reports the container exited with code 143, ports `5432/8008` are free, `/srv/spilo/pgdata/pgroot/data.failed` is preserved under numeric `101:103`, and the Patroni DCS prefix `/service/lorawan-postgres-ha` currently returns no keys. Exact-image inspection confirms the mismatch: the template always renders `locale: {{INITDB_LOCALE}}.UTF-8`, the default is `INITDB_LOCALE=en_US`, and `locale -a` contains only `C`, `C.utf8`, and `POSIX`. Therefore the default becomes unavailable `en_US.UTF-8`, matching the runtime failure. The probe environment reports `LANG=''`, `LANGUAGE=''`, and `LC_ALL=C.UTF-8`. The candidate override is now proven: a disposable exact-image PostgreSQL 18.6 `initdb` run as numeric `101:103`, with no network and only a temporary `/tmp` data directory, successfully initialized with `--encoding=UTF8 --locale=C.UTF-8 --data-checksums --no-sync`. The generated cluster reported `PG_VERSION=18`,

wrote all locale settings as C.UTF-8, and finished with C.UTF-8 PostgreSQL 18 initdb: PASS. This probe also reports initdb (PostgreSQL) 18.6 (Ubuntu 18.6-1.pgdg22.04+2) in the final hardened GHCR image. Earlier 18.3 evidence in this manual belongs to the upstream/candidate-image validation before the package-upgrade hardening step; do not reinterpret that historical checkpoint as the final patched image's current minor version. The locale fix is now deployed and live-validated on all three nodes. ulc-01, ulc-02, and ulc-03 each retain /etc/lorawan-cloud/spilo/spilo.env at mode 0600 numeric 0:0, contain exactly one INITDB_LOCALE=C entry, pass docker compose config --quiet, and decode the effective container environment to exactly INITDB_LOCALE = 'C'; every node finished with INITDB_LOCALE live value: PASS. The stray [REDACTED: obtain from authorized secret store].: command not found shown after the ulc-01 PASS occurred at the interactive shell prompt after validation and is unrelated to Spilo or Compose. Spilo remains stopped after the failed bootstrap. The ulc-01 retry-cleanup gate is now **PASS**. The stopped Spilo container was confirmed exited; /srv/spilo/pgdata/pgroot/data was absent; the failed data.failed tree and pg_log were moved intact to /srv/spilo/bootstrap-failures/locale-en_US-20260822-135900/; the active pgroot then contained no data or data.failed state; /service/lorawan-postgres-ha remained empty in etcd; ports 5432/8008 remained free; and Compose still decoded INITDB_LOCALE=C. The archive root is 0700 root:root; its preserved PostgreSQL artifacts retain numeric 101:103 ownership even though Ubuntu displays those IDs as unrelated host account names. The gate ended with ULC-01 RETRY CLEANUP GATE: PASS. The controlled force-recreate retry on ulc-01 then consumed INITDB_LOCALE=C and successfully completed initdb with C.UTF-8, data checksums enabled, and PostgreSQL 18.6. The container remained running with Docker RestartCount=0; Patroni REST became reachable; PostgreSQL accepted connections; actual listeners were private-only at 10.104.0.2:5432 and 10.104.0.2:8008; and runtime SQL confirmed pg_is_in_recovery=false, max_connections=40, shared_buffers=128MB, work_mem=2MB, maintenance_work_mem=32MB, ssl=on, and all four locale settings at C.UTF-8. Active PGDATA is mode 0700 numeric 101:103. This closes the locale/bootstrap and conservative-parameter gates. However, the first immediate Patroni JSON sample reported role: primary together with cluster_unlocked: true; treat that as an unresolved DCS-leader-lock checkpoint rather than declaring the first primary fully validated. The repeated LC_ALL=en_US.utf-8 warnings during post-bootstrap did not prevent database creation or readiness but remain a separate container-locale hygiene issue to clean up after cluster correctness is established. The follow-up primary gate is now **PASS**. Patroni REST no longer reports cluster_unlocked; /leader and /primary both returned HTTP 200; etcd contains /service/lorawan-postgres-ha/{config,initialize,leader,members/ulc-01,status}; the leader key value is exactly ulc-01; and patronictl list shows ulc-01 at 10.104.0.2 as Leader, running, timeline 1. A real PostgreSQL client connection using host=postgres-ha.internal, hostaddr=10.104.0.2, sslmode=verify-full, and the installed CA succeeded with TLS 1.3 / TLS_AES_256_GCM_SHA384; SQL reported server_addr=10.104.0.2/32 and pg_is_in_recovery=false. The recent Patroni log repeatedly reports I am (ulc-01), the leader with the lock. This closes the DCS-lock and runtime TLS gates. One bootstrap log line still requires a bounded check before the first replica is admitted: Patroni 4.1.0 logged User creation is not be supported starting from v4.0.0. Please use "bootstrap.post_bootstrap" script to create users. Because replica cloning depends on the configured standby role and authentication state, verify that the expected standby/admin roles actually exist, that the generated Patroni authentication section points to the intended replication role, and that pg_hba is TLS-only for non-local connections. Keep ulc-02 and ulc-03 stopped until that role/generated-config check passes. The LC_ALL=en_US.utf-8 warning remains a separate locale-hygiene cleanup item, not a current database correctness failure.

6.1 Goal and boundary

Create a three-member PostgreSQL cluster managed exclusively by Patroni and packaged in a reviewed Spilo image.

For this **tiny HA proof of concept**, the same PostgreSQL cluster stores both:

```
chirpstack
lorawan_telemetry
```

lorawan_telemetry is the Timescale-enabled telemetry database. The POC removes only the **separate TimescaleDB server**, not the TimescaleDB feature. The same pinned TimescaleDB extension build must be available on PostgreSQL/Patroni-1, -2, and -3 before any member is considered promotion-eligible.

Use Timescale hypertables for telemetry.uplinks and telemetry.measurements. Keep telemetry.fabric_outbox as an ordinary PostgreSQL table in the same database because it is a transactional work queue rather than time-series storage.

PostgreSQL replication provides availability, not protection against logical deletion. For the POC, keep at least a logical dump before destructive tests; production-grade WAL/object-storage recovery is a later sizing/design step.

Phase 6 activation preflight - run before Section 6.2

Run the following **read-only** block separately on ulc-01, ulc-02, and ulc-03 and retain each complete output. Do not create /srv/spilo, do not write an environment file, and do not start PostgreSQL yet.

```
printf '\n=== HOST ===\n'
hostname
uname -r

printf '\n=== MEMORY / ROOT DISK ===\n'
free -h
df -h /
findmnt /

printf '\n=== DOCKER EVIDENCE GAPS ===\n'
docker version --format 'Server={{.Server.Version}}' 2>/dev/null || docker version
docker compose version
docker info --format 'Storage={{.Driver}} CgroupDriver={{.CgroupDriver}} CgroupVersion={{.CgroupVersion}}
LoggingDriver={{.LoggingDriver}}'

printf '\n=== RUNNING CONTAINERS ===\n'
docker ps --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}'

printf '\n=== CLUSTER NAME RESOLUTION ===\n'
getent hosts ulc-01 ulc-02 ulc-03

printf '\n=== PORT OWNERSHIP BEFORE POSTGRESQL ===\n'
sudo ss -lntp | grep -E ':(2379|2380|5432|8008)\b' || true

printf '\n=== EXISTING POSTGRES/PATRONI PATHS ===\n'
for p in /opt/lorawan/postgres /opt/lorawan/patroni /srv/spilo /etc/lorawan-cloud/spilo; do
  if [ -e "$p" ]; then
    sudo ls -ld "$p"
    sudo find "$p" -mindepth 1 -maxdepth 2 -printf '%M %u:%g %p\n' 2>/dev/null | head -n 80
  else
    printf 'ABSENT %s\n' "$p"
  fi
done

printf '\n=== ETCD HEALTH FROM THIS NODE ===\n'
docker exec etcd etcdctl \
  --endpoints=http://10.104.0.2:2379,http://10.104.0.4:2379,http://10.104.0.8:2379 \
  endpoint health
```

Expected before we continue:

- each host still reports the accepted Ubuntu/kernel and roughly 2-GiB/50-GiB baseline;
- Docker/Compose work and the exact Compose version plus logging driver are finally captured;
- only the existing etcd container is expected from the currently completed clustered layer;
- 2379/2380 belong to etcd on the private/east-west addresses;
- 5432 and 8008 must not already be occupied by an unidentified PostgreSQL/Patroni process;
- any existing PostgreSQL/Patroni directory is inspected rather than deleted;
- all three etcd endpoints report healthy from every database host.

Stop after this block and review all three outputs. The next step is image/version selection and container-user discovery. We do not choose storage ownership or initialize a PostgreSQL data directory until the image tells us its real UID/GID and supported PostgreSQL/TimescaleDB combination.

Why: the first PostgreSQL bootstrap writes a database cluster and Patroni identity. Discovering an occupied port, stale data directory, wrong Docker runtime setting, or incompatible image after that point creates a destructive recovery problem that is easy to avoid now.

Preflight result - 2026-08-21

The read-only preflight passed on all three active hosts.

```
ulc-01  kernel 6.8.0-138-generic  RAM 1.9 GiB  root free ~45 GiB
ulc-02  kernel 6.8.0-138-generic  RAM 1.9 GiB  root free ~45 GiB
ulc-03  kernel 6.8.0-138-generic  RAM 1.9 GiB  root free ~45 GiB

all three:
  Docker Engine 29.7.2
  Docker Compose v5.5.0
  storage driver overlayfs
  cgroup driver systemd
  cgroup v2
  default Docker logging driver json-file
  no swap
  only the validated etcd container running
  PostgreSQL 5432 not listening
  Patroni REST 8008 not listening
  /opt/lorawan/postgres exists and is empty
  /opt/lorawan/patroni exists and is empty
  /srv/spilo absent
  /etc/lorawan-cloud/spilo absent
  all three etcd endpoints healthy from the host
```

Observed memory at the checkpoint was approximately 394-417 MiB used with about 1.5 GiB available on each node. This is a baseline observation only, not a guarantee that the final full-feature stack will fit; continue measuring after every added service.

The default Docker logging driver is now proven to be `json-file`, not the planned rotating `local` default. Do not rewrite history and claim host-wide log rotation is already configured. Before Spilo is started, either give the Spilo service an explicit bounded logging policy in Compose or perform a separately controlled Docker-daemon logging change that does not jeopardize the healthy etcd layer.

Name-resolution detail discovered during preflight: each host's own hostname resolves through both the standard Ubuntu 127.0.1.1 entry and its documented 10.104.0.x east-west entry. Therefore Patroni/PostgreSQL must use explicit 10.104.0.2, 10.104.0.4, and 10.104.0.8 bind/connect/advertise addresses. Do **not** rely on hostname resolution to choose `postgresql.connect_address` or `restapi.connect_address`, because a loopback address advertised into the DCS would make remote members/HAProxy unable to reach that node.

Preflight decision: **PASS**. Proceed to image/version selection only. No storage initialization or Patroni bootstrap is authorized yet.

Pinned upstream source checkpoint - 2026-08-21

The upstream trigger branch was queried from ulc-03 with Git 2.43.0 and returned:

```
95139b4de7a33aec1f788ad7bb863c92edbe2ee8 refs/heads/trigger
```

For the remainder of this review, 95139b4de7a33aec1f788ad7bb863c92edbe2ee8 is the **candidate Spilo source commit**. Do not replace it with whatever trigger points to later without repeating the compatibility/security review and recording the change.

The exact commit was then fetched into /home/opsadmin/spilo-review-95139b4de7a3 on ulc-03 and checked out detached. Verification returned the same SHA exactly. Captured metadata:

```
commit=95139b4de7a33aec1f788ad7bb863c92edbe2ee8
commit_date=2026-07-22T15:27:42+02:00
subject=inject IRSA credentials into wal-g envdir for standby and clone clusters (#1206)
worktree=HEAD (no branch)
```

Source-fetch decision: **PASS**. We have the exact source we intended to inspect.

Source-inspection checkpoint - 2026-08-21

Inspection of this exact checkout proved the following source defaults/build inputs:

```
base image reference: ubuntu:22.04
PGVERSION default: 18
PGOLDVERSIONS: 14 15 16 17
Patroni version: 4.1.3
TIMESCALEDB_APACHE_ONLY default: true
TIMESCALEDB_TOOLKIT default: true
ETCD3_HOSTS: supported by the generic DCS parser; required for our etcd 3.5 cluster
ETCD_HOSTS: legacy etcd path also supported, but not for this deployment
RESTAPI_CONNECT_ADDRESS: supported
PG_CONNECT_ADDRESS: supported through explicit placeholder override
```

These are **source facts, not deployment approvals**. In particular, PostgreSQL 18 is only the Dockerfile default at this checkpoint; the project has not yet selected a PostgreSQL major.

The detailed package inspection confirmed that the pinned source is **not fully reproducible as-is**. TimescaleDB is installed by package name (timescaledb-2-oss-postgresql- $\{version\}$ when TIMESCALEDB_APACHE_ONLY=true) from the live Timescale package repository with no exact package version in the install command. The same Git SHA can therefore resolve a different TimescaleDB package if the repository changes. protobuf is also installed without a version, pg_view is installed from the moving upstream master branch, and the base image is the mutable tag ubuntu:22.04. These are build-reproducibility debts that must be resolved either by using and pinning a reviewed upstream image digest or by making a controlled reproducible-build patch before approval.

TIMESCALEDB_TOOLKIT=true does **not** mean the Toolkit is installed in the current default build. The install branch runs Toolkit only when TIMESCALEDB_APACHE_ONLY is not true. With the observed defaults (TIMESCALEDB_APACHE_ONLY=true, TIMESCALEDB_TOOLKIT=true), the build selects the OSS TimescaleDB package and skips the Toolkit install branch. The project currently requires core TimescaleDB hypertables, not Toolkit-specific features, so this is not by itself a deployment failure; it must nevertheless be recorded accurately.

Patroni itself is explicitly constrained to 4.1.3, and the DCS compatibility check is now **PASS** for our etcd 3.5.15 cluster. The pinned source declares etcd3 as a supported Patroni DCS and its generic environment parser accepts

prefixes in the form `ETCD3_*`. Therefore this deployment must use `ETCD3_HOSTS`, which generates Patroni's `etcd3:` configuration. Do **not** use `ETCD_HOSTS` here: that prefix maps to the separate legacy `etcd:` configuration path.

The source also does not hard-code the PostgreSQL container UID/GID in the inspected Dockerfile/build scripts; it relies on the `postgres` account created by installed packages and later `chowns $PGHOME/$RW_DIR`. Therefore storage ownership must be discovered from the actual reviewed image with `id postgres` before creating the host persistence path. For the selected image the final native host path is `/srv/spilo/pgdata`, mounted at `/home/postgres/pgdata`.

This SHA pins source only. It does **not** yet approve a PostgreSQL major, exact TimescaleDB build, image tag, image digest, container UID/GID, or deployment.

6.2 Select a compatible Spilo image

Spilo combines PostgreSQL and Patroni. Its maintainers recommend building reviewed images from source because public images are not released on a regular cadence.

Identify the Spilo source commit, PostgreSQL major/minor, Patroni version, **TimescaleDB extension version/build**, build tag, registry, immutable image digest, base-image digest, successful build/scan result, and rollback digest. Keep them with the database-volume and backup references because a PostgreSQL data directory can be opened only by a compatible major version and a failed rollout must return every member to a known image.

Stop here. Do not bootstrap from `latest`, an unscanned image, or an image whose PostgreSQL major version does not match the data-volume lifecycle.

Upstream selection note - 2026-08-21

A current upstream check on 2026-08-21 shows the Zalando Postgres Operator defaulting to `ghcr.io/zalando/spilo-18:4.1-p2`. Therefore the older `ghcr.io/zalando/spilo-17:4.0-p3` reference is now historical comparison only. `spilo-18:4.1-p2` is the better current binary comparison candidate, but it is not approved for our non-Kubernetes deployment until its immutable digest and actual contents are inspected.

Therefore the deployment decision is:

```
spilo-17:4.0-p3 -> older reference only
spilo-18:4.1-p2 -> current upstream comparison candidate; exact amd64 manifest pulled on ulc-03 for
inspection only
OCI index digest -> sha256:258a87d34699387f3b6b45d30874c21c6b838ed51c9371ae2a70151d57137990
linux/amd64 manifest -> sha256:cf11c4e237777b03d9867bf53ae33a77980e1826c07d7c053de034dce695392
linux/arm64 manifest -> sha256:7c3313a3860f2a95a24f165e4172927fceb693c9a8be25582fac6a9bde8a9bf4
pinned source 95139b4... -> source-review checkpoint
approved deployment digest -> NOT YET SELECTED; pulled digest passed isolated content inspection, but
published-image etcd3/runtime checks still remain
```

The unknown/unknown entries in this OCI index are attestation manifests associated with the runnable architecture manifests; do not treat them as additional database platforms. The active Droplets are x86-64, so the `linux/amd64` manifest digest is the relevant runnable candidate.

Manifest inspection is **PASS as a discovery checkpoint only**. The exact `linux/amd64` digest was then pulled on `ulc-03` without starting a Spilo container. The pull reported the expected digest and left the running-container set unchanged (`etcd` only). The image reports size 628727591 bytes; host root usage increased from about 2.8G to 4.9G because downloaded/container-image storage includes unpacked layers and metadata, not only the logical image-size field.

Metadata inspection found an important source-vs-binary difference: this published image exposes `PATRONIVERSION=4.1.0`, whereas the separately pinned 2026-07-22 source commit exposes `PATRONIVERSION=4.1.3`.

Therefore **do not infer published-image contents from the later source checkout**. Treat them as separate artifacts.

The exact amd64 image then passed an isolated content inspection with `--network none`, a read-only root filesystem, all capabilities dropped, `no-new-privileges`, no persistent mounts, and `/bin/bash` overriding the normal Spilo entrypoint. Observed contents were Ubuntu 22.04.5 LTS, PostgreSQL 18.3, Patroni 4.1.0, postgres UID 101/GID 103, TimescaleDB OSS 2.26.2 for PostgreSQL 18, no TimescaleDB Toolkit control file, `PGRROOT=/home/postgres/pgdata/pgroot`, and `PGDATA=/home/postgres/pgdata/pgroot/data`. The running-container set stayed unchanged with etcd only.

The published image's own Patroni etcd3 module was then verified successfully: Patroni reports 4.1.0, Python reports package version 4.1.0, `patroni.dcs.etcd3` imports successfully, and `/scripts/configure_spilo.py` lists etcd3 in `PATRONI_DCS` plus the generic `ETCD3_*` parser. DCS compatibility for this exact image is therefore **PASS**.

A second disposable test initialized PostgreSQL 18.3 entirely under `/tmp`, started it with `shared_preload_libraries=timescaledb`, created TimescaleDB 2.26.2 successfully, and left the live etcd container unchanged. During extension startup an internal TimescaleDB background job logged functionality not supported under the current "apache" license; this is a capability boundary of the Apache-only build and must not be ignored. The first hypertable statement then failed only because the test command lost SQL string quoting and sent `create_hypertable(sensor_probe, by_range(ts))` instead of quoted names. That attempt was a **test-command defect, not a TimescaleDB failure**.

The corrected test was rebuilt as `/home/opsadmin/spilo-hypertable-test.sh`, syntax-checked, SHA-256 verified as `30716e0483f2fcd8a33b6247c2ef2d6065eba8d6595aed590cd7714f185feb5c`, and executed through `docker run --rm -i` against the exact immutable amd64 digest with `--network none`, read-only root filesystem, no persistent mounts, UID/GID 101:103, and disposable `/tmp PGDATA`. PostgreSQL 18.3 started successfully; `CREATE EXTENSION timescaledb` succeeded; `create_hypertable('sensor_probe', by_range('ts'))` returned `(1,t)`; an inserted test-device row with value 42.5 was read back; `timescaledb_information.hypertables` reported `public.sensor_probe` with one dimension; `pg_extension` reported TimescaleDB 2.26.2; clean shutdown completed; and the wrapper reported `TEST_RC=0`. The live service inventory before and after remained the existing etcd container only. **Core PostgreSQL + TimescaleDB hypertable capability: PASS**.

A dedicated, syntax-checked retention probe was then executed against the same immutable amd64 digest. PostgreSQL 18.3 started, TimescaleDB 2.26.2 loaded, and the disposable hypertable was created successfully. `SELECT add_retention_policy('sensor_probe', INTERVAL '30 days');` returned `RETENTION_RC=1` with the server error function "add_retention_policy" is not supported under the current "apache" license. The wrapper itself returned `TEST_RC=0` by design and the live etcd container remained unchanged. **Retention-policy automation is therefore confirmed unavailable in this Apache-only image**. This is an optional-feature capability boundary, not a core TimescaleDB failure.

A subsequent read-only inspection of the exact immutable image confirmed that TimescaleDB 2.26.2 contains SQL definitions for `compress_chunk`, `decompress_chunk`, `add_compression_policy`, `remove_compression_policy`, and `add_columnstore_policy`, together with the TimescaleDB 2.26.2 native shared object. This proved **API presence only**, so a dedicated runtime probe was executed next.

The verified compression probe created a one-day `sensor_probe` hypertable and three chunks successfully, then attempted `ALTER TABLE ... SET (timescaledb.compress, timescaledb.compress_segmentby = 'device_id')`. The server rejected that operation under the current Apache license and returned `ENABLE_COMPRESSION_RC=1`. Because compression could not be enabled, the probe correctly skipped the manual `compress_chunk()` step. A separate `add_compression_policy('sensor_probe', INTERVAL '7 days')` call was also rejected under the Apache license with `COMPRESSION_POLICY_RC=1`. The wrapper returned `TEST_RC=0` by design and the live etcd container remained unchanged. **Compression and compression-policy automation are therefore confirmed unavailable in this Apache-only image**.

Image decision: this exact linux/amd64 digest is accepted as the **functional candidate for the narrow HA POC** because PostgreSQL 18.3, Patroni 4.1.0 with etcd3, TimescaleDB 2.26.2, extension loading, hypertable creation,

and telemetry insert/read have all passed, while the POC intentionally leaves destructive retention and compression disabled. It is **not** a full-feature TimescaleDB image for the wider integration profile: `add_retention_policy()`, `timescaledb.compress`, and `add_compression_policy()` are unavailable under its Apache-only build. Final deployment approval still requires the separate exact-image security/provenance gate. Until that gate is closed, do not create persistent Spilo storage and do not connect Patroni to live etcd.

The scanner-availability check on `u1c-03` found no Docker Scout command and no installed Trivy, Gype, Syft, or Cosign binaries. Do not add a scanner package to the production host merely to close this gate. Use a reviewed official scanner image pinned by digest, avoid exposing the Docker daemon socket to it, and scan this exact Spilo digest directly from the registry. The scan host/container may use network access for the vulnerability database and registry layers, but it must receive no Spilo secrets, no PostgreSQL mounts, and no etcd credentials.

The first formatted `docker image inspect` command failed only because `.Config.User` was absent from the image config map. The isolated container later proved that an overridden entrypoint starts as root and that the internal postgres account is UID 101/GID 103. Therefore persistent PostgreSQL paths must be owned for UID 101/GID 103, not inferred from the container's initial root identity.

Do not infer the TimescaleDB version from the newest standalone TimescaleDB release. The version bundled by Spilo is determined by the selected image/build, so inspect the exact immutable amd64 image before choosing the PostgreSQL major or creating data.

6.2.1 Security hardening and vulnerability scan checkpoint - 2026-08-21

The selected Spilo image was hardened through a controlled image patch process. The goal was not to replace the HA design, but to reduce known vulnerabilities before PostgreSQL/Patroni deployment.

Final test image:

```
spilo-18-walg309-ospatched:test
```

Changes applied:

```
Base image:
ghcr.io/zalando/spilo-18@sha256:cf4d11c4e23777b03d9867bf53ae33a77980e1826c07d7c053de034dce695392

OS packages:
Ubuntu package upgrade performed

WAL-G:
v3.0.8 -> v3.0.9

WAL-G binary:
wal-g-pg-22.04-amd64
PostgreSQL build
libsodium enabled
```

Security scan tool:

```
Trivy
scanner:
vulnerability scanner only
```

Results:

Before hardening:

```
CRITICAL  4
HIGH      41
```

MEDIUM	459
LOW	69

After hardening:

CRITICAL	0
HIGH	9
MEDIUM	51
LOW	20

Applied vulnerability fixes

The following vulnerabilities were reduced through the Ubuntu package upgrade:

```
OpenSSL:
  updated from
  3.0.2-0ubuntu1.23

  to
  3.0.2-0ubuntu1.26

rsync:
  updated from
  3.2.7-0ubuntu0.22.04.4

  to
  3.2.7-0ubuntu0.22.04.7
```

WAL-G CVE review

Trivy reported:

```
CVE-2021-38599
Package:
github.com/wal-g/wal-g
```

Disposition:

NOT APPLICABLE

Verification evidence:

```
wal-g version:
v3.0.9 3e49318

Build tags:
-tags=brotli,libsodium,lzo

Binary inspection:
libsodium.Writer
libsodium.Reader
libsodium.Crypter
```

Reason:

The CVE affects WAL-G versions before 1.1 when a non-libsodium build ignores encryption keys. The selected WAL-G v3.0.9 PostgreSQL binary contains libsodium support and does not match the vulnerable condition.

Remaining HIGH findings

Remaining HIGH findings originate from the Go runtime embedded inside the WAL-G static binary:

```
Go runtime:
v1.25.11
```

These are accepted for this deployment checkpoint.

Reason:

Removing these findings requires rebuilding WAL-G using a newer Go toolchain. This creates a custom WAL-G maintenance path. The current deployment prioritizes using the reviewed upstream WAL-G release with verified encryption support and stable operational behaviour.

Security acceptance decision

APPROVED FOR CURRENT HA POC

Included:

- Spilo image
- Ubuntu security updates
- WAL-G v3.0.9
- libsodium-enabled WAL-G encryption support

Accepted residual:

- Go stdlib vulnerability findings inside WAL-G binary

Reason:

- embedded dependency
- requires custom rebuild
- no evidence of direct deployment exposure in current usage

This checkpoint must be repeated if the Spilo base digest, WAL-G binary, PostgreSQL major version, or build process changes.

Patched image artifact and private registry publication

The completed image was exported as a portable Docker artifact after validation.

The selected production distribution method is GitHub Container Registry (GHCR) using a private container repository.

```
Artifact:
spilo-18-walg309-ospatched.tar
```

```
Size:
1.3G
```

```
Local image digest:
sha256:6bf45913616f2e524555973bfdd34bae1607a709dc3548ab25c8b32a454a9519
```

Purpose:

The tar archive provides a rollback and transfer point.
It allows the validated Spilo image to be imported on another HA node without rebuilding the image again.

Restore command:

```
docker load -i spilo-18-walg309-ospatched.tar
```

Private GHCR publication checkpoint - 2026-08-22

The validated patched image has now been published to the selected private GitHub Container Registry repository and independently pulled/inspected on all three HA nodes.

```
Private registry tag:
ghcr.io/jervis-org/spilo/spilo-18-walg309-ospatched:v1

Immutable deployment digest:
sha256:6bf45913616f2e524555973bfdd34bae1607a709dc3548ab25c8b32a454a9519

Registry index:
application/vnd.oci.image.index.v1+json

Runnable platform:
linux/amd64

Runnable manifest:
sha256:e415ee43994e79b42542479a7cda97b33f487289948cfcfae3628f22ff817a3f

Additional unknown/unknown manifest:
OCI attestation manifest associated with the linux/amd64 image; not another database runtime platform
```

The registry index digest exactly matches the validated local image digest. Pull/inspect verification on `u1c-01`, `u1c-02`, and `u1c-03` returned the same GHCR repository digest. **Private-registry distribution checkpoint: PASS.**

Use the immutable reference for deployment, not the mutable `v1` tag:

```
ghcr.io/jervis-org/spilo/spilo-18-walg309-
ospatched@sha256:6bf45913616f2e524555973bfdd34bae1607a709dc3548ab25c8b32a454a9519
```

Keep `spilo-18-walg309-ospatched.tar` as an offline rollback/transfer artifact, but GHCR is now the normal distribution path. Registry credentials are node-local secrets and must never be committed to this repository.

Persistent-storage ownership checkpoint - 2026-08-22

The selected image reports:

```
postgres UID = 101
postgres GID = 103
```

`/srv/spilo/pgroot` was originally prepared on all three nodes with mode `0700` and numeric ownership `101:103`. On Ubuntu the host may display those numeric IDs using unrelated local account names such as `messagebus/uuidd`; verification must therefore use `ls -ldn`, not host account names. A later exact-image probe proved the deployment should instead mount `/srv/spilo/pgdata` at `/home/postgres/pgdata`. On 2026-08-22, all three nodes returned `PGRROOT EMPTY - PASS`; the empty superseded directory was then removed with `rmdir` and replaced by `/srv/spilo/pgdata` with mode `0700` and numeric ownership `101:103` on `u1c-01`, `u1c-02`, and `u1c-03`. Native host-storage migration: **PASS.**

Environment-file pre-bootstrap checkpoint - 2026-08-22

`/etc/lorawan-cloud/spilo/spilo.env` exists on all three nodes with numeric ownership `0:0` and mode `0600`. On 2026-08-22, all three files were updated and verified with the tested native path pair `PGRROOT=/home/postgres/pgdata/pgroot` and `PGDATA=/home/postgres/pgdata/pgroot/data`. `SCOPE=lorawan-postgres-ha` and `PGVERSION=18` remain common. The earlier literal `ALLOW_NOSSL=false` value is rejected: exact-image Mustache testing proved any non-empty string is truthy, so the TLS-only representation must be the explicit

empty value `ALLOW_NOSSL=`. The node-specific connect addresses remain correct: `ulc-01` uses `10.104.0.2:5432/8008`, `ulc-02` uses `10.104.0.4:5432/8008`, and `ulc-03` uses `10.104.0.8:5432/8008`. Environment-path migration: **PASS**. After correcting `ETCD3_HOSTS` to the Compose-safe outer-single-quoted form, all three nodes returned Compose syntax: `PASS`; `docker compose config --format json` also showed that the value handed to the service is exactly `"10.104.0.2:2379", "10.104.0.4:2379", "10.104.0.8:2379"`. Compose dotenv parsing is therefore **PASS on all three nodes**. Exact-image source inspection confirmed that `ETCD3_HOSTS` enters DCS `etcd3` parameter `hosts`. For `hosts`, Spilo does not call `yaml.safe_load()` directly on this string: when it does not start with `-` and contains no `[`, the code first wraps it as `[{0}]`. A direct decoder test therefore correctly showed the current unwrapped string alone raises `ParserError`, while the bracketed form decodes to exactly three strings. The first isolated `get_dcs_config()` call failed with `KeyError: 'NAMESPACE'` because the test dictionary omitted a placeholder that normal Spilo initialization supplies before this function is called. The corrected read-only call supplied `NAMESPACE=default` and the same Compose-decoded host string. The exact image returned `{'etcd3': {'hosts': ['10.104.0.2:2379', '10.104.0.4:2379', '10.104.0.8:2379']}}`, with `hosts` type `list`, count 3, and each endpoint exactly matching the east-west `etcd` members. `ETCD3_HOSTS` end-to-end parsing is therefore **PASS** and the deployed environment value must remain unchanged. A non-root `ls/cat` may report `Permission denied` because the parent configuration directory is intentionally root-only; use `sudo` for verification.

Do not start Spilo yet. The exposed first-draft role passwords were replaced, and one shared secret per role is now consistent across all three members. The cluster scope currently deployed in the node environment files is standardized as `SCOPE=lorawan-postgres-ha`; do not change that after bootstrap without an explicit Patroni/DCS migration plan.

The only values that should differ between node environment files at this stage are node-specific advertise/connect addresses. Exact-image source inspection found no `PATRONI_NAME` reference. Instead, the Patroni template renders name: `'{{{instance_data.id}}}'`, and `get_instance_metadata()` initializes `instance_data.id` from `socket.gethostname()`. An unpinned Docker hostname was observed as transient container ID `f895bcfe6e37`, so that default is **not safe** for Patroni membership. The deterministic fix has now been proven against the exact immutable image: a network-isolated disposable probe with `--hostname ulc-03` and `SPILO_PROVIDER=local` returned `provider = local`, `socket.gethostname() = ulc-03`, and `instance_data.id = ulc-03`. The probe monkey-patched only `getaddrinfo()` because `--network none` intentionally prevents hostname-to-IP resolution; the member-name path itself remained the real `socket.gethostname()` call. Therefore final Compose must set `hostname`: explicitly to `ulc-01`, `ulc-02`, or `ulc-03`, and the container environment must set `SPILO_PROVIDER=local`. Do not add `PATRONI_NAME`. `SCOPE=lorawan-postgres-ha`, PostgreSQL major, `etcd` endpoint list, role names, and role passwords remain cluster-consistent.

TLS variable support is confirmed against the exact patched GHCR digest: `configure_spilo.py` consumes `SSL_CA_FILE`, `SSL_CERTIFICATE_FILE`, `SSL_PRIVATE_KEY_FILE`, `ALLOW_NOSSL`, `PG_CONNECT_ADDRESS`, and `RESTAPI_CONNECT_ADDRESS`. Its certificate loader first checks whether `SSL_PRIVATE_KEY_FILE` already exists; when a pre-provisioned key exists and configuration is not forced, it returns without overwriting the files or running its later `chmod/owner` adjustment. Therefore this build can use the planned read-only certificate bind mount.

Installed TLS state is now **PASS on all three nodes**. `/etc/lorawan-pki/postgres` is mode `0750` numeric owner/group `0:103`; `ca.crt` and `server.crt` are `0644 0:0`; `server.key` is `0600 101:103`. The CA SHA-256 fingerprint is identical on all members:

`99:00:4B:B3:2D:7D:78:FA:38:61:7C:78:89:6D:7A:7E:FF:9F:A6:10:FC:8F:07:D4:E2:5E:35:25:36:E6:CB:3E`. On each node, CA-chain verification, node-hostname verification, `postgres-ha.internal` verification, private-IP verification, SAN inspection, and installed key/certificate public-key matching all passed. For TLS-only policy, use `ALLOW_NOSSL=` as an explicit empty value. Do **not** write `ALLOW_NOSSL=false`: the exact image treats that non-empty string as true and would render the non-SSL-allowed branch. On 2026-08-22 the operator corrected this on `ulc-01`, `ulc-02`, and `ulc-03`; every node reported `count=1`, Compose syntax: `PASS`, effective `ALLOW_NOSSL = ''`, and `ALLOW_NOSSL` empty-string policy: `PASS`. `ALLOW_NOSSL` deployment gate: **PASS**.

6.3 Build and inspect the image

Use a controlled CI runner. Illustrative flow:

```
git clone https://github.com/zalando/spilo.git
cd spilo
git checkout <APPROVED_SPILO_COMMIT>
git status --short --branch
git show --stat --oneline HEAD
```

Inspect the Dockerfile, build scripts, installed PostgreSQL/Patroni versions, entrypoint, default users, environment settings, and the selected commit's TimescaleDB build arguments. Spilo supports TimescaleDB in its image build, but the **exact supported PostgreSQL/TimescaleDB combination is commit-dependent**; do not infer it from an old release note.

Build from the documented postgres-appliance directory for the selected commit. A typical build keeps TimescaleDB enabled and selects only the PostgreSQL major deliberately:

```
cd postgres-appliance
docker build \
  --build-arg PGVERSION=<APPROVED_POSTGRES_MAJOR> \
  --tag <REGISTRY>/<PROJECT>/spilo:<VERSIONED_TAG> .
```

If the selected commit exposes TIMESCALEDB_APACHE_ONLY, TIMESCALEDB_TOOLKIT, or other Timescale-related build args, record their values in the software worksheet. Do not change license/feature variants accidentally between nodes.

Before pushing the image, prove the Timescale control/library files exist inside **this exact build** using paths discovered from the image rather than assumed host paths:

```
docker run --rm --entrypoint bash \
  <REGISTRY>/<PROJECT>/spilo:<VERSIONED_TAG> \
  -lc 'set -e; pg_config --version; \
    find /usr/share/postgresql -path "*/extension/timescaledb.control" -print; \
    find /usr/lib/postgresql -name "timescaledb*.so" -print'
```

The command must print the PostgreSQL version plus TimescaleDB extension/library files for the intended major. **Stop here** if it prints no TimescaleDB control/library files or the PostgreSQL major is wrong.

Run tests and scan the image. Push it, obtain the repository digest, sign it when the organization supports image signing, and deploy **the same digest** on all three members.

6.4 Host storage preparation

The minimum test profile uses each Droplet's included SSD; it does **not** require a separate block volume. First verify the root filesystem and free space:

```
lsblk -o NAME,SIZE,FSTYPE,UUID,MOUNTPOINTS
df -h /
findmnt /
```

Keep PostgreSQL data under a dedicated host path such as `/srv/spilo` on that node's own root SSD. If a block volume is added later because measurements require it, verify the exact device and persistent mount before moving data. Example directory layout:

```
sudo install -d -m 700 -o 101 -g 103 /srv/spilo/pgdata
sudo install -d -m 700 -o root -g root /etc/lorawan-cloud/spilo
sudo install -d -m 750 -o root -g 103 /etc/lorawan-pki/postgres
```

Container UID/GID values depend on the built image. Inspect them and use the exact values; do not copy the example ownership blindly.

Verify the selected filesystem survives reboot and the `/srv/spilo` path has the expected ownership before writing PostgreSQL data.

6.5 Protect the environment file

Create `/etc/lorawan-cloud/spilo/spilo.env` mode `600`. Required categories include:

```
SCOPE=<PG_SCOPE>
PGVERSION=<APPROVED_POSTGRES_MAJOR>
PGROOT=/home/postgres/pgdata/pgroot
PGDATA=/home/postgres/pgdata/pgroot/data

# Keep PGROOT and PGDATA as this exact tested pair. Overriding only PGROOT leaves the image-baked PGDATA
unchanged.

# Set these per host. Never let hostname auto-discovery choose 127.0.1.1.
PG_CONNECT_ADDRESS=<THIS_NODE_10.104_IP>:5432
RESTAPI_CONNECT_ADDRESS=<THIS_NODE_10.104_IP>:8008

# Current tested etcd endpoints on the east-west network.
# This file is parsed by Docker Compose dotenv rules. The outer single quotes keep the
# complete host string intact; Spilo auto-wraps it in [...] before YAML decoding.
ETCD3_HOSTS="'10.104.0.2:2379','10.104.0.4:2379','10.104.0.8:2379'"

PGUSER_SUPERUSER=postgres
PGPASSWORD_SUPERUSER=<LOAD_FROM_SECRET_STORE>
PGUSER_STANDBY=standby
PGPASSWORD_STANDBY=<LOAD_FROM_SECRET_STORE>
PGUSER_ADMIN=<NAMED_ADMIN_ROLE>
PGPASSWORD_ADMIN=<LOAD_FROM_SECRET_STORE>

SSL_CA_FILE=/run/postgres-certs/ca.crt
SSL_CERTIFICATE_FILE=/run/postgres-certs/server.crt
SSL_PRIVATE_KEY_FILE=/run/postgres-certs/server.key
# Empty is intentional: Spilo Mustache treats any non-empty string, including "false", as true.
ALLOW_NOSSL=
```

The minimal HA POC intentionally does **not** require WAL-G, object-storage endpoints, or cloud backup access keys. Do not populate fake `WALG_*` / `AWS_*` values merely because a production Spilo example contains them. Add those variables later only when the production backup/PITR profile is intentionally enabled and tested.

Confirm exact variable names against the checked-out Spilo `ENVIRONMENT.rst`, Patroni version, and built entrypoint before writing the final environment file. The etcd cluster currently uses HTTP only on `10.104.0.0/20`; do not add `ETCD_CACERT`, `ETCD_CERT`, or `ETCD_KEY` placeholders unless etcd TLS has actually been deployed and tested. Spilo documents insecure example default passwords; override every credential explicitly.

Use one identical `SCOPE` and etcd endpoint list on all members. Use unique member identity/hostname values where the image requires them.

6.6 Container definition

Use the exact immutable image reference and an explicit per-node hostname. The example below shows `u1c-01`; use `u1c-02` and `u1c-03` on the other nodes.

```

services:
  spilo:
    image: ghcr.io/jeravis-org/spilo/spilo-18-walg309-
ospatched@sha256:6bf45913616f2e524555973bfdd34bae1607a709dc3548ab25c8b32a454a9519
    container_name: spilo
    hostname: ulc-01
    network_mode: host
    restart: unless-stopped
    env_file:
      - /etc/lorawan-cloud/spilo/spilo.env
    volumes:
      - /srv/spilo/pgdata:/home/postgres/pgdata
      - /etc/lorawan-pki/postgres:/run/postgres-certs:ro
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "5"
    stop_grace_period: 2m

```

The protected `spilo.env` must include `SPILO_PROVIDER=local` plus the verified TLS path variables `SSL_CA_FILE=/run/postgres-certs/ca.crt`, `SSL_CERTIFICATE_FILE=/run/postgres-certs/server.crt`, and `SSL_PRIVATE_KEY_FILE=/run/postgres-certs/server.key`. **Do not assume** `PG_CONNECT_ADDRESS` or `RESTAPI_CONNECT_ADDRESS` restrict socket binding: exact-image inspection proves the template renders PostgreSQL `listen: '*:{{PGPORT}}'` and Patroni REST `listen: ':{{APIPORT}}'`, while the two `*_CONNECT_ADDRESS` values are separate advertised addresses. Under `network_mode: host`, those wildcard listeners cover all host interfaces, including the public NIC. Exact-image source inspection proves `SPILO_CONFIGURATION` is the supported precedence path: Spilo YAML-decodes it, copies it, then calls `deep_update(user_config_copy, config)`. The merge function keeps user-supplied scalar/list values and recursively fills unspecified dictionary keys from the generated configuration. Therefore per-node `restapi.listen` and `postgresql.listen` overrides can replace only the wildcard listener values without replacing the rest of the Patroni configuration. The first isolated render probe failed at an unrelated diagnostic assumption, `merged["name"]`, with `KeyError: 'name'`; the corrected probe then proved the actual template places the member identity at `postgresql.name: '{{instance_data.id}}'`, while `instance_data.id` is `ulc-03`. More importantly, the corrected exact-image merge printed and asserted `restapi.listen = 10.104.0.8:8008`, `restapi.connect_address = 10.104.0.8:8008`, `postgresql.listen = 10.104.0.8:5432`, `postgresql.connect_address = 10.104.0.8:5432`, and the expected three-member `etcd3.hosts` list, finishing with `LISTENER OVERRIDE RENDER: PASS`. The listener override mechanism and final in-memory merge are therefore proven. The exact Compose dotenv representation is proven: write the compact JSON as one outer-single-quoted value, for example `SPILO_CONFIGURATION='{"restapi":{"listen":"10.104.0.8:8008"},"postgresql":{"listen":"10.104.0.8:5432"}}'` on `ulc-03`, with the equivalent private address on the other nodes. A temporary Compose file decoded this back to the identical JSON and finished with `SPILO_CONFIGURATION` dotenv encoding: `PASS`. This is now applied per node after mapping physical hostname to its intended `10.104.0.x` address. All three live Compose configurations decoded to the expected private-only listener pair and returned `LIVE LISTENER OVERRIDE: PASS`, so the wildcard-listener blocker is closed. The bounded `json-file` settings prevent an unhealthy container from consuming the small 50-GiB root disk with unbounded Docker logs.

Validate without printing expanded secrets:

```

sudo docker compose -f /etc/lorawan-cloud/spilo/compose.yml config --quiet
sudo docker pull <PINNED_IMAGE_REFERENCE>
sudo docker image inspect <PINNED_IMAGE_REFERENCE> --format '{{json .RepoDigests}}'

```

6.7 Bootstrap order

1. Prove the current etcd quorum and `10.104.0.x:2379` reachability from every database node. The present etcd deployment is HTTP on the east-west network; do not assume TLS exists.
2. Confirm all three Spilo data directories are empty for a new cluster.

3. Start PostgreSQL/Patroni on ha-01 and watch it acquire the initial leader lock and initialize PostgreSQL.
4. Verify PostgreSQL and Patroni before starting the next node.
5. Start PostgreSQL/Patroni on ha-02; wait until it is a streaming replica.
6. Start PostgreSQL/Patroni on ha-03; wait until it is a streaming replica.
7. Do not start all nodes blindly and ignore conflicting bootstrap logs.

```
sudo docker compose -f /etc/lorawan-cloud/spilo/compose.yml up -d
sudo docker logs --since=10m --tail=300 spilo
curl --cacert <PATRONI_CA_IF_ENABLED> https://<DB_PRIVATE_IP>:8008/patroni
```

Patroni REST protection depends on the pinned configuration. Keep the API private and use TLS/authentication where supported.

6.8 Cluster inspection

Use `patronictl` from a protected administrative environment with the exact DCS configuration used by the pinned Patroni build:

```
patronictl -c <PATRONI_CONFIG> list <PG_SCOPE> --extended
```

Expected:

- one primary with leader lock;
- two streaming replicas;
- one timeline;
- no pending restart unless planned;
- acceptable receive/replay lag;
- distinct member names and private addresses.

Database checks:

```
SELECT pg_is_in_recovery();
SELECT current_setting('server_version');
SELECT application_name, client_addr, state, sync_state,
       write_lag, flush_lag, replay_lag
FROM pg_stat_replication
ORDER BY application_name;
```

Run the replication query on the primary.

6.9 Patroni dynamic configuration

For this 2-GiB POC, do not wait until after bootstrap to introduce the conservative memory/connection values. Exact-image inspection shows Spilo auto-tunes local `shared_buffers` to roughly one quarter of detected memory and auto-tunes `max_connections` with a floor of 100. The bootstrap DCS template contains `postgresql.parameters.max_connections`, while the local PostgreSQL section contains `shared_buffers`. To make the first cluster start deterministic, place the same four conservative values in both the bootstrap DCS parameters and the local PostgreSQL parameters through the already-proven `SPILO_CONFIGURATION` merge. The exact-image render test is now **PASS**: before the user override, this node rendered `shared_buffers = 491MB` and bootstrap `max_connections = 100`; after the merge, both local and bootstrap parameter maps rendered `max_connections = 40`, `shared_buffers = 128MB`, `work_mem = 2MB`, and `maintenance_work_mem = 32MB`, and the private listeners stayed

at 10.104.0.8:8008 and 10.104.0.8:5432. Before changing live env files, validate the expanded compact JSON through Docker Compose dotenv parsing, because the listener-only value is already live and known-good.

After bootstrap, inspect the DCS state before editing:

```
patronictl -c <PATRONI_CONFIG> show-config <PG_SCOPE>
```

Review values such as:

```
loop_wait: 10
retry_timeout: 10
ttl: 30
maximum_lag_on_failover: <POC_APPROVED_BYTES>
postgresql:
  use_pg_rewind: true
  use_slots: true
  parameters:
    max_connections: 40
    shared_buffers: 128MB
    work_mem: 2MB
    maintenance_work_mem: 32MB
    shared_preload_libraries: 'timescaledb'
    wal_level: replica
    hot_standby: "on"
    password_encryption: scram-sha-256
    ssl: "on"
```

These are intentionally small POC starting values for the 2-GiB nodes. PgBouncer keeps application concurrency from becoming an oversized PostgreSQL connection budget. If the pinned image or measured workload needs more memory, change one value at a time and record why.

The Patroni timing relationship and minimum values vary by version; validate the resulting configuration with the pinned Patroni release.

Replication mode decision

For asynchronous mode, define an accepted RPO and `maximum_lag_on_failover`.

For synchronous mode, decide whether writes may block when no synchronous standby is available. Change mode through Patroni's cluster configuration, not by editing `synchronous_standby_names` manually.

Perform load and failure tests for the selected mode.

Runtime replication/authentication checkpoint - 2026-08-22

u1c-01 now passes the replication-role path needed by replicas. Runtime role inspection shows standby has LOGIN and REPLICATION and is neither superuser nor CREATEROLE/CREATEDB. `/run/postgres.yml` resolves `postgresql.authentication.replication.username` to standby and contains a configured replication password without exposing it. A real `sslmode=verify-full` connection as standby to logical name `postgres-ha.internal` and `hostaddr=10.104.0.2` succeeded and `pg_stat_ssl` reported `tls=true`. Patroni remained leader after the test.

The effective non-local HBA is TLS-only because `hostnossl all all all reject` precedes the catch-all TLS rules; replication currently matches `hostssl replication standby all md5`. This is sufficient for the controlled first-replica join because the standby credential has been proven over verify-full TLS and PostgreSQL itself binds only to 10.104.0.2. It is **not the final least-privilege HBA target**: source address is still `all` and the rule token is `md5`, while the design below calls for explicit east-west/app subnets and SCRAM-oriented rules. Tighten that after the HA members are stable instead of changing HBA during the first replica bootstrap.

The admin-role semantics are now **resolved and require no role mutation before replica admission**. Runtime SQL shows admin exists as NOLOGIN, non-replication, non-superuser, non-CREATEROLE, CREATEDB, INHERIT, with no stored password; cron_admin is granted to admin; and ulc-01 remained healthy leader throughout. Exact-image source inspection explains this state. configure_spilo.py still emits a legacy bootstrap.users stanza for PGUSER_ADMIN with a password plus createrole and createdb when PGPASSWORD_ADMIN is present, but Patroni 4.1.0 reports that this user-creation mechanism is unsupported. Spilo's own /scripts/post_init.sh then explicitly normalizes admin with ALTER ROLE admin WITH CREATEDB NOLOGIN NOCREATEROLE NOSUPERUSER NOREPLICATION INHERIT (or creates admin CREATEDB if absent) and grants cron_admin to it. Therefore the observed admin role is intentional Spilo post-init behavior, not a half-created human-login account. Do not convert admin to LOGIN merely to consume PGPASSWORD_ADMIN; that protected value is effectively unused for this NOLOGIN role under the current Patroni-4 bootstrap path. The attempted Python subprobe for the generated bootstrap-user shape printed no output because the script was sent to docker exec without interactive stdin, but the exact source plus live role/membership state is sufficient to resolve the semantics. Streaming-replica authentication remains independently proven, so ulc-02 may now be admitted as the first replica under controlled observation.

First-replica join checkpoint - 2026-08-22

ulc-02 first-replica admission is now **PASS**. Before startup, ulc-01 still returned HTTP 200 on /leader; all three etcd endpoints were healthy; 5432/8008 were free on ulc-02; /srv/spilo/pgdata was empty at mode 0700 numeric 101:103; there was no stale /service/lorawan-postgres-ha/members/ulc-02 key; and Compose syntax passed. The recreated Spilo container reported hostname ulc-02, INITDB_LOCALE=C, Status=running, and RestartCount=0.

Patroni then reported Lock owner: ulc-01; I am ulc-02, cloned from leader ulc-01 using basebackup_fast_xlog, and logged replica has been created plus bootstrapped from leader 'ulc-01'. Runtime listeners are private-only at 10.104.0.4:5432 and 10.104.0.4:8008. patronictl list shows ulc-01 as Leader/running and ulc-02 as Replica/streaming, timeline 1, receive/replay LSN 0/3000060, lag 0. Local SQL on ulc-02 confirms pg_is_in_recovery=true, PostgreSQL 18.6, max_connections=40, shared_buffers=128MB, work_mem=2MB, maintenance_work_mem=32MB, ssl=on, and replay LSN 0/3000060. Active PGDATA is mode 0700 numeric 101:103.

The pg_receivewal message about not renaming a .partial WAL file occurred during bootstrap, but the final Patroni state is streaming with zero reported lag and PostgreSQL is ready. Treat it as startup context unless it recurs with stalled replication. The recurring LC_ALL=en_US.utf-8 warnings remain the already-known container-locale hygiene issue and did not prevent clone or streaming startup.

The follow-up ulc-02 runtime gate is now **PASS**, so the second replica may be admitted under the same one-node-at-a-time process. Patroni role endpoints returned HTTP 200 for ulc-01 /leader and ulc-02 /replica; Docker showed Spilo running, not restarting, with RestartCount=0; and patronictl list still showed ulc-01 as Leader/running and ulc-02 as Replica/streaming, timeline 1, receive/replay LSN 0/3000168, lag 0.

Primary-side pg_stat_replication proved the actual replication connection from 10.104.0.4 is streaming, async, TLS-enabled with TLS 1.3 / TLS_AES_256_GCM_SHA384, and had sent_lsn=write_lsn=flush_lsn=replay_lsn=0/3000168 with calculated byte lag 0. A direct client connection to ulc-02 using logical certificate name postgres-ha.internal, physical hostaddr=10.104.0.4, and sslmode=verify-full succeeded; SQL returned server address 10.104.0.4, pg_is_in_recovery=true, TLS enabled, TLS 1.3, and the same cipher. pg_stat_wal_receiver on ulc-02 reported streaming from 10.104.0.2:5432, slot ulc_02, timeline 1, latest end LSN 0/3000168. The child-shell wrapper exited 0 while the parent SSH session remained alive. This closes the first-replica runtime/TLS/streaming gate. ulc-03 is now the next controlled join; do not start multiple remaining services or change HBA/locale hygiene at the same time.

SSH-session drop during the next ulc-02 gate

The first attempt at the follow-up runtime gate caused the operator's interactive SSH session to end after pasting a block that began with bare set -euo pipefail. The connection-drop investigation shows this was **not a host reboot, OOM event, or Spilo restart**: ulc-02 had been up continuously since 2026-08-20 05:55, kernel logs contained no OOM/kernel-kill evidence, memory was about 536 MiB used with about 1.4 GiB available, Spilo

remained running with Docker RestartCount=0, etcd remained up, and patronictl list still showed ulc-01 as Leader/running and ulc-02 as Replica/streaming with receive/replay LSN 0/3000168 and lag 0. The most likely cause is shell behavior: set -e was applied directly to the interactive login shell, so a later non-zero command could terminate that shell and therefore the SSH session. Treat this as an operator-shell issue, not a capacity or HA failure. For future paste-and-run diagnostics, put strict mode inside a child bash -s <<'EOF' ... EOF block so a failed check exits only the child script and leaves the SSH login alive.

The SSH journal also shows repeated unrelated Internet pre-authentication probes against invalid users/root. Those entries do not explain the operator session drop; the operator's own sessions were accepted by public key. Keep the existing SSH/firewall hardening controls in place and do not infer a compromise from these background probes alone.

Second-replica join checkpoint - 2026-08-22

ulc-03 second-replica admission is now **PASS**. Before startup, ulc-01 /leader and ulc-02 /replica both returned HTTP 200; all three etcd endpoints were healthy; 5432/8008 were free on ulc-03; /srv/spilo/pgdata was empty at mode 0700 numeric 101:103; no stale /service/lorawan-postgres-ha/members/ulc-03 key existed; Compose syntax passed; and the effective locale input was still INITDB_LOCALE=C. Spilo was force-recreated on ulc-03 only with hostname ulc-03, INITDB_LOCALE=C, running state, and RestartCount=0.

Patroni reported Lock owner: ulc-01; I am ulc-03, cloned from leader ulc-01 using basebackup_fast_xlog, logged replica has been created and bootstrapped from leader 'ulc-01', and PostgreSQL became ready after about 10 seconds. Runtime listeners are private-only at 10.104.0.8:5432 and 10.104.0.8:8008. patronictl list now shows all three members on timeline 1: ulc-01 Leader/running, ulc-02 Replica/streaming, and ulc-03 Replica/streaming; both replicas reported receive/replay LSN 0/5000000 and lag 0 at this checkpoint. Local SQL on ulc-03 confirmed pg_is_in_recovery=true, PostgreSQL 18.6, max_connections=40, shared_buffers=128MB, work_mem=2MB, maintenance_work_mem=32MB, ssl=on, and replay LSN 0/5000000. Active PGDATA is mode 0700 numeric 101:103.

The recurring LC_ALL=en_US.utf-8 warnings remain a known container-locale hygiene issue and did not prevent cloning or streaming. The pg_stat_kcache.linux_hz auto-detection log on ulc-03 reported 500000; do not treat that alone as a database/replication failure because Patroni and PostgreSQL reached healthy streaming state.

Final three-member cluster-establishment checkpoint - 2026-08-22

PASS. Patroni role endpoints returned HTTP 200 for ulc-01 /leader, ulc-02 /replica, and ulc-03 /replica. The DCS leader key resolved to ulc-01 and exactly three member keys existed. patronictl list showed ulc-01 as Leader/running and ulc-02 plus ulc-03 as Replica/streaming, all on timeline 1, with both replicas at receive/replay LSN 0/6000000 and reported lag 0.

A verify-full connection to the primary showed two and only two pg_stat_replication rows: ulc-02 from 10.104.0.4 and ulc-03 from 10.104.0.8, both streaming, async, TLS-enabled with TLS 1.3 / TLS_AES_256_GCM_SHA384, with equal sent/write/flush/replay LSN 0/6000000 and calculated byte lag 0. Structural summary was 2|1|1. Physical replication slots ulc_02 and ulc_03 were both active. A direct sslmode=verify-full connection to ulc-03 returned server address 10.104.0.8, pg_is_in_recovery=true, TLS 1.3, and the same cipher. pg_stat_wal_receiver on ulc-03 reported streaming from 10.104.0.2:5432 using slot ulc_03, timeline 1. The child gate exited 0 and the parent SSH shell remained alive. **The infrastructure-level three-member PostgreSQL/Patroni cluster is established.** This does not yet complete Phase 6 because the application databases, TimescaleDB database extension/schema, backup boundary, and controlled switchover remain outstanding.

Database commissioning preflight - 2026-08-22

PASS on the current primary ulc-01 as a discovery checkpoint. /leader returned HTTP 200. The only non-template database is currently postgres, using UTF-8 with C.UTF-8 collation/ctype. Existing roles are Spilo/Patroni

baseline roles only: postgres is LOGIN/SUPERUSER/REPLICATION, standby is LOGIN/REPLICATION, and admin remains the intentional NOLOGIN/CREATEDB privilege role; there is no chirpstack, telemetry_admin, telemetry_writer, telemetry_reader, or fabric_adapter role yet. pg_available_extensions on the final hardened live primary reports TimescaleDB default version 2.29.2, not the historical pre-hardening candidate value 2.26.2. shared_preload_libraries includes timescaledb together with the Spilo monitoring/auth extensions. TimescaleDB is not yet installed in the postgres database. Both replicas remained streaming, async, with calculated byte lag 0 during the preflight.

The 2.29.2 observation is authoritative for the current live primary package set; earlier 2.26.2 results remain historical evidence for the upstream/candidate image before the final Ubuntu package-upgrade hardening. Before creating lorawan_telemetry, query pg_available_extensions and shared_preload_libraries directly on all three current members and require the same TimescaleDB version/preload state so every member is promotion-ready.

The first three-member TimescaleDB consistency-gate attempt stopped on ulc-01 after the SQL returned inet_server_addr()::text = 10.104.0.2/32 while the shell comparison expected the bare string 10.104.0.2. This is a **test-comparison defect, not a PostgreSQL, TLS, role, or TimescaleDB failure**. The same returned row already proved ulc-01 is the primary, PostgreSQL is 18.6, TimescaleDB default version is 2.29.2, the extension is not installed in postgres, timescaledb is preloaded, and the 2.29.2 extension files are available. The corrected gate normalized the server address with host(inet_server_addr()) and then completed successfully on all three members.

Three-member TimescaleDB consistency checkpoint - 2026-08-22: **PASS**. Direct verify-full checks returned ulc-01|10.104.0.2|primary|18.6 ...|2.29.2|NOT_INSTALLED|preload=yes|version_files=yes, ulc-02|10.104.0.4|replica|18.6 ...|2.29.2|NOT_INSTALLED|preload=yes|version_files=yes, and ulc-03|10.104.0.8|replica|18.6 ...|2.29.2|NOT_INSTALLED|preload=yes|version_files=yes. patronictl list simultaneously showed ulc-01 Leader/running and both replicas streaming on timeline 1 with lag 0. Primary-side pg_stat_replication returned exactly two rows: ulc-02|10.104.0.4|streaming|async|0 and ulc-03|10.104.0.8|streaming|async|0. The child gate exited 0 and the parent SSH login remained alive. The recurring LC_ALL=en_US.utf-8/Perl locale warnings remain the known container-locale hygiene issue and did not affect PostgreSQL, replication, TLS, or TimescaleDB consistency.

This closes the package/preload consistency gate.

Live TimescaleDB 2.29.2 functionality checkpoint - 2026-08-22: **PASS**. On current leader ulc-01, the disposable database timescale_probe_2292 was created only after confirming it did not already exist. CREATE EXTENSION timescaledb loaded version 2.29.2; a probe.sensor_probe table was converted to a one-dimensional hypertable with create_hypertable(..., by_range('ts')); one probe-device row with value 42.5 was inserted and read back; and the in-database assertion block completed successfully. Direct verify-full reads from ulc-02 and ulc-03 each returned the temporary database, their expected private address, role replica, TimescaleDB 2.29.2, hypertable count 1, and matching row count 1, proving both extension catalog state and user data replicated correctly. The temporary database was then dropped on ulc-01; all three members subsequently returned probe-database count 0. Final Patroni state remained ulc-01 Leader/running, ulc-02 Replica/streaming, ulc-03 Replica/streaming, timeline 1, lag 0; primary-side replication remained two streaming|async|0 rows. Child gate exit code was 0 and the parent SSH shell stayed alive. The recurring locale warnings remain a separate non-blocking container-hygiene issue.

Why this matters: the final hardened image is no longer validated only by package metadata. The actual live HA cluster has now proven TimescaleDB 2.29.2 can load, create a hypertable, accept a row, replicate that extension-backed database to both promotion candidates, and cleanly remove the disposable probe. Permanent chirpstack and lorawan_telemetry database commissioning may now begin.

Permanent database structure checkpoint - 2026-08-22: **PASS**. The gate first re-proved ulc-01 /leader HTTP 200 and exactly two streaming replicas. Neither target database nor any target application role existed before the mutation. Five locked role shells were created with NOLOGIN and no stored password: chirpstack, telemetry_admin, telemetry_writer, telemetry_reader, and fabric_adapter. Database chirpstack was created with owner

chirpstack; lorawan_telemetry was created and kept owned by postgres at this checkpoint. TimescaleDB 2.29.2 was enabled only in lorawan_telemetry, and schema telemetry was created with owner telemetry_admin. chirpstack correctly reported zero TimescaleDB extension rows.

Direct verify-full checks against both replicas proved the permanent state replicated: each replica saw exactly two target databases, all five target roles, TimescaleDB 2.29.2 in lorawan_telemetry, and schema telemetry owned by telemetry_admin. Final Patroni state remained ulc-01 Leader/running, ulc-02 Replica/streaming, ulc-03 Replica/streaming; primary-side replication remained two streaming|async|0 rows. The child gate exited 0 and the SSH login shell stayed alive. The repeated locale warnings remain the known non-blocking container-hygiene issue.

Telemetry database ownership normalization, runtime credential activation, and real runtime-role authentication are now **PASS**. lorawan_telemetry is owned by the permanent NOLOGIN role telemetry_admin, and schema telemetry has the same owner on all three members. PostgreSQL reports password_encryption=scram-sha-256; sanitized verifier checks proved chirpstack, telemetry_writer, telemetry_reader, and fabric_adapter each have a SCRAM-SHA-256 verifier while telemetry_admin remains passwordless and NOLOGIN. The four runtime identities are LOGIN roles. Database-level CONNECT boundaries are proven: chirpstack can connect only to chirpstack, while telemetry_writer, telemetry_reader, and fabric_adapter can connect only to lorawan_telemetry. Both replicas replicated the final credential structure (4|1), Patroni remained one leader plus two streaming replicas, and primary-side calculated replication lag remained 0 for both replicas. Four independent psql -w sessions then proved the actual entered secrets work over sslmode=verify-full: chirpstack authenticated to chirpstack, and the three telemetry runtime roles authenticated to lorawan_telemetry; all four sessions reached 10.104.0.2 and pg_stat_ssl reported ssl=t, TLSv1.3, cipher TLS_AES_256_GCM_SHA384. No password value was printed.

Telemetry object schema commissioning is now **PASS**. The primary precheck confirmed lorawan_telemetry|telemetry_admin|telemetry_admin|2.29.2, zero target objects, and two streaming replicas. One transaction running as telemetry_admin created telemetry.uplinks, telemetry.measurements, telemetry.device_registry, telemetry.latest_uplinks, telemetry.latest_measurements, and telemetry.schema_version; uplinks and measurements became one-dimensional Timescale hypertables, all six named objects are owned by telemetry_admin, schema version 3 records that retention is not enabled, and the Timescale retention-job count is 0. Writer/reader ACL structure matched the intended least-privilege matrix while fabric_adapter retained no telemetry-object access. A real SET ROLE telemetry_writer INSERT/SELECT probe succeeded and was rolled back; the follow-up count proved zero probe rows remained. telemetry_reader SELECT statements executed successfully against the empty tables/views; zero returned groups are expected because there is no sensor data yet, not a permission failure. Both replicas reported 2 hypertables, 6 named schema objects, and schema-version row count 1. Patroni stayed one leader plus two streaming replicas and primary-side calculated lag remained 0 for both replicas. fabric_adapter still receives no broad telemetry-table grants; its object privileges belong to the Fabric outbox step.

The read-only HBA hardening preflight is now **PASS**. All three members expose the same effective pg_hba.conf, with zero parse errors. The current local Patroni configuration owns postgresql.pg_hba; neither bootstrap.dcs.postgresql.pg_hba nor current DCS postgresql.pg_hba is set. Therefore the persistent control point is the local Spilo-generated configuration, not a DCS-only edit. postgres, standby, chirpstack, telemetry_writer, telemetry_reader, and fabric_adapter all have SCRAM-SHA-256 verifiers; admin and telemetry_admin remain passwordless NOLOGIN. Remote replication is already TLS but still uses broad hostssl replication standby all md5; the remote application catch-all is hostssl all all all md5, and remote +zalandos PAM is also broad. hostnossl all all all reject proves non-TLS remote traffic is rejected. Primary-side replication remains TLSv1.3 with zero calculated lag to both replicas.

The first ulc-02 HBA canary attempt is **INCONCLUSIVE / ROLLED BACK BEFORE HBA RELOAD**. Pre-mutation HA checks passed and root-only rollback evidence was created. The candidate 20-rule /32 SCRAM HBA was written into the persistent SPILO_CONFIGURATION and Compose syntax still passed, but the attempt stopped before editing the live /run/postgres.yml: docker cp copied the temporary JSON file into /run/hba-target.json with restrictive host mktemp permissions, while the subsequent in-container Python process could not read that root-owned 0600 file and failed with PermissionError: [Errno 13] Permission denied. The automatic cleanup

attempted to restore the saved `spilo.env`, restored the saved local Patroni config, requested a reload for `u1c-02`, and `/replica` remained HTTP 200; `patronictl list` still showed `u1c-01` leader and both replicas streaming. The cleanup-time `patroni --validate-config ... -i` printed `name is not defined`; do not treat that validator message as proof of rollback failure because the runtime cluster remained healthy, but do not reuse that validator command until its Spilo-specific behavior is understood.

The first rollback-verification script then stopped immediately with `missing rollback file: .../spilo.env`, but this was a test-harness permission bug: the rollback directory is intentionally `0700 root:root`, while the first script tested `-f` as unprivileged `opsadmin`. The corrected sudo-based verification proved the rollback directory and all three evidence files really exist with the intended protections (`0700 0:0` directory; `0600 0:0` files), then exposed a real persistent-state mismatch: current `/etc/lorawan-cloud/spilo/spilo.env` SHA-256 is `da86466be7cc54be3e989a0e8cf070d414aa357cb160526e1fdec69edd879714`, while the pre-canary backup is `006398906b944ddc793802648081097d1f9250f66b36592e6a145c8ed455acad`.

The subsequent sanitized drift diagnosis resolves that mismatch precisely. Both files contain the same 20 environment keys; only `SPILO_CONFIGURATION` differs, and the decoded non-HBA portions are exactly equal. The current persistent env contains the failed candidate 20-rule explicit `/32 SCRAM postgresql.pg_hba` list, while the pre-canary backup contains **no explicit postgresql.pg_hba key** (`pg_hba count = 0`). That zero count is expected: before the canary, Spilo generated the original 10-rule local HBA into `/run/postgres.yml`; it was not persisted as an explicit `SPILO_CONFIGURATION` override. The running container still has exactly that original 10-rule local HBA and PostgreSQL still exposes the same 10 effective rules with zero parse errors; `u1c-01 /leader`, both `/replica` endpoints, Patroni membership, and the `u1c-02` WAL receiver remain healthy and streaming. Therefore the failed canary changed only the host-side persistent env; it never activated the 20-rule HBA in the running Patroni/PostgreSQL instance. The earlier statement that automatic cleanup had restored the persistent environment was too strong: cleanup attempted the restore, but current evidence proves the host file remained on the candidate value.

Persistent-only repair on `u1c-02` is now **PASS**. Before mutation, all three Patroni role endpoints returned HTTP 200. The failed 20-rule host file was preserved root-only as `/etc/lorawan-cloud/spilo/hba-rollback-20260822-235341/spilo.env.failed-20rule-20260823-000801`. `/etc/lorawan-cloud/spilo/spilo.env` was atomically restored from the known pre-canary backup and is byte-identical to it at SHA-256 `006398906b944ddc793802648081097d1f9250f66b36592e6a145c8ed455acad`, mode `0600 0:0`. The restored persistent `SPILO_CONFIGURATION` again has no explicit `postgresql.pg_hba` override, and `docker compose config --quiet` passes. No PostgreSQL/Patroni reload, restart, or container recreate was performed. The running local Patroni HBA remained at 10 rules, effective PostgreSQL HBA structural summary remained `10|0|4|0|3`, `u1c-02` WAL receiver remained `streaming|10.104.0.2|5432|u1c_02`, and final Patroni state remained `u1c-01` leader with both replicas streaming at lag 0. The persistent/runtime split returned to the proven pre-canary baseline.

Corrected live HBA hardening canary on `u1c-02` - 2026-08-23: **PASS**. The persistent env was first re-proved to have no explicit `postgresql.pg_hba` override. An exact runtime rollback copy of `/run/postgres.yml` was retained as `/run/postgres.yml.hba-canary-20260823-001550`. The live local Patroni HBA was changed to the 20-rule candidate and parsed successfully with member name `u1c-02` and exactly 20 rules before reload. `patronictl reload` was accepted and the effective PostgreSQL HBA converged to structural summary `20|0|0|14|3|3|3|3|2`: 20 total rules, zero parse errors, zero MD5 rules, 14 SCRAM rules, three replication `/32` SCRAM rules, three `postgres /32` SCRAM rules, three ChirpStack `/32` SCRAM rules, three telemetry-runtime `/32` SCRAM rules, and two final reject rules. Direct verify-full PostgreSQL access to `10.104.0.4` succeeded with TLSv1.3 / `TLS_AES_256_GCM_SHA384`; a physical replication-protocol `IDENTIFY_SYSTEM` session using the standby role succeeded; a non-TLS PostgreSQL connection failed with the expected HBA rejection. `u1c-02 /replica` remained HTTP 200, its WAL receiver stayed `streaming|10.104.0.2|5432|u1c_02`, and final Patroni state remained one leader plus two streaming replicas with zero reported lag. The persistent env remained intentionally unchanged with no HBA override. This proves the hardened rule set live on one replica; do not restart or recreate `u1c-02` until the same already-proven rule set is persisted into `SPILO_CONFIGURATION`.

First hardened-HBA persistence attempt on `u1c-02` - 2026-08-23: **INCONCLUSIVE / STOPPED BEFORE PERSISTENT MUTATION**. HA prechecks passed and the persistent env was still at the intended old baseline with

no explicit `postgresql.pg_hba`; the effective database HBA was still the proven live 20-rule set (20|0|0|14|3|3|3|2). The attempt failed while exporting that live HBA to a host `mktemp` path: `sudo tee /tmp/tmp... returned Permission denied`. Because the script sets `MUTATED=1` only after the later `spilo.env` rewrite, this failure occurred with `MUTATED=0`; no persistent env write, PostgreSQL reload, Patroni restart, or container recreate occurred. Also note a test-harness oversight in the attempted gate: its step-4 heredoc used `docker exec` without `-i`, so the in-container `python3 - validation` produced no output and did not actually consume the heredoc. The live effective-HBA query immediately afterward still independently proved the hardened 20-rule state.

Corrected hardened-HBA persistence on `u1c-02` - 2026-08-23: **PASS**. The retry removed the host temp-file handoff and used `docker exec -i` for stdin-fed Python. Before mutation, all three Patroni role endpoints returned HTTP 200, the persistent env still had no explicit `postgresql.pg_hba`, and the live local Patroni config exactly matched the previously proven 20-rule candidate. Effective PostgreSQL HBA again returned 20|0|0|14|3|3|3|2. The live HBA JSON was captured directly into memory, then a root-only rollback copy of the pre-change env was created at `/etc/lorawan-cloud/spilo/hba-persist-20260823-004235/spilo.env` under a `0700 root:root` directory. The exact live 20-rule list was inserted into persistent `SPILO_CONFIGURATION`; `docker compose config --quiet` passed; sanitized comparison reported persistent `pg_hba` count = 20 and persistent/live exact HBA equality: PASS; the env remained `0600 0:0`. No PostgreSQL reload, Patroni restart, or container recreate occurred: container identity, start time, and `RestartCount=0` were unchanged before/after. The effective live HBA remained 20|0|0|14|2 for total|errors|md5|scram|reject-all, `u1c-02` WAL receiver remained streaming|10.104.0.2|5432|`u1c-02`, and final Patroni state remained `u1c-01` leader with `u1c-02` and `u1c-03` streaming at lag 0. `u1c-02` is now aligned at both layers: the proven hardened 20-rule HBA is active live and persisted for future container recreation.

Cross-host future-primary replication-auth gate to `u1c-02` - 2026-08-23: **PASS from both remaining members**. From `u1c-01`, `ip route get 10.104.0.4` selected `eth1` source `10.104.0.2`; the target `/replica` endpoint was HTTP 200; a verify-full physical-replication `IDENTIFY_SYSTEM` session using the standby role succeeded and returned system identifier `7676855802088521796`, timeline 1, WAL position `0/A000000`; the target remained HTTP 200 after the probe. The same test from `u1c-03` selected `eth1` source `10.104.0.8` and returned the same system identifier, timeline, and WAL position, again with `u1c-02 /replica` staying HTTP 200. Both child gates exited 0 and both SSH parent shells remained alive. This proves the explicit .2/32 and .8/32 replication SCRAM rules on hardened `u1c-02` work cross-host, not only from localhost. `u1c-02` is therefore suitable as a future Patroni primary from the HBA/replication-auth perspective.

Live HBA hardening canary on `u1c-03` - 2026-08-23: **PASS**. Baseline role endpoints were all HTTP 200, Patroni remained `u1c-01` leader with both replicas streaming at lag 0, `u1c-03` persistent `SPILO_CONFIGURATION` had no explicit `postgresql.pg_hba`, and its effective HBA was the original 10|0|4|0|3 baseline. A runtime rollback copy was retained at `/run/postgres.yml.hba-canary-20260823-005007`. The exact already-proven 20-rule candidate was written only into live `/run/postgres.yml`, validated as member `u1c-03` with exactly 20 rules, and reloaded only on `u1c-03`. Effective PostgreSQL HBA converged to 20|0|0|14|3|3|3|2. Direct verify-full postgres access to `10.104.0.8` succeeded over TLSv1.3 / TLS_AES_256_GCM_SHA384; standby-role `IDENTIFY_SYSTEM` returned `7676855802088521796|1|0/A000000|; sslmode=disable` failed with the expected `pg_hba.conf` rejects connection ... no encryption. `u1c-03 /replica` remained HTTP 200, its WAL receiver remained streaming|10.104.0.2|5432|`u1c-03`, the persistent env remained untouched with no HBA override, and the Spilo container ID/start time/`RestartCount=0` were unchanged. Final Patroni state remained healthy with both replicas streaming at lag 0.

Hardened-HBA persistence on `u1c-03` - 2026-08-23: **PASS**. The persistence gate first re-proved all three Patroni role endpoints at HTTP 200, the persistent env at the no-HBA-override baseline, exact equality of live `/run/postgres.yml` to the proven 20-rule candidate, and effective live HBA summary 20|0|0|14|2. The live HBA was captured directly into memory. A root-only rollback backup was created at `/etc/lorawan-cloud/spilo/hba-persist-20260823-005339/spilo.env` under a `0700 root:root` directory. The exact live list was then inserted into persistent `SPILO_CONFIGURATION`; `Compose parsing` passed; the persistent HBA count became 20; persistent/live exact equality passed; and `/etc/lorawan-cloud/spilo/spilo.env` remained `0600 0:0`. The live database remained untouched: post-persistence HBA summary stayed 20|0|0|14|2, WAL receiver stayed streaming|10.104.0.2|5432|`u1c-03`, and the Spilo container ID/start time/`RestartCount=0` were identical before

and after. Final Patroni state stayed `u1c-01` leader with both replicas streaming at lag 0. `u1c-03` is now aligned at both live and persistent HBA layers.

Cross-host future-primary replication-auth gate to `u1c-03` - 2026-08-23: **PASS from both remaining members.** From `u1c-01`, `ip route get 10.104.0.8` selected `eth1` source `10.104.0.2`; `u1c-03 /replica` returned HTTP 200; a standby-role physical replication `IDENTIFY_SYSTEM` session over `sslmode=verify-full` succeeded and returned system identifier `7676855802088521796`, timeline 1, WAL position `0/A000000`; the target remained HTTP 200 after the probe. From `u1c-02`, the same gate selected `eth1` source `10.104.0.4`; the same SCRAM + verify-full replication-protocol session returned the same system identifier, timeline, and WAL position; `u1c-03 /replica` again stayed HTTP 200. Both child gates exited 0 and both SSH parent shells remained alive. This proves the explicit `.2/32` and `.4/32` replication rules on hardened `u1c-03` work in real new cross-host sessions. Both replicas are now live-hardened, persistently hardened, and proven as future-primary replication endpoints.

Leader live HBA hardening canary on `u1c-01` - 2026-08-23: **PASS.** The gate first re-proved all three Patroni role endpoints at HTTP 200, `u1c-01` as the actual PostgreSQL primary, the leader persistent `SPILO_CONFIGURATION` at the old no-HBA-override baseline, and the effective original HBA at `10|0|4|0|3`. A runtime rollback copy was retained at `/run/postgres.yml.hba-canary-20260823-010121`. The exact already-proven 20-rule HBA was written only into live `/run/postgres.yml`, validated as member `u1c-01` with exactly 20 rules, and reloaded only on the leader. Effective PostgreSQL HBA converged to `20|0|0|14|3|3|3|3|2`: 20 total rules, zero parse errors, zero MD5 rules, 14 SCRAM rules, three replication /32 rules, three postgres /32 rules, three ChirpStack /32 rules, three telemetry-runtime /32 rules, and two final reject rules. The leader endpoint stayed HTTP 200; direct verify-full postgres access to `10.104.0.2` returned `10.104.0.2|f|t|TLSv1.3|TLS_AES_256_GCM_SHA384`; `sslmode=disable` failed with the expected HBA rejection. Existing replication remained healthy: `u1c-02|10.104.0.4|streaming|async|t|TLSv1.3|TLS_AES_256_GCM_SHA384|0` and `u1c-03|10.104.0.8|streaming|async|t|TLSv1.3|TLS_AES_256_GCM_SHA384|0`; both replica role endpoints remained HTTP 200. The leader persistent env remained intentionally unchanged with no explicit HBA override, and the Spilo container ID/start time/RestartCount=0 were identical before and after. Final Patroni state remained `u1c-01` leader with both replicas streaming at lag 0. This proves the hardened policy live on the current leader, but the pre-existing replication streams are not fresh HBA-authentication proof. Before persisting the leader policy, create new standby-role `IDENTIFY_SYSTEM` sessions from `u1c-02` and `u1c-03` to `10.104.0.2` and verify their VPC source addresses.

Fresh cross-host replication-auth gate to hardened leader `u1c-01` - 2026-08-23: **PASS from both replicas.** From `u1c-02`, `ip route get 10.104.0.2` selected `eth1` source `10.104.0.4`; `u1c-01 /leader` returned HTTP 200; a new standby-role physical-replication `IDENTIFY_SYSTEM` session over `sslmode=verify-full` succeeded and returned system identifier `7676855802088521796`, timeline 1, WAL position `0/A000408`; the leader remained HTTP 200 after the probe. From `u1c-03`, the same gate selected `eth1` source `10.104.0.8`; the same fresh SCRAM + verify-full replication-protocol session returned `7676855802088521796|1|0/A000408|`; the leader again stayed HTTP 200. Both child gates exited 0 and both SSH parent shells remained alive. This closes the fresh HBA-authentication proof for the leader's `.4/32` and `.8/32` replication rules. All three members now have live hardened HBA, and each member has been proven to accept fresh replication-protocol authentication from both other member source addresses.

Hardened-HBA persistence on leader `u1c-01` - 2026-08-23: **PASS.** The persistence gate re-proved `u1c-01 /leader`, `u1c-02 /replica`, and `u1c-03 /replica` at HTTP 200, confirmed local PostgreSQL role primary, confirmed the leader persistent env still had no explicit `postgresql.pg_hba`, and confirmed live `/run/postgres.yml` exactly matched the proven 20-rule policy. Effective HBA remained `20|0|0|14|2`. A root-only rollback copy was retained at `/etc/lorawan-cloud/spilo/hba-persist-20260823-010732/spilo.env`. The exact live HBA was captured in memory and written into persistent `SPILO_CONFIGURATION`; `docker compose config --quiet` passed; persistent HBA count became 20; persistent/live exact equality passed; and `/etc/lorawan-cloud/spilo/spilo.env` remained `0600 root:root`. No PostgreSQL reload, Patroni restart, switchover, or container recreate occurred. Container ID/start time/RestartCount=0 were identical before and after. The live HBA remained `20|0|0|14|2`; primary-side replication still showed `u1c-02` and `u1c-03` streaming asynchronously over `TLSv1.3 / TLS_AES_256_GCM_SHA384` with zero byte lag at the checkpoint; and final Patroni state remained `u1c-01` leader with both replicas streaming at lag 0. All three members are now aligned at both HBA layers: the proven 20-rule policy is active live and persisted for future recreation.

Final three-node HBA parity + TLS/negative gate - 2026-08-23: **PASS on ulc-01, ulc-02, and ulc-03.** Every node first re-proved ulc-01 /leader, ulc-02 /replica, and ulc-03 /replica at HTTP 200. Each protected /etc/lorawan-cloud/spilo/spilo.env remained 0600 root:root and passed docker compose config --quiet. On all three members the live local Patroni HBA contained exactly the canonical 20 rules, the persistent SPILO_CONFIGURATION.postgresql.pg_hba also contained 20 rules, and persistent/live exact equality passed. Effective PostgreSQL HBA summary was identically 20|0|0|14|3|3|3|3|2|0: 20 total rules, zero parse errors, zero MD5 rules, 14 SCRAM rules, three exact replication /32 rules, three exact postgres /32 rules, three exact ChirpStack /32 rules, three exact telemetry-runtime /32 rules, two broad-address reject rules, and **zero broad-address permissive rules**. Verify-full postgres sessions succeeded to each node at its expected role: 10.104.0.2|f|t|TLSv1.3|TLS_AES_256_GCM_SHA384, 10.104.0.4|t|t|TLSv1.3|TLS_AES_256_GCM_SHA384, and 10.104.0.8|t|t|TLSv1.3|TLS_AES_256_GCM_SHA384. On every member, sslmode=disable failed specifically with pg_hba.conf rejects connection ... no encryption, proving the non-TLS reject path. On the leader, pg_stat_replication showed exactly ulc-02 and ulc-03 streaming asynchronously over TLSv1.3 / TLS_AES_256_GCM_SHA384 with zero byte lag at the checkpoint (2|1|1 structural check). On each replica, the WAL receiver remained streaming|10.104.0.2|5432|ulc_02 or ... |ulc_03 as appropriate. Each local Patroni role endpoint remained HTTP 200, each child gate exited 0, and each SSH parent shell stayed alive. The HBA rollout itself is therefore complete at live, persistent, TLS, SCRAM, negative-test, and replication-health layers. One final application-role login check under the hardened HBA remains before moving to the logical backup boundary. Do not perform a Patroni switchover before the logical backup boundary exists.

First post-hardening application-role auth attempt - 2026-08-23: **TEST HARNESS FAILURE; NO APPLICATION AUTH RESULT.** The three-node HA baseline passed, but the first chirpstack -> chirpstack probe ended with fe_sendauth: no password supplied. The failure came from the wrapper, not PostgreSQL policy: bash -s <<'EOF' already uses standard input to deliver the script, while the outer read -rsp also attempted to read the hidden password from standard input. As a result, the password was not reliably read from the operator terminal and no valid password-authentication test occurred. This does **not** invalidate the completed HBA gate and is not evidence of an HBA rejection, bad SCRAM verifier, or bad application password. Retry the read-only application-role gate with password input explicitly attached to /dev/tty; only a real returned login row counts as the post-hardening application-auth result.

Corrected post-hardening application-role authentication gate - 2026-08-23: **PASS.** The retry attached every hidden password read explicitly to /dev/tty, so the heredoc could no longer consume operator input. The pre-gate HA baseline returned HTTP 200 for ulc-01 /leader, ulc-02 /replica, and ulc-03 /replica. Four independent verify-full sessions to the current primary at 10.104.0.2:5432 then succeeded through the hardened HBA: chirpstack|chirpstack|10.104.0.2|f|t|TLSv1.3|TLS_AES_256_GCM_SHA384, telemetry_writer|lorawan_telemetry|10.104.0.2|f|t|TLSv1.3|TLS_AES_256_GCM_SHA384, telemetry_reader|lorawan_telemetry|10.104.0.2|f|t|TLSv1.3|TLS_AES_256_GCM_SHA384, and fabric_adapter|lorawan_telemetry|10.104.0.2|f|t|TLSv1.3|TLS_AES_256_GCM_SHA384. Passwords remained hidden and were not printed. The final HA check again returned all three role endpoints at HTTP 200; patronictl list showed ulc-01 leader and ulc-02/ulc-03 streaming on timeline 1, both at receive/replay LSN 0/A000510 with reported lag 0. The child gate exited 0 and the SSH login shell remained alive. This closes post-hardening application authentication. **Next establish the POC logical backup boundary before any Patroni switchover or destructive failure test.**

6.10 PostgreSQL TLS and pg_hba

Require TLS for application, replication, backup, and administrative connections unless a documented private-network exception is approved.

Allow only exact VPC sources and named roles. Conceptual rules:

```
hostssl replication <REPLICATION_ROLE> <POSTGRES_SUBNET> scram-sha-256
hostssl <CHIRPSTACK_DB> <CHIRPSTACK_ROLE> <APP_SUBNET> scram-sha-256
```

```
hostssl postgres <MONITOR_ROLE> <MONITOR_SUBNET> scram-sha-256
```

Do not use trust, broad 0.0.0.0/0, or application superusers.

6.11 Create both POC databases and roles

Connect through the verified primary path. Create role shells locked first, establish ownership, then activate only the runtime identities after password and permission verification:

```
CREATE ROLE chirpstack NOLOGIN;
-- Set the chirpstack password safely in the later credential gate.
CREATE DATABASE chirpstack OWNER chirpstack;
```

Then create the telemetry database on the **same Patroni cluster**:

```
CREATE ROLE telemetry_admin NOLOGIN;
-- telemetry_admin remains passwordless and NOLOGIN as the ownership role.
CREATE DATABASE lorawan_telemetry OWNER telemetry_admin;
```

Inside lorawan_telemetry, create separate runtime identities for the small POC:

```
CREATE ROLE telemetry_writer NOLOGIN;
CREATE ROLE telemetry_reader NOLOGIN;
CREATE ROLE fabric_adapter NOLOGIN;
```

Set passwords only for chirpstack, telemetry_writer, telemetry_reader, and fabric_adapter using \password or the approved secret workflow; do not put live passwords in Markdown or shell history. Verify SCRAM-SHA-256 storage before enabling LOGIN. Keep telemetry_admin passwordless and NOLOGIN.

Why separate logical databases/roles even in a tiny POC: it preserves the future security and ownership boundaries without paying for another database service.

Create separate roles for monitoring, migrations, and PgBouncer administration only when needed by the POC.

6.12 Enable TimescaleDB inside the Patroni cluster

TimescaleDB is a PostgreSQL extension, so there is no fourth database server. However, every Patroni member must have a compatible TimescaleDB library available locally before that member may be promoted.

Step 1 - prove the extension files exist on all three members

Run against each PostgreSQL member directly over the private network using an administrative connection:

```
SELECT name, default_version
FROM pg_available_extensions
WHERE name = 'timescaledb';
```

Pass only when all three members report timescaledb and the intended build is compatible with the pinned PostgreSQL major. Record the image/package version for each node. A replica with missing or mismatched TimescaleDB binaries is **not promotion-ready**.

Step 2 - preload TimescaleDB consistently

Keep `timescaledb` in `shared_preload_libraries` through Patroni-managed PostgreSQL configuration on all members. If another required preload library already exists, append TimescaleDB; do not overwrite the existing list.

After the controlled restart required by the pinned PostgreSQL/Timescale version, verify on every member:

```
SHOW shared_preload_libraries;
```

Step 3 - enable the extension only in `lorawan_telemetry`

Connect through the verified writable-primary path:

```
\c lorawan_telemetry
CREATE EXTENSION IF NOT EXISTS timescaledb;

SELECT extname, extversion
FROM pg_extension
WHERE extname = 'timescaledb';
```

Do not enable TimescaleDB in the `chirpstack` database unless a real ChirpStack requirement appears. ChirpStack remains an ordinary PostgreSQL database.

Step 4 - create the real telemetry schema

Reuse the generic multi-sensor schema from [./integrations/timescaledb/02-create-telemetry-schema.md](https://github.com/chirpstack/chirpstack/blob/master/docs/integrations/timescaledb/02-create-telemetry-schema.md), adapting only the connection command because this cloud POC connects through PgBouncer/HAProxy instead of a standalone `telemetry-db` container.

Required POC shape:

```
lorawan_telemetry [TimescaleDB]
telemetry.uplinks      -> hypertable
telemetry.measurements -> hypertable
telemetry.device_registry
telemetry.latest_uplinks
telemetry.latest_measurements
telemetry.fabric_outbox -> ordinary PostgreSQL table
```

Use the hypertable creation syntax supported by the **pinned TimescaleDB version**. Keep uniqueness rules compatible with the time partition column. Do not enable a destructive retention policy until the intended retention period and backup boundary are approved.

Step 5 - prove Timescale survives PostgreSQL failover

Before the first full host-failure test, perform a controlled Patroni switchover and then query through the unchanged PgBouncer endpoint:

```
SELECT extname, extversion
FROM pg_extension
WHERE extname = 'timescaledb';

SELECT hypertable_schema, hypertable_name
FROM timescaledb_information.hypertables
ORDER BY 1, 2;
```

Pass when the promoted primary reports the same TimescaleDB extension and both telemetry hypertables without reinstalling or recreating anything.

6.13 POC backup boundary

Do not add paid object storage merely to prove the HA topology unless the test specifically includes disaster recovery.

Before any Patroni switchover or destructive failure test, create validated custom-format logical dumps of both application databases on the current primary host and then copy them off that Droplet:

```
chirpstack.dump
lorawan_telemetry.dump
SHA256SUMS
```

This boundary has **two separate gates**:

1. **Server-side dump gate** - both archives exist, are non-empty, `pg_restore --list` can read them, hashes are recorded, and Patroni remains healthy.
2. **Off-Droplet copy gate** - both archives and SHA256SUMS are copied to a trusted operator workstation or other independent location, and the copied hashes match.

Do not call the backup boundary complete after only gate 1. A dump that exists solely on the same Droplet is still lost if that Droplet or its disk is lost.

Step 1 - create and validate the dumps on the current leader

Run this on `ulc-01` **only while it is still the Patroni leader**. The command uses the local PostgreSQL Unix socket inside the Spilo container, so no database password is placed in shell history. It does not reload PostgreSQL, restart Patroni, or trigger failover.

```
sudo -v

bash -s <<'EOF'
set -euo pipefail
set +x

if [ "$(hostname)" != 'ulc-01' ]; then
    echo 'FAIL: run this gate on ulc-01 only'
    exit 1
fi

BACKUP_ROOT="$HOME/backups/ha-poc"
STAMP="$(date -u +%Y%m%d-%H%M%S)"
BACKUP_DIR="$BACKUP_ROOT/$STAMP"

umask 077
mkdir -p "$BACKUP_ROOT"
chmod 700 "$BACKUP_ROOT"
mkdir "$BACKUP_DIR"
chmod 700 "$BACKUP_DIR"

echo '=== POC LOGICAL BACKUP - SERVER-SIDE GATE ==='

echo
echo '=== 1. HA BASELINE ==='
for SPEC in \
    'ulc-01|10.104.0.2|leader' \
    'ulc-02|10.104.0.4|replica' \
    'ulc-03|10.104.0.8|replica'
do
    IFS='|' read -r NODE IP ENDPOINT <<< "$SPEC"
    HTTP=$(curl -s -o /dev/null -w '%{http_code}' --max-time 5 \
        "http://${IP}:8008/${ENDPOINT}" || true)
    echo "$NODE /$ENDPOINT = HTTP $HTTP"
    [ "$HTTP" = '200' ] || { echo 'FAIL: Patroni HA baseline is not healthy'; exit 1; }
done
```

```

echo
echo '=== 2. SPACE PRECHECK ==='
DB_BYTES=$(
    sudo docker exec -e LC_ALL=C spilo \
        psql -U postgres -d postgres -X -Atc \
        "SELECT COALESCE(sum(pg_database_size(datname)),0) FROM pg_database WHERE datname IN
('chirpstack','lorawan_telemetry');"
)
FREE_BYTES=$(df -PB1 "$BACKUP_ROOT" | awk 'NR==2 {print $4}')
MIN_FREE=$((DB_BYTES + 1073741824))
printf 'database bytes = %s\nfree bytes      = %s\nrequired floor = %s\n' \
    "$DB_BYTES" "$FREE_BYTES" "$MIN_FREE"
[ "$FREE_BYTES" -ge "$MIN_FREE" ] || { echo 'FAIL: insufficient free space for conservative backup floor';
exit 1; }

echo
echo '=== 3. CREATE + VALIDATE CUSTOM ARCHIVES ==='
for DB in chirpstack lorawan_telemetry
do
    FINAL="$BACKUP_DIR/${DB}.dump"
    PARTIAL="$BACKUP_DIR/${DB}.dump.partial"

    echo "--- dumping $DB ---"
    sudo docker exec -e LC_ALL=C spilo \
        pg_dump -U postgres --no-password --format=custom "$DB" \
        > "$PARTIAL"

    [ -s "$PARTIAL" ] || { echo "FAIL: $DB archive is empty"; exit 1; }

    sudo docker exec -i -e LC_ALL=C spilo \
        pg_restore --list < "$PARTIAL" >/dev/null

    mv "$PARTIAL" "$FINAL"
    chmod 600 "$FINAL"
    echo "$DB archive: PASS"
done

echo
echo '=== 4. RECORD HASHES ==='
(
    cd "$BACKUP_DIR"
    sha256sum chirpstack.dump lorawan_telemetry.dump > SHA256SUMS
)
chmod 600 "$BACKUP_DIR/SHA256SUMS"
cat "$BACKUP_DIR/SHA256SUMS"

echo
echo '=== 5. FILE EVIDENCE ==='
stat -c '%n|bytes=%s|mode=%a|uid=%u|gid=%g' \
    "$BACKUP_DIR/chirpstack.dump" \
    "$BACKUP_DIR/lorawan_telemetry.dump" \
    "$BACKUP_DIR/SHA256SUMS"

echo
echo '=== 6. FINAL HA CHECK ==='
for SPEC in \
    'ulc-01|10.104.0.2|leader' \
    'ulc-02|10.104.0.4|replica' \
    'ulc-03|10.104.0.8|replica'
do
    IFS='|' read -r NODE IP ENDPOINT <<< "$SPEC"
    HTTP=$(curl -s -o /dev/null -w '%{http_code}' --max-time 5 \
        "http://${IP}:8008/${ENDPOINT}" || true)
    echo "$NODE /$ENDPOINT = HTTP $HTTP"
    [ "$HTTP" = '200' ] || { echo 'FAIL: HA state changed during backup gate'; exit 1; }
done

sudo docker exec -e LC_ALL=C spilo \
    patronictl -c /run/postgres.yml list

echo
echo 'SERVER-SIDE POC LOGICAL BACKUP CREATION: PASS'
echo "Backup directory: $BACKUP_DIR"

```

```
echo 'BACKUP BOUNDARY IS NOT COMPLETE UNTIL OFF-DROPLET COPY + HASH VERIFICATION PASS'
EOF

RC=$?
echo
echo "Child gate exit code = $RC"
echo 'SSH LOGIN SHELL IS STILL ALIVE'
```

Why these checks exist: `pg_dump --format=custom` creates a portable PostgreSQL archive, while `pg_restore --list` proves the archive container can be parsed before it is trusted as evidence. `SHA256SUMS` gives the later off-Droplet copy a byte-for-byte integrity check. The free-space test deliberately leaves an additional 1 GiB floor instead of assuming compression will save enough space. Failed `.partial` files are left in the timestamped directory for diagnosis rather than silently deleted.

`pg_restore --list` is **not** a restore rehearsal. A later full restore test must also follow the TimescaleDB restore procedure. This step only establishes the POC rollback artifact before controlled HA testing.

Step 2 - copy off the Droplet and verify hashes

After Step 1 passes, copy the timestamped directory to a trusted workstation or independent host and verify both hashes there against `SHA256SUMS`. Record that result before starting the first controlled Patroni switchover.

For the current POC checkpoint, the server-side source directory is:

```
/home/opsadmin/backups/ha-poc/20260823-013606
```

Copy that whole directory, not just the two `.dump` files, so the independently stored `SHA256SUMS` travels with the archives.

Recorded POC server-side checkpoint - 2026-08-23: PASS. On `ulc-01`, `/home/opsadmin/backups/ha-poc/20260823-013606` contains validated custom-format `chirpstack.dump` (37,050 bytes, SHA-256 61095053c9fead75f1fde16a6c0c163fa7064819476b8329bafb53d16a4a61b1), `lorawan_telemetry.dump` (59,383 bytes, SHA-256 5d84dee3d696014a944b39a26191adcfc3803cd0f90ccab144f9340260b28c85), and `SHA256SUMS`; each file is mode `0600`. `pg_restore --list` parsed both archives. The TimescaleDB archive emitted the known continuous_agg circular-foreign-key warning during a full dump, but the dump completed and parsed successfully. Patroni stayed healthy with one leader and two streaming zero-lag replicas. This checkpoint proves only the server-side gate; the backup boundary remains open until the directory is copied off the Droplet and the copied hashes pass.

First off-Droplet transfer attempt - 2026-08-23: no transfer. A Windows `scp` attempt to `opsadmin@143.198.205.54` failed at SSH authentication with `Permission denied (publickey)`. The failure happened before any file transfer, so the server-side backup artifacts remain unchanged. This matches the project access model: `opsadmin` has a separate `Ed25519` identity for another device, while the earlier workstation hardening proof authenticated to `ulc-01` as `jervis`. Do not disable public-key-only SSH and do not move an administration private key onto a server. Use the workstation private key actually authorized for `opsadmin`, or deliberately use the already-proven `jervis` workstation path only after ensuring it has read-only access to this backup directory. The backup boundary remains **OPEN** until both dump files exist off-Droplet and their SHA-256 values match the server-side records.

Recorded off-Droplet checkpoint - 2026-08-23: PASS. The complete `20260823-013606` directory was copied to `C:\Users\admin\lorawan-poc-backups\20260823-013606` on an independent Windows operator workstation using the authorized `opsadmin` SSH identity. Copied sizes were 37,050 bytes for `chirpstack.dump`, 59,383 bytes for `lorawan_telemetry.dump`, and 171 bytes for `SHA256SUMS`, exactly matching the source evidence. Workstation SHA-256 verification matched the server records byte-for-byte: `chirpstack.dump = 61095053c9fead75f1fde16a6c0c163fa7064819476b8329bafb53d16a4a61b1`; `lorawan_telemetry.dump = 5d84dee3d696014a944b39a26191adcfc3803cd0f90ccab144f9340260b28c85`. The workstation ended OFF-DROPLET POC BACKUP HASH GATE: PASS.

Backup-boundary decision: COMPLETE. The planned Patroni switchover may now proceed because the rollback artifact exists independently of the database host and its integrity is verified. This still does not replace a later restore rehearsal or WAL/PITR design.

This is enough for the HA POC rollback boundary. It is intentionally smaller than a full disaster-recovery design: database-global roles, WAL archiving, PITR, automated retention, and restore automation remain separate production backup work.

HA and backup are different concepts: three PostgreSQL replicas do not protect against an accidental DELETE, DROP, or other logical mistake that is correctly replicated to all three nodes.

6.14 Protect Patroni ownership

Disable any host `postgresql.service` that could start the same data directory outside Patroni:

```
systemctl list-unit-files | grep -i postgres
```

Do not disable an unidentified unit until its paths are inspected. The completion state must have one owner for PostgreSQL lifecycle: Patroni inside Spilo.

6.15 Planned switchover

The POC logical backup boundary is complete, so the first controlled role-change gate is now authorized. The current leader is `ulc-01`; deliberately promote `ulc-02` first while keeping `ulc-03` as the untouched second replica.

Current first-switchover target:

```
before: ulc-01 = leader, ulc-02 = replica, ulc-03 = replica
action: planned Patroni switchover ulc-01 -> ulc-02
after:  ulc-02 = leader, ulc-01 = replica, ulc-03 = replica
```

Use Patroni's **switchover** operation, not **failover**. A switchover is the healthy-cluster path: it names the current leader and the exact candidate rather than simulating an unexpected primary loss.

Run from `ulc-01`:

```
sudo docker exec -e LC_ALL=C spilo \
  patronictl -c /run/postgres.yml switchover lorawan-postgres-ha \
    --leader ulc-01 \
    --candidate ulc-02 \
    --force
```

Do not run `docker restart`, `docker compose up --force-recreate`, `patronictl failover`, or manually stop PostgreSQL as part of this gate. Patroni must perform the demotion/promotion itself.

Immediately verify:

- `ulc-02` /leader becomes HTTP 200;
- `ulc-01` /replica becomes HTTP 200;
- `ulc-03` /replica remains HTTP 200;
- `patronictl list` shows exactly one leader and two streaming replicas;
- the new leader's timeline advances as expected after promotion;
- both replicas converge with no material/stuck lag;

- local SQL on new leader `u1c-02` reports `pg_is_in_recovery() = false`;
- TimescaleDB remains 2.29.2 in `lorawan_telemetry` and both telemetry hypertables remain present;
- the old leader reattaches as a replica without manual reinitialization.

Why: this proves Patroni can transfer ownership of the same PostgreSQL/TimescaleDB cluster using the hardened HBA and replication credentials already proven on all three VPC addresses. Because HAProxy, PgBouncer, and ChirpStack application containers are not deployed yet in the current phase, this first database-layer switchover must not claim application-routing behavior. Those routing and retry checks belong after Phase 7/9 services exist.

Stop and diagnose if the candidate does not become leader, the old leader does not become a streaming replica, `u1c-03` drops out, or the cluster exposes more or fewer than one leader. Do not use failover as an automatic recovery shortcut for a failed planned switchover.

Recorded first controlled switchover - 2026-08-23: PASS. The precheck showed `u1c-01` as the healthy leader with exactly two streaming replicas. Patroni reported Successfully switched over to "`u1c-02`". During the transition, the old leader briefly appeared `Replica/stopped` and `/replica` returned HTTP 503; this recovered automatically by the third two-second poll. Final Patroni state showed `u1c-02` `Leader/running` on timeline 2, `u1c-01` `Replica/running`, and `u1c-03` `Replica/running`, with all expected role endpoints HTTP 200. Local SQL on `u1c-01` proved `pg_is_in_recovery=true`, and its WAL receiver reported `streaming|10.104.0.4|5432`, proving the demoted member attached to the new leader without reinitialization. The `u1c-01` Spilo container `identity/start timestamp/RestartCount=0` did not change.

Recorded promoted-primary validation on `u1c-02` - 2026-08-23: PASS. All expected role endpoints remained HTTP 200. Local SQL on `u1c-02` returned `pg_is_in_recovery=false`; a direct verify-full session to `10.104.0.4` negotiated TLSv1.3 / TLS_AES_256_GCM_SHA384. Application database ownership remained `chirpstack|chirpstack` and `lorawan_telemetry|telemetry_admin`. `lorawan_telemetry` retained TimescaleDB 2.29.2, both expected hypertables (`telemetry.measurements`, `telemetry.uplinks`), and all six commissioned telemetry objects. A rollback-only write probe created and inserted into `public.__patroni_switchover_probe` inside a transaction, then ROLLBACK removed it completely, proving the promoted node is writable without leaving test state. Primary-side replication showed exactly two TLSv1.3 streaming replicas: `u1c-01` from `10.104.0.2` and `u1c-03` from `10.104.0.8`, both with zero byte lag at the checkpoint. Final Patroni state remained `u1c-02` `Leader/running` on timeline 2, with `u1c-01` and `u1c-03` streaming on timeline 2; child gate exit code was 0 and the SSH shell remained alive.

Recorded post-promotion application-role authentication on `u1c-02` - 2026-08-23: PASS. Hidden-password verify-full logins to the promoted primary succeeded for all four runtime identities: `chirpstack -> chirpstack`, `telemetry_writer -> lorawan_telemetry`, `telemetry_reader -> lorawan_telemetry`, and `fabric_adapter -> lorawan_telemetry`. Every session reached `10.104.0.4`, reported `pg_is_in_recovery=false`, used TLSv1.3 / TLS_AES_256_GCM_SHA384, and returned the expected user/database pair. Primary-side replication still showed `u1c-01|10.104.0.2|streaming|async` and `u1c-03|10.104.0.8|streaming|async`; Patroni remained `u1c-02` `Leader/running` on timeline 2 with both replicas streaming at reported lag 0. All final role endpoints were HTTP 200; child gate exit code was 0, and the SSH login shell remained alive.

Decision: Phase 6 database-layer commissioning and planned-switchover validation are closed for the current POC boundary. The switchover proved Patroni role transfer, promoted-node PostgreSQL/TimescaleDB usability, application database/owner preservation, rollback-only write ownership, replication reattachment, and real application authentication on the promoted primary. HAProxy/PgBouncer failover-routing behavior is intentionally not claimed yet because those services are Phase 7 and are not deployed. Next: activate Phase 7 with a read-only three-host preflight before installing or binding HAProxy/PgBouncer.

6.16 Failed replica recovery

Do not delete the data directory as a first response. Diagnose disk, network, timeline, WAL availability, and logs.

When reinitialization is approved and the node is confirmed to be a replica:

```
patronictl -c <PATRONI_CONFIG> reinit <PG_SCOPE> <FAILED_REPLICA_NAME>
```

This is destructive to that replica's local data. Confirm the target twice and preserve incident evidence first.

6.17 Final checks

- All three members use the same approved image digest and PostgreSQL major version.
- One primary and two streaming replicas are visible through Patroni and SQL.
- Database and Patroni ports are private-only.
- `chirpstack` and `lorawan_telemetry` both exist on the same Patroni cluster.
- The same pinned TimescaleDB extension build is available on all three PostgreSQL members.
- `timescaledb` is enabled in `lorawan_telemetry`, not in `chirpstack`.
- `telemetry.uplinks` and `telemetry.measurements` are Timescale hypertables; `telemetry.fabric_outbox` is an ordinary PostgreSQL table.
- A Patroni switchover preserves the extension and hypertables without reinstalling them.
- Application roles use TLS and separate credentials.
- Planned switchover succeeds through PgBouncer and HAProxy without application DSN edits for ChirpStack, Node-RED, Grafana, or the Fabric adapters.
- Logical POC dumps exist before destructive failure tests.
- No standalone service can start PostgreSQL outside Patroni.

Next: [07-haproxy-and-pgbouncer.md](#)

Supporting runbook: 07-haproxy-and-pgbouncer.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

7. PgBouncer and HAProxy Core Service Routing

Status: CORE + EVIDENCE AUTH COMPLETE / PASS. HAProxy 2.8.16-0ubuntu0.24.04.3 and PgBouncer 1.22.0-1build4 are commissioned on all three nodes. The stable database client path is client TLS -> local PgBouncer :6432 -> local HAProxy primary route :15432 -> current Patroni leader over verified PostgreSQL TLS. The original four-role SCRAM/failover boundary remains historical PASS, and all three nodes now carry the same authoritative ten-entry/six-evidence SCRAM userlist SHA-256 665f1592c96ca276681a454b9cbcd6ab8ab0cbfb4594b8ddd443239db58df391. Fresh strict verify-full evidence sessions through all three physical PgBouncer endpoints passed. Historical Phase 7 sections that say “exactly four” describe the original commissioning baseline; this current header and the newest evidence-expansion checkpoint supersede them for live userlist count.

7.1 Purpose

The tiny HA POC runs PgBouncer and the private PostgreSQL HAProxy frontend on **all three hosts**. ha-01 and ha-02 are the public-ingress candidates. The original target design reserved ha-03:18883 for Node-RED, but live Phase 9 uses :18883 -> :8885 only on ulc-01/02 for ChirpStack. The refined Phase 12A design instead commissions a separate Node-RED mTLS frontend on ulc-03:18884 -> Mosquitto :8886; treat that later route as authoritative for Node-RED.

```
local database client
-> pgbouncer.internal.lorawan.com:6432
-> PgBouncer on this host
-> postgres-ha.internal:15432
-> HAProxy on this host
-> current Patroni primary on private TCP/5432

Clients include:
ha-01 ChirpStack-1 + Fabric adapter-1
ha-02 ChirpStack-2 + Fabric adapter-2
ha-03 Node-RED + Grafana
```

6432 is the stable local pool endpoint. 15432 is the local HAProxy PostgreSQL-primary endpoint and avoids colliding with PostgreSQL 5432 on the same Droplet. Map the logical service names to the current host's private VPC IP for each local client/container.

This gives every database client the same failover shape without another database router. ha-03 is not a public Reserved-IP candidate; its HAProxy instance is reused for private database routing and, when Phase 12A is activated, the dedicated Node-RED :18884 MQTT route described in [12a-node-red-timescale-telemetry.md](#).

HAProxy on ha-01 and ha-02 also provides the stable private OpenBao KMS route used by the two Fabric adapter workers:

```
Fabric adapter on this app host
-> openbao-kms.internal.lorawan.com:18200
-> HAProxy on this app host
-> one initialized, unsealed OpenBao-1/2/3 API backend on private TCP/8200
```

The KMS frontend stays in TCP mode so application TLS remains end-to-end to OpenBao. OpenBao Integrated Storage handles active/standby behavior and request forwarding inside the three-member cluster; HAProxy's job is only to keep one stable client endpoint and avoid routing to an unusable backend.

7.2 Preconditions

- Patroni cluster has one primary and healthy replicas.
- Patroni REST endpoints are reachable from all three hosts on private port 8008.
- PostgreSQL TLS certificates and CA are installed.
- HAProxy and PgBouncer versions are pinned and supported.
- ChirpStack database role exists.
- Maximum PostgreSQL connections and application concurrency are approved.
- Before enabling the KMS frontend, OpenBao-1/2/3 exist as one initialized Integrated Storage/Raft cluster, all intended usable members are unsealed, and their API certificates validate the approved private KMS identity.

7.2.1 Activation preflight - run before package installation

Run this same read-only gate once on ulc-01, ulc-02, and ulc-03. Do not run apt update, install packages, create certificates, or start services yet. Capture all three outputs first.

```
sudo -v

bash -s <<'EOF'
set -euo pipefail
set +x

NODE="$(hostname)"

case "$NODE" in
    ulc-01) NODE_IP='10.104.0.2' ;;
    ulc-02) NODE_IP='10.104.0.4' ;;
    ulc-03) NODE_IP='10.104.0.8' ;;
    *) echo 'FAIL: run only on ulc-01, ulc-02, or ulc-03'; exit 1 ;;
esac

echo "=== $NODE PHASE 7 HAPROXY/PGBouncer READ-ONLY PREFLIGHT ==="

echo
echo '=== 1. POSTGRES/PATRONI BASELINE ==='
for SPEC in \
    'ulc-02|10.104.0.4|leader' \
    'ulc-01|10.104.0.2|replica' \
    'ulc-03|10.104.0.8|replica'
do
    IFS='|' read -r CHECK_NODE IP ENDPOINT <<< "$SPEC"
    HTTP=$(curl -s -o /dev/null -w '%{http_code}' --max-time 5 \
        "http://$IP:8008/$ENDPOINT" || true)
    echo "$CHECK_NODE /$ENDPOINT = HTTP $HTTP"
    [ "$HTTP" = '200' ] || { echo 'FAIL: Patroni baseline changed'; exit 1; }
done

sudo docker exec -e LC_ALL=C spilo \
    patronictl -c /run/postgres.yml list

echo 'Patroni baseline: PASS'
```

```

echo
echo '=== 2. CURRENT HOST RESOURCE SNAPSHOT ==='
printf 'host = %s\n' "$NODE"
printf 'expected VPC IP = %s\n' "$NODE_IP"
ip -br address show
free -m
df -h /

echo
echo '=== 3. PACKAGE STATE - NO INSTALL ==='
for PKG in haproxy pgbouncer; do
    INSTALLED=$(dpkg-query -W -f='${Status}|${Version}' "$PKG" 2>/dev/null || true)
    if [ -n "$INSTALLED" ]; then
        echo "$PKG installed = $INSTALLED"
    else
        echo "$PKG installed = absent"
    fi

    echo "--- apt policy: $PKG ---"
    apt-cache policy "$PKG" | sed -n '1,12p'
done

echo
echo '=== 4. SERVICE UNIT STATE ==='
for UNIT in haproxy.service pgbouncer.service; do
    LOAD=$(systemctl show "$UNIT" -p LoadState --value 2>/dev/null || true)
    ACTIVE=$(systemctl is-active "$UNIT" 2>/dev/null || true)
    ENABLED=$(systemctl is-enabled "$UNIT" 2>/dev/null || true)
    echo "$UNIT load=${LOAD:-not-found} active=${ACTIVE:-unknown} enabled=${ENABLED:-unknown}"
done

echo
echo '=== 5. PLANNED DATABASE ROUTING PORTS MUST BE FREE ==='
COLLISIONS=$(sudo ss -ltnpH | awk '
    $4 ~ /:6432$/ ||
    $4 ~ /:15432$/ ||
    $4 ~ /:15433$/ { print }
')

if [ -n "$COLLISIONS" ]; then
    printf '%s\n' "$COLLISIONS"
    echo 'FAIL: one or more Phase 7 database-routing ports are already occupied'
    exit 1
fi

echo '6432, 15432, 15433 listener collision check: PASS'

echo
echo '=== 6. POSTGRES CA MATERIAL ==='
PG_CA='/etc/lorawan-pki/postgres/ca.crt'

# /etc/lorawan-pki/postgres is intentionally restrictive. Use privileged
# traversal so an operator outside the service group does not get a false
# "missing" result from a plain shell [ -f ] test.
if ! sudo test -f "$PG_CA"; then
    echo "FAIL: privileged check cannot find $PG_CA"
    exit 1
fi

sudo stat -c '%a|%u|%g|%n' "$PG_CA"
sudo openssl x509 -in "$PG_CA" -noout -subject -issuer -fingerprint -sha256
echo 'PostgreSQL CA: PASS'

echo
echo '=== 7. PGBOUNCER CLIENT-TLS MATERIAL INVENTORY ==='
for FILE in \
    /etc/lorawan-pki/pgbouncer/ca.crt \
    /etc/lorawan-pki/pgbouncer/server.crt \
    /etc/lorawan-pki/pgbouncer/server.key
do
    if sudo test -e "$FILE"; then
        sudo stat -c 'present|%a|%u|%g|%n' "$FILE"
    else

```

```

    echo "not-provisioned|$FILE"
  fi
done

echo
echo '=== 8. EXISTING CONFIG INVENTORY ==='
for FILE in \
  /etc/haproxy/haproxy.cfg \
  /etc/pgbouncer/pgbouncer.ini \
  /etc/pgbouncer/userlist.txt
do
  if sudo test -e "$FILE"; then
    sudo stat -c 'present|%a|%u|%g|%n' "$FILE"
  else
    echo "absent|$FILE"
  fi
done

echo
echo "$NODE PHASE 7 READ-ONLY PREFLIGHT: PASS"
EOF

RC=$?
echo
echo "Child gate exit code = $RC"
echo 'SSH LOGIN SHELL IS STILL ALIVE'

```

Why: Phase 7 introduces three new private listeners on every host. Checking the ports before installation separates a real collision from a later HAProxy/PgBouncer configuration problem. Capturing apt-cache policy before installation records exactly what Ubuntu would install instead of silently assuming a version. The PostgreSQL CA is already required for PgBouncer's verified server connection. PgBouncer's own client-facing certificate bundle may still be not-provisioned; record that state rather than loosening client_tls_sslmode or hostname verification as a shortcut.

Recorded u1c-03 HAProxy canary harness attempt - 2026-08-23: INCONCLUSIVE / NO MUTATION. The database HA baseline passed, then the script exited with code 141 at the package-candidate extraction step before apt-get install was reached. Root cause is shell behavior: with set -o pipefail, apt-cache policy haproxy | awk '/Candidate:/ {print \$2; exit}' lets awk close the pipe as soon as it finds the candidate; apt-cache then receives SIGPIPE and returns 141, making the command substitution fail. This is not evidence against HAProxy 2.8.16-0ubuntu0.24.04.3. Use a parser that consumes the full stream, such as sed -n 's/^ Candidate: //p', and avoid grep -q pipelines where an upstream writer could receive SIGPIPE. Because the failure occurred before installation, no HAProxy package, service, listener, or config mutation was made.

The preflight is PASS when Patroni stays healthy, the three planned database-routing ports are free, and the PostgreSQL CA exists. Package/config/PgBouncer-certificate lines are inventory evidence and can legitimately report absent or not-provisioned at this point. Review all three hosts before installing anything.

7.2.2 Recorded u1c-03 HAProxy database-routing canary - 2026-08-23

PASS. The corrected retry first re-proved the Patroni baseline, the pinned HAProxy package candidate, package absence, and free 15432/15433 ports. apt-get -s showed only haproxy 2.8.16-0ubuntu0.24.04.3 plus liblua5.4-0, with no removals. The exact HAProxy version was then installed. The package-default configuration was retained at /etc/haproxy/phase7-canary-20260823-141310/haproxy.cfg.package-default before the canary configuration replaced /etc/haproxy/haproxy.cfg.

The canary configuration passed offline and installed syntax validation. Ubuntu's HAProxy package post-install automatically created the systemd enablement symlink; the gate then stopped the package-default service before replacing the configuration, started the validated private-only canary, and explicitly confirmed enablement again after runtime checks. ss showed exactly 10.104.0.8:15432 and 10.104.0.8:15433, with no wildcard/public database-routing listener. A verify-full PostgreSQL session through 10.104.0.8:15432 landed on the current primary as 10.104.0.4|10.104.0.8|f|t|TLSv1.3|TLS_AES_256_GCM_SHA384. The replica frontend 10.104.0.8:15433 landed

on ulc-01 as 10.104.0.2|10.104.0.8|t|t|TLSv1.3|TLS_AES_256_GCM_SHA384. Patroni remained ulc-02 leader with ulc-01 and ulc-03 streaming replicas and all expected role endpoints HTTP 200. Child gate exit code was 0. Future node rollouts should explicitly disable --now haproxy immediately after package installation so boot persistence is intentionally restored only after routing validation.

Why this matters: it proves the HAProxy-only database-routing design independently from PgBouncer. PostgreSQL TLS stays end-to-end through HAProxy, Patroni REST decides which backend is eligible, and the local HAProxy source address matches the hardened /32 PostgreSQL HBA rules already commissioned. No PostgreSQL/Patroni configuration changed, and PgBouncer was not installed or started.

Next: repeat this exact database-routing behavior on ulc-01; only after the second replica-host rollout passes should ulc-02 receive HAProxy.

7.2.3 Recorded ulc-01 HAProxy database-routing rollout - 2026-08-23

PASS. The second-host rollout re-proved the expected Patroni topology (ulc-02 /leader, ulc-01 /replica, ulc-03 /replica, all HTTP 200), pinned candidate haproxy 2.8.16-0ubuntu0.24.04.3, package absence, and free 15432/15433 ports. apt-get -s again showed only liblua5.4-0 and the pinned HAProxy package, with no removals. After installation, the package-created systemd enablement was deliberately neutralized with systemctl disable --now haproxy; observed state was active=inactive, enabled=disabled before any custom listener was started. The package-default configuration was backed up at /etc/haproxy/phase7-rollout-20260823-142347/haproxy.cfg.package-default.

The node-specific configuration passed both off-path and installed syntax checks. HAProxy then started in runtime-only state while remaining disabled. ss showed exactly 10.104.0.2:15432 and 10.104.0.2:15433. A verify-full PostgreSQL session through the primary frontend returned

10.104.0.4|10.104.0.2|f|t|TLSv1.3|TLS_AES_256_GCM_SHA384, proving traffic landed on current leader ulc-02 while PostgreSQL saw HAProxy source 10.104.0.2. The replica frontend returned

10.104.0.2|10.104.0.2|t|t|TLSv1.3|TLS_AES_256_GCM_SHA384, which is valid because ulc-01 is itself a healthy replica and may be selected locally for replica traffic. Patroni remained healthy with both replicas streaming and reported lag 0. Only after these runtime checks did the gate explicitly enable HAProxy for boot persistence. Child gate exit code was 0; PgBouncer remained uninstalled and PostgreSQL/Patroni configuration was unchanged.

Why this matters: HAProxy database routing is now proven on two independent replica-side hosts, including the /32 source-address behavior required by the hardened HBA. The final rollout target is ulc-02, the current PostgreSQL leader; its rollout must prove installing HAProxy does not change PostgreSQL leadership, Patroni state, or Spilo container identity.

7.2.4 Recorded ulc-02 current-leader HAProxy rollout - 2026-08-23

PASS. Before mutation, ulc-02 /leader, ulc-01 /replica, and ulc-03 /replica all returned HTTP 200; local pg_is_in_recovery() was f; and the Spilo container identity was 2313de94ee2c5dcc292ac28ea7ae8359fbe31b1ea6ab171276306716cff58762|2026-08-22T14:19:16.323317366Z|0. HAProxy was absent, 15432/15433 were free, the candidate was the pinned 2.8.16-0ubuntu0.24.04.3, and the apt simulation showed only liblua5.4-0 plus HAProxy with no removals.

After exact-version installation, the package-created systemd state was deliberately neutralized to active=inactive, enabled=disabled. PostgreSQL was immediately rechecked and remained leader with pg_is_in_recovery()=f. The package-default config was preserved at /etc/haproxy/phase7-rollout-20260823-143511/haproxy.cfg.package-default. The node-specific config passed off-path and installed syntax checks, then HAProxy started while still disabled. Exact listeners were only 10.104.0.4:15432 and 10.104.0.4:15433.

Primary-route verify-full evidence was 10.104.0.4|10.104.0.4|f|t|TLSv1.3|TLS_AES_256_GCM_SHA384, proving the local HAProxy primary frontend correctly selected the local current leader. Replica-route evidence was 10.104.0.2|10.104.0.4|t|t|TLSv1.3|TLS_AES_256_GCM_SHA384, proving the replica frontend selected a healthy

standby while PostgreSQL saw the hardened /32 HAProxy source 10.104.0.4. The Spilo container identity after rollout was byte-for-byte identical to the pre-rollout identity. Final Patroni state remained u1c-02 Leader/running on timeline 2 with u1c-01 and u1c-03 streaming and reported lag 0. Only after all runtime checks passed was HAProxy enabled. Child gate exit code was 0; PgBouncer remained uninstalled and PostgreSQL/Patroni configuration was unchanged.

Three-node HAProxy database-routing boundary: PASS. Do not add another Patroni switchover merely to prove HAProxy yet. The permanent PgBouncer verification name is `pgbouncer.internal.lorawan.com`.

PgBouncer CA read-only inventory - 2026-08-23: PASS. On u1c-03, `/root/lorawan-pg-ca` remained mode 0700 root:root. The commissioned CA certificate is exactly `/root/lorawan-pg-ca/ca.crt`, subject/issuer CN = LoRaWAN PostgreSQL Internal CA, valid 2026-08-22 through 2036-08-19, with SHA-256 fingerprint 99:00:4B:B3:2D:7D:78:FA:38:61:7C:78:89:6D:7A:7E:FF:9F:A6:10:FC:8F:07:D4:E2:5E:35:25:36:E6:CB:3E. Its certificate public-key SHA-256 is 2da3b7630e38a1e80469c4d5d1f1c4ac9f1125fb47d6c5a4c2d411960340a2f9, and only `/root/lorawan-pg-ca/ca.key` matched that public key. Existing PostgreSQL member keys did not match, as expected. The inventory printed no private-key contents and made no filesystem mutation.

First PgBouncer issuance harness attempt - 2026-08-23: INCONCLUSIVE / NO CERTIFICATE MUTATION. The gate re-proved the CA directory, CA certificate, CA key permissions, commissioned CA fingerprint, and that the CA certificate and CA private key both derive the same public-key SHA-256 2da3b7630e38a1e80469c4d5d1f1c4ac9f1125fb47d6c5a4c2d411960340a2f9. The harness then failed before staging-directory creation because its hard-coded expected hash accidentally omitted the final 9, so the correct observed 64-hex hash did not equal the typoed expectation. Child exit code was 1. No staging directory, node key, CSR, certificate, PgBouncer package, HAProxy change, or PostgreSQL/Patroni change was reached. Correct the expected hash and rerun the same issuance design.

Corrected PgBouncer three-node TLS issuance - 2026-08-23: PASS. The corrected gate used the observed CA public-key SHA-256 ending in `...0a2f9`, re-proved PgBouncer was still absent, hashed the existing `ca.srl`, and created root-only staging directory `/root/lorawan-pg-ca/pgbouncer-issuance-20260823-150532`. It generated one unique RSA-3072 key and CSR per node, signed each certificate with the existing CN = LoRaWAN PostgreSQL Internal CA for 825 days using a random 128-bit serial, and applied `serverAuth` plus exact SANs for `pgbouncer.internal.lorawan.com`, the node hostname, and node VPC IP.

Issued identities:

- u1c-01 / 10.104.0.2: serial 7C5887FBE0338797CAAC8230AD7D89F8, certificate fingerprint A4:EC:DF:86:30:68:29:88:0F:52:05:0A:E1:B7:E5:F9:3E:B3:4A:72:72:81:96:40:BC:10:7A:EC:94:D2:6D:E1, key/certificate public-key SHA-256 ba76dd9cde0722cb1377446837cf6f29fffc38550b42a63667c9f2fd8787fc5.
- u1c-02 / 10.104.0.4: serial 16F56AB3A41FF77DB93EE38EE377164D, certificate fingerprint 3B:25:17:30:2B:FB:26:7D:49:F7:C5:24:C5:B0:47:F6:BF:D1:88:64:8D:FD:0E:05:9B:ED:08:32:A1:50:02:EA, key/certificate public-key SHA-256 3dcd2400c7ed3936aa3fb5aa0360c4e3c366153d597edbc20ec6243ec09c68cf.
- u1c-03 / 10.104.0.8: serial 058F982D16D50B7AE8FF266ACFFBCBBE, certificate fingerprint BC:50:50:8D:FC:60:37:5E:E8:B6:A0:3B:93:41:D7:AB:53:CB:BD:C7:E1:86:3B:1D:FD:AD:D8:94:9E:45:7F:6E, key/certificate public-key SHA-256 4784b3d7c22993eb5e5577b136962a29902d504f59aae71cc2c9de733ab0a716.

Every certificate passed CA-chain verification, `-purpose sslserver`, `-verify_hostname` `pgbouncer.internal.lorawan.com`, node-hostname verification, node-IP verification, and private-key/certificate public-key equality. Unique key, certificate-fingerprint, and serial counts were all 3. Existing `ca.srl` SHA-256 stayed 966208f925453dbbfa43947a1ad8051097569f3c765cba0ed217b48a8dfe54e6 before and after. The CA private key was not copied into any node bundle; PgBouncer remained uninstalled; HAProxy and PostgreSQL/Patroni were unchanged; child gate exit code was 0.

Exact PgBouncer package read-only inspection on u1c-03 - 2026-08-23: PASS. The staged u1c-03 certificate fingerprint remained BC:50:50:8D:FC:60:37:5E:E8:B6:A0:3B:93:41:D7:AB:53:CB:BD:C7:E1:86:3B:1D:FD:AD:D8:94:9E:45:7F:6E and still

verified for `pgbouncer.internal.lorawan.com`. PgBouncer remained absent and port 6432 free. Ubuntu Noble candidate 1.22.0-1build4 matched the pin; apt simulation showed PgBouncer plus its normal dependencies with no removals. The exact `.deb` was downloaded and extracted off-path only. Its systemd unit is `/usr/lib/systemd/system/pgbouncer.service`, with `User=postgres`, no explicit `Group=`, and `ExecStart=/usr/sbin/pgbouncer /etc/pgbouncer/pgbouncer.ini`. The package declares `/etc/pgbouncer/pgbouncer.ini` and `/etc/pgbouncer/userlist.txt` as `conffiles`. Its `postinst` uses Debian/systemd helpers that enable new installations and attempt to start/restart the service, so the install canary must suppress service starts before `apt-get install`; merely planning to stop the service afterward is not enough. The inspection installed nothing, created no listener, and child exit code was 0.

Why this changes the install order: do not guess a pgbouncer Unix account. The exact package runs as host user `postgres`. Because `Group=` is absent, systemd uses the service user's normal group context; the install canary must discover the actual host `postgres` primary group after package dependencies are configured, then install `/etc/lorawan-pki/pgbouncer` as root-owned with only that real service group able to traverse/read the key. Keep the package-default PgBouncer configuration stopped and disabled until the TLS, auth file, logical DNS mapping, and HAProxy upstream settings are validated off-path.

ulc-03 PgBouncer package + local TLS install canary - 2026-08-23: PASS. The gate re-proved Patroni role endpoints HTTP 200, HAProxy active with exact listeners `10.104.0.8:15432` and `10.104.0.8:15433`, and Spilo identity `7e1c213d1694f37aa08bf2a996a64947b207df5ca9c4366ff0debfb8f2bb123|2026-08-22T14:32:18.26248358Z|0`. The staged `ulc-03` certificate fingerprint and key hash matched the issued identity. A temporary `/usr/sbin/policy-rc.d` returned 101, so package configuration did not start `ssl-cert.service`, `postgresql.service`, or `pgbouncer.service`. The exact PgBouncer 1.22.0-1build4 package installed with its dependency set. PgBouncer was observed inactive but package-enabled, then explicitly disabled to inactive/disabled; the temporary package policy was restored/removed afterward.

The installed package created host service identity `uid=110(postgres) gid=114(postgres)`, with supplementary group `ssl-cert` GID 113; systemd still declares `User=postgres` and no explicit `Group=`. `pg_lsclusters` returned no host PostgreSQL cluster. Package-default files were backed up under `/etc/pgbouncer/phase7-package-default-20260823-151910`; observed packaged modes were `0640 postgres:postgres` for `pgbouncer.ini` and `userlist.txt`, and `0644 root:root` for `/etc/default/pgbouncer`. The local TLS directory `/etc/lorawan-pki/pgbouncer` is `0750 root:postgres`; `ca.crt`, `server.crt`, and `server.key` are each `0640 root:postgres`. The `postgres` service identity can read all three while `opsadmin` cannot read `server.key`. Installed certificate fingerprint, chain, shared logical hostname, node hostname, node IP, and key/certificate public-key equality all re-verified successfully. PgBouncer remained inactive/disabled with no `:6432` listener, Spilo identity stayed byte-for-byte unchanged, HAProxy stayed active, and all Patroni role endpoints remained HTTP 200.

ulc-03 PgBouncer dependency-service collateral check - 2026-08-23: PASS. PgBouncer remained inactive/disabled with no `:6432` listener. `postgresql.service` is loaded, inactive, enabled, `Type=oneshot`, and its unit is only the PostgreSQL meta-service with `ExecStart=/bin/true`; `pg_lsclusters` returned none, no `postgresql@*.service` instance was loaded or running, and `/etc/postgresql` contains only the package-created top-level directory. `ssl-cert.service` is also loaded, inactive, enabled, `Type=oneshot`; its only purpose is generating the default snakeoil certificate when `/etc/ssl/private/ssl-cert-snakeoil.key` is absent. The temporary `policy-rc.d` is absent after restoration. PgBouncer TLS permissions remain `0750 root:postgres` for the directory and `0640 root:postgres` for `ca.crt`, `server.crt`, and `server.key`; the service identity can still read the key. HAProxy listeners remain exactly `10.104.0.8:15432` and `10.104.0.8:15433`, and Patroni role endpoints remain HTTP 200 for `ulc-02 /leader`, `ulc-01 /replica`, and `ulc-03 /replica`. Child gate exit code was 0 and the SSH shell remained alive.

ulc-03 PgBouncer dependency-service hygiene - 2026-08-23: PASS. `postgresql.service` and `ssl-cert.service` were disabled and remained inactive; neither package was uninstalled and neither unit was masked. `pg_lsclusters` still returned no host cluster, no running `postgresql@*.service` existed, and the existing `/etc/ssl/private/ssl-cert-snakeoil.key` was only inventoried (`0640 root:ssl-cert`) rather than deleted. PgBouncer stayed inactive/disabled with no `:6432` listener. `/etc/lorawan-pki/pgbouncer` remained `0750 root:postgres` and its `CA/certificate/key` remained `0640 root:postgres`, with the `postgres` service identity still able to read the private key. HAProxy listeners stayed exactly `10.104.0.8:15432` and `10.104.0.8:15433`; all expected

Patroni role endpoints remained HTTP 200; and Spilo identity remained byte-for-byte unchanged at 7e1c213d1694f37aa08bf2a996a64947b207df5ca9c4366ff0debfa8f2bb123|2026-08-22T14:32:18.26248358Z|0. Child gate exit code was 0.

Decision: package-side cleanup is complete on the canary. Recheck these disabled dependency units after future PgBouncer/postgresql-common package upgrades. Keep PgBouncer stopped while the next read-only gate proves postgres-ha.internal resolves to the local HAProxy address on ulc-03 and confirms the four intended application roles have non-null SCRAM-SHA-256 verifiers. Only then create the protected userList.txt and off-path PgBouncer configuration.

7.3 Install and inspect

```
haproxy -vv
pgbouncer --version
systemctl cat haproxy
systemctl cat pgbouncer
```

Use supported distribution packages or pinned artifacts that match the operating-system release. Keep the observed HAProxy and PgBouncer versions with the package source and previous package reference because configuration directives and rollback behavior can differ between releases.

Observed on 2026-08-23: the initial preflight found HAProxy and PgBouncer absent on all three nodes. Ubuntu Noble offers HAProxy 2.8.16-0ubuntu0.24.04.3 from noble-updates / noble-security and PgBouncer 1.22.0-1build4 from noble/universe. HAProxy is installed and live-validated on all three nodes. Three unique PgBouncer server identities for pgbouncer.internal.lorawan.com are issued and verified. PgBouncer is installed only on canary node ulc-03, where it remains stopped/disabled with no :6432 listener and its verified local certificate/key/CA are installed under /etc/lorawan-pki/pgbouncer; ulc-01 and ulc-02 still have no PgBouncer package or installed PgBouncer private key. On ulc-03, the package-created postgresql.service and ssl-cert.service are explicitly inactive/disabled, no host PostgreSQL cluster exists, and no dependency package was removed. Continue with logical-name resolution and protected SCRAM-verifier preflight before writing or starting the PgBouncer configuration.

7.4 HAProxy configuration

Back up /etc/haproxy/haproxy.cfg, then configure private database frontends. Replace IPs and certificate behavior with the approved design.

```
global
    log /dev/log local0
    log /dev/log local1 notice
    user haproxy
    group haproxy
    daemon
    maxconn <HAPROXY_MAX_CONNECTIONS>

    ssl-default-bind-options ssl-min-ver TLSv1.2
    ssl-default-server-options ssl-min-ver TLSv1.2

defaults
    log global
    mode tcp
    option tcplog
    timeout connect 5s
    timeout client 60s
    timeout server 60s
    timeout check 5s

frontend postgres_primary
```

```

bind <THIS_HOST_PRIVATE_IP>:15432
default_backend patroni_primary

backend patroni_primary
    option httpchk GET /primary
    http-check expect status 200
    default-server inter 2s fall 3 rise 2 on-marked-down shutdown-sessions
    server ha-01 <HA01_PRIVATE_IP>:5432 check port 8008
    server ha-02 <HA02_PRIVATE_IP>:5432 check port 8008
    server ha-03 <HA03_PRIVATE_IP>:5432 check port 8008

frontend postgres_replicas
    bind <THIS_HOST_PRIVATE_IP>:15433
    default_backend patroni_replicas

backend patroni_replicas
    balance roundrobin
    option httpchk GET /replica?lag=<APPROVED_MAX_REPLICA_LAG>
    http-check expect status 200
    default-server inter 3s fall 3 rise 2
    server ha-01 <HA01_PRIVATE_IP>:5432 check port 8008
    server ha-02 <HA02_PRIVATE_IP>:5432 check port 8008
    server ha-03 <HA03_PRIVATE_IP>:5432 check port 8008

```

Patroni /primary returns HTTP 200 only for the member that is primary and owns the leader lock. Replica checks can include a maximum lag. Verify endpoint semantics against the pinned Patroni version.

If Patroni REST uses TLS or authentication, configure HAProxy accordingly; do not disable REST protection to simplify health checks.

OpenBao KMS frontend

Add a private TCP frontend on each adapter host:

```

frontend openbao_kms
    mode tcp
    bind <THIS_HOST_PRIVATE_IP>:18200
    default_backend openbao_nodes

backend openbao_nodes
    mode tcp
    balance roundrobin
    option httpchk GET /v1/sys/health?standbyok=true
    http-check expect status 200
    default-server inter 2s fall 3 rise 2

    # Client traffic remains raw TLS pass-through because the server lines do
    # not enable `ssl` for normal proxied traffic. `check-ssl` encrypts only
    # the HAProxy health check.
    server openbao-1 <HA01_PRIVATE_IP>:8200 check check-ssl verify required ca-file /etc/lorawan-
pki/openbao/ca.crt check-sni openbao-kms.internal.lorawan.com
    server openbao-2 <HA02_PRIVATE_IP>:8200 check check-ssl verify required ca-file /etc/lorawan-
pki/openbao/ca.crt check-sni openbao-kms.internal.lorawan.com
    server openbao-3 <HA03_PRIVATE_IP>:8200 check check-ssl verify required ca-file /etc/lorawan-
pki/openbao/ca.crt check-sni openbao-kms.internal.lorawan.com

```

Validate that the pinned HAProxy version supports the shown check-ssl and check-sni syntax before reload. If it differs, adapt only the syntax, not the behavior: the health check must use HTTPS, verify the OpenBao CA/name, call /v1/sys/health?standbyok=true, and accept only an initialized/unsealed usable backend.

Do **not** accept plain TCP-connect-only health as the final KMS test. OpenBao's health endpoint can distinguish initialized/unsealed active or standby nodes from sealed/uninitialized nodes.

Why standby is valid here: an unsealed OpenBao HA standby can forward requests to the active node. A sealed node is not usable redundancy and must not be considered a healthy adapter backend.

7.5 Validate HAProxy before reload

```
sudo haproxy -c -V -f /etc/haproxy/haproxy.cfg
sudo systemctl reload haproxy
sudo systemctl status haproxy --no-pager -l
sudo ss -lntp | grep -E ':(15432|15433|18200|18883)\b'
```

The PostgreSQL HAProxy listeners bind only to each host's private interface. The OpenBao KMS listener exists only on ha-01/02, also private. Live Phase 9 uses private ChirpStack MQTT :18883 on u1c-01/02; Phase 12A later adds the separate private Node-RED MQTT frontend :18884 on u1c-03. None of these private listeners is exposed directly to the Internet.

Check each Patroni endpoint directly from the current host:

```
for host in <DB_01_PRIVATE_IP> <DB_02_PRIVATE_IP> <DB_03_PRIVATE_IP>; do
    printf '%s ' "$host"
    curl -sS -o /dev/null -w '%{http_code}\n' "http://$host:8008/primary"
done
```

Exactly one should return 200. Adapt for HTTPS and client certificates when enabled.

7.6 PostgreSQL routing test

```
psql 'host=postgres-ha.internal hostaddr=<THIS_HOST_PRIVATE_IP> port=15432 dbname=postgres user=
<MONITOR_ROLE> sslmode=verify-full' \
-c "SELECT inet_server_addr(), pg_is_in_recovery();"
```

Expected: pg_is_in_recovery = false.

For the replica frontend:

```
psql 'host=postgres-ha.internal hostaddr=<THIS_HOST_PRIVATE_IP> port=15433 dbname=postgres user=
<MONITOR_ROLE> sslmode=verify-full' \
-c "SELECT inet_server_addr(), pg_is_in_recovery();"
```

Expected: true. Do not send ChirpStack writes to the replica endpoint.

7.7 PgBouncer baseline

Create /etc/pgbouncer/pgbouncer.ini and protect the authentication material.

```
[databases]
chirpstack = host=postgres-ha.internal port=15432 dbname=chirpstack
lorawan_telemetry = host=postgres-ha.internal port=15432 dbname=lorawan_telemetry

[pgbouncer]
listen_addr = <THIS_HOST_PRIVATE_IP>
listen_port = 6432
unix_socket_dir = /run/postgresql

pool_mode = session

# Tiny POC starting limits. These are intentionally small because PostgreSQL
# itself starts with max_connections=40 and only a few sensors are active.
max_client_conn = 50
default_pool_size = 3
min_pool_size = 0
```

```

reserve_pool_size = 1
reserve_pool_timeout = 5
max_db_connections = 8

server_connect_timeout = 5
server_login_retry = 5
server_lifetime = 3600
server_idle_timeout = 300
server_fast_close = 1
client_login_timeout = 30
query_wait_timeout = 15

server_tls_sslmode = verify-full
server_tls_ca_file = /etc/lorawan-pki/pgbouncer/ca.crt

client_tls_sslmode = require
client_tls_cert_file = /etc/lorawan-pki/pgbouncer/server.crt
client_tls_key_file = /etc/lorawan-pki/pgbouncer/server.key

auth_type = scram-sha-256
auth_file = /etc/pgbouncer/userlist.txt

admin_users = <PGBOUNCER_ADMIN_ROLE>
stats_users = <PGBOUNCER_STATS_ROLE>

log_connections = 1
log_disconnections = 1
log_pooler_errors = 1
stats_period = 60

```

Use the PgBouncer bundle's `ca.crt` for `server_tls_ca_file` as well as distributing that CA to clients. It is byte-for-byte the same commissioned internal CA that signed PostgreSQL, but it lives under the host PgBouncer service's own readable PKI directory. Do **not** loosen `/etc/lorawan-pki/postgres` just to let the host postgres account traverse it; that directory's numeric ownership was commissioned for the Spilo/container boundary and maps differently on the host. Keeping the public CA copy in `/etc/lorawan-pki/pgbouncer/ca.crt` preserves both boundaries while `verify-full` still validates `postgres-ha.internal`.

Start with session pooling because it preserves connection-level behavior. Transaction pooling can reduce server connections further but may break applications that rely on session state, temporary tables, advisory locks, certain prepared-statement behavior, or SET persistence. Enable it only after ChirpStack version-specific staging tests.

7.8 Authentication choices

Option A: protected auth file

Use SCRAM verifier strings, not plaintext passwords. Keep `/etc/pgbouncer/userlist.txt` mode 640 or stricter and owned by the PgBouncer service group.

Rotation sequence:

1. change PostgreSQL role secret in a controlled window;
2. update the PgBouncer verifier atomically;
3. run `RELOAD`;
4. reconnect affected pools;
5. verify application recovery;
6. remove old secret from the approved store.

Option B: auth_query

Use a dedicated non-login or tightly restricted authentication role and a reviewed SECURITY DEFINER function owned by a trusted administrator. The function must have a fixed search_path, expose only username and password verifier, and grant execution only to the PgBouncer auth role.

Do not grant the PgBouncer process general access to pg_shadow, superuser, or application tables.

7.9 POC pool sizing

Do not size PgBouncer for the future fleet yet. The POC starts with default_pool_size=3, reserve_pool_size=1, and max_db_connections=8 on each host while PostgreSQL starts at max_connections=40.

Why so small: a few sensors do not need dozens of simultaneous database server sessions. PgBouncer is present to prove the future connection-pooling boundary, not to generate artificial connection load.

Watch SHOW POOLS during failover. If clients wait under the actual POC workload, increase the pool gradually and record the reason. Do not raise PostgreSQL and PgBouncer limits preemptively.

7.10 Start and validate PgBouncer

```
sudo pgbouncer -t /etc/pgbouncer/pgbouncer.ini 2>/dev/null || true
sudo systemctl restart pgbouncer
sudo systemctl status pgbouncer --no-pager -l
sudo ss -lntp | grep ':6432'
```

Use the validation option supported by the installed version; inspect pgbouncer --help first.

Test both logical databases through the local pool:

Use the shared certificate name for hostname verification while forcing the connection to this host's private IP:

```
psql 'host=pgbouncer.internal.lorawan.com hostaddr=<THIS_HOST_PRIVATE_IP> port=6432 dbname=chirpstack
user=chirpstack sslmode=verify-full sslrootcert=/etc/lorawan-pki/pgbouncer/ca.crt' \
-c "SELECT current_database(), inet_server_addr(), pg_is_in_recovery();"

psql 'host=pgbouncer.internal.lorawan.com hostaddr=<THIS_HOST_PRIVATE_IP> port=6432 dbname=lorawan_telemetry
user=telemetry_reader sslmode=verify-full sslrootcert=/etc/lorawan-pki/pgbouncer/ca.crt' \
-c "SELECT current_database(), inet_server_addr(), pg_is_in_recovery();"
```

Run the tests appropriate to each host: ChirpStack on ha-01/02; telemetry clients on ha-03; adapter outbox access on ha-01/02. PgBouncer stays on the host's restricted private interface. Its server connection through HAProxy to PostgreSQL must use verified TLS.

7.11 Administration and metrics

Connect to the virtual pgbouncer database with the stats/admin role:

```
SHOW VERSION;
SHOW CONFIG;
SHOW DATABASES;
SHOW POOLS;
SHOW CLIENTS;
SHOW SERVERS;
SHOW STATS;
```

Alert when:

- `cl_waiting` remains above zero;
- `maxwait` increases;
- server login failures occur;
- pools approach `max_db_connections`;
- frequent server disconnects follow database failover;
- file descriptors or memory approach limits.

7.12 Planned database maintenance

For a controlled Patroni switchover:

1. confirm PgBouncer is healthy on all three hosts;
2. observe `SHOW POOLS`;
3. perform Patroni switchover;
4. allow each local HAProxy to select the new primary;
5. issue `RECONNECT chirpstack`; or `RECONNECT lorawan_telemetry`; only if old server connections remain;
6. verify a ChirpStack DB query, one Node-RED telemetry insert, and one Grafana read through the unchanged endpoints;
7. when the reviewed Fabric adapter is deployed, also verify one adapter outbox query; otherwise verify the outbox directly and record adapter execution as BLOCKED.

7.13 Failover validation

During a staging primary failure:

- HAProxy must mark the old primary down;
- one replica must be promoted by Patroni;
- HAProxy must route new connections only to the promoted primary;
- ChirpStack must reconnect through HAProxy; if PgBouncer is enabled, it must discard or replace broken server connections;
- ChirpStack must retry and recover without manual DSN changes;
- the test transaction sequence must match the selected RPO mode.

7.14 Final checks

- The private PostgreSQL HAProxy frontend and PgBouncer validate on all three hosts.
- Public ChirpStack/MQTT HAProxy routing remains only on `ha-01/02`, with public listeners bound to each host's anchor IP and reached through the single Reserved IPv4.
- Exactly one PostgreSQL primary is routed by every local 15432 frontend.
- Both `chirpstack` and `lorawan_telemetry` exist in PgBouncer.
- PgBouncer uses the intentionally small POC limits and shows no sustained client wait under the few-sensor workload.
- Patroni changes recover without editing ChirpStack, Node-RED, or Grafana database endpoints; when Fabric adapters are deployed, their endpoint remains unchanged too.

7.15 Phase 7 validation progress record

Current POC validation progress:

Completed:

- PgBouncer TLS listener validation on ulc-01.
- PgBouncer package, TLS, SCRAM, configuration, and runtime validation completed on ulc-02.
- SCRAM authentication validation for:
 - chirpstack.
 - fabric_adapter.
 - telemetry_reader.
 - telemetry_writer.
- Verified PgBouncer routes successful sessions through the PostgreSQL HAProxy primary endpoint.
- Verified returned backend state is the writable PostgreSQL leader (`pg_is_in_recovery() = false`).
- Verified backend TLS encryption from PgBouncer to PostgreSQL using TLS 1.3.
- Verified PgBouncer idle timeout behavior after the 75-second regression test without connection failure.
- Verified HAProxy 360-second PostgreSQL timeout ordering remains above PgBouncer `server_idle_timeout = 300s`.

Troubleshooting note:

- PgBouncer admin console access is separate from application database users.
- A failed `SHOW POOLS` login does not indicate PostgreSQL failure. It only means the PgBouncer admin identity is not configured or the supplied admin credential is invalid.
- When generating or checking PgBouncer administrative access, perform PostgreSQL role operations against the current Patroni leader only. Running `CREATE ROLE` on a replica will fail with `cannot execute CREATE ROLE in a read-only transaction`.

Next validation:

1. Configure or verify the PgBouncer admin/stats identities.
2. Run `SHOW POOLS`, `SHOW SERVERS`, and `SHOW STATS`.
3. Perform session reuse and failover connection validation before enabling PgBouncer at boot.

Latest ULC-02 commissioning evidence:

- PgBouncer service enabled for boot startup.
- PgBouncer active state verified after enabling.
- Private listener confirmed on `10.104.0.4:6432`.
- Client TLS certificate validation succeeded using the PgBouncer certificate identity.
- TLS negotiation verified with TLS 1.3 and `TLS_AES_256_GCM_SHA384`.
- Application database connectivity validated through PgBouncer for:
 - chirpstack.
 - fabric_adapter.
 - telemetry_reader.
 - telemetry_writer.
- PgBouncer backend TLS connections successfully established to PostgreSQL through local HAProxy.
- The observed server `conn crashed? (age=60s)` messages occurred during the idle timeout regression period and were followed by successful reconnections; the 75-second idle regression test completed

successfully.

7.16 PgBouncer administration console validation

The PgBouncer admin console is not enabled by default. `admin_users` and `stats_users` must be explicitly configured before running `SHOW POOLS`, `SHOW SERVERS`, or `SHOW STATS`.

Application database users and PgBouncer administration users are separate identities.

Before enabling administrative access:

1. Decide the dedicated PgBouncer admin identity.
2. Configure `admin_users` and/or `stats_users` in `pgbouncer.ini`.
3. Reload PgBouncer.
4. Validate only the intended administrative commands.

Do not use PostgreSQL application users as a substitute for PgBouncer admin access.

7.17 Multi-node PgBouncer commissioning record

The production-style POC deployment uses one PgBouncer instance on each PostgreSQL HAProxy node.

Commissioning status:

Node	Private IP	PgBouncer	TLS	SCRAM auth	Backend test
ulc-01	10.104.0.2	commissioned	verified	verified	passed
ulc-02	10.104.0.4	commissioned	verified	verified	passed
ulc-03	10.104.0.8	commissioned	verified	verified	passed

For each node:

1. Install the pinned PgBouncer package.
2. Keep the service stopped until TLS, authentication, and configuration are installed.
3. Install the node certificate, private key, and CA bundle under `/etc/lorawan-pki/pgbouncer`.
4. Generate `/etc/pgbouncer/userlist.txt` from PostgreSQL SCRAM verifiers.

Use a direct `docker compose exec spilo psql` extraction when generating the file. Avoid deeply nested shell quoting around SQL because it can silently produce an empty `userlist`. Validate that exactly four SCRAM entries exist before installation: `- chirpstack - fabric_adapter - telemetry_reader - telemetry_writer`

5. Install the local PgBouncer configuration using the node private IP for `listen_addr`.
6. Validate TLS before opening the listener.
7. Test application database login through 6432.
8. Enable the service only after successful validation.

The ULC-03 foundation stage originally completed with the package installed but intentionally stopped while its TLS/SCRAM configuration was prepared. That historical checkpoint is superseded by section 7.21: ULC-03 was subsequently commissioned with the same validated TLS, SCRAM, backend-routing, and enablement sequence used by ULC-01 and ULC-02.

7.18 Phase 7 ULC-03 TLS deployment record

ULC-03 PgBouncer TLS material installation was validated before PgBouncer configuration.

Validation completed:

- Source certificate bundle located from the approved issuance directory.
- /etc/lorawan-pki/pgbouncer created with ownership root:postgres.
- CA certificate installed.
- PgBouncer server certificate installed.
- PgBouncer private key installed.
- TLS files protected with mode 640.
- PostgreSQL service account access verified.
- Certificate chain verification passed.
- Certificate hostname verification passed for pgbouncer.internal.lorawan.com.
- Certificate public key and private key public key hashes matched.

7.19 Phase 7 ULC-03 PgBouncer SCRAM and configuration workflow

ULC-03 follows the same protected authentication workflow validated on ULC-01 and ULC-02.

Operational commands must be recorded with the deployment procedure because PgBouncer authentication depends on PostgreSQL SCRAM verifier extraction, not plaintext passwords.

SCRAM extraction workflow:

```
sudo docker compose \\  
-f /etc/lorawan-cloud/spilo/compose.yml \\  
exec -T spilo \\  
psql \\  
-X \\  
-U postgres \\  
-d postgres \\  
-A \\  
-t \\  
-v ON_ERROR_STOP=1 \\  
-c "  
SELECT  
  '\"' || rolname || '\" \"' || rolpassword || '\"'  
FROM pg_authid  
WHERE rolname IN (  
  'chirpstack',  
  'fabric_adapter',  
  'telemetry_reader',  
  'telemetry_writer'  
)  
AND rolcanlogin  
AND rolpassword LIKE 'SCRAM-SHA-256\\$%'  
ORDER BY rolname;  
" |  
sudo tee /run/pgbouncer-userlist.final >/dev/null
```

Install the generated verifier file:

```
sudo install \\  
-m 640 \\  
-o root \\  
-g postgres \\  
/run/pgbouncer-userlist.final /etc/pgbouncer/
```

```
/run/pgbouncer-userlist.final \\  
/etc/pgbouncer/userlist.txt
```

Install and verify the node-specific PgBouncer configuration:

```
sudo install \  
-m 640 \  
-o root \  
-g postgres \  
/tmp/pgbouncer-ulc03.ini \  
/etc/pgbouncer/pgbouncer.ini  
  
sudo grep -E \  
'^(listen_addr|listen_port|pool_mode|max_client_conn|default_pool_size|max_db_connections|server_idle_timeout  
|server_tls_sslmode|server_tls_ca_file|client_tls_sslmode|client_tls_cert_file|client_tls_key_file|auth_type|  
auth_file)[[:space:]]*=' \  
/etc/pgbouncer/pgbouncer.ini
```

Validation requirements:

- Exactly four SCRAM entries must exist.
- Entries must be `chirpstack`, `fabric_adapter`, `telemetry_reader`, and `telemetry_writer`.
- `/etc/pgbouncer/userlist.txt` must remain unreadable by `opsadmin`.
- PostgreSQL service account must be able to read the file.

The same installation boundary is used for ULC-01, ULC-02, and ULC-03 before enabling PgBouncer services.

7.20 Phase 7 ULC-03 pre-start validation record

ULC-03 PgBouncer configuration reached the pre-start validation boundary.

Validated:

- PgBouncer configuration installed with node-local listener:
- `listen_addr` = `10.104.0.8`
- `listen_port` = `6432`
- TLS material:
- CA certificate readable by PostgreSQL service account.
- Server certificate readable by PostgreSQL service account.
- Private key readable by PostgreSQL service account.
- Certificate chain verification passed.
- SCRAM authentication file:
- `/etc/pgbouncer/userlist.txt` installed.
- Four SCRAM verifier entries present.
- File ownership and permissions validated.
- HAProxy PostgreSQL endpoints available:
- `10.104.0.8:15432`
- `10.104.0.8:15433`
- Local resolver mapping validated:
- `postgres-ha.internal -> 10.104.0.8`

Historical checkpoint: these were the next ULC-03 steps at the time this pre-start record was captured. They were subsequently completed successfully. Section 7.21 contains the authoritative final three-node state.

7.21 Phase 7 Complete PgBouncer HA Commissioning Record

Objective

Deploy PgBouncer on all PostgreSQL HA nodes as the stable client connection layer while keeping Patroni responsible for PostgreSQL leader election and HAProxy responsible for primary routing.

Final architecture:

```
Application clients
  |
  v
PgBouncer :6432
  |
  v
HAProxy :15432
  |
  v
Patroni PostgreSQL primary
  |
  v
Current leader
```

PgBouncer does not replace Patroni or HAProxy. It provides connection pooling and TLS termination for database clients while HAProxy continues selecting the writable PostgreSQL leader.

Phase 7 Execution Record

Step 1 - PgBouncer package installation

Performed on:

- ULC-01
- ULC-02
- ULC-03

Validation:

```
pgbouncer --version
systemctl is-active pgbouncer
systemctl is-enabled pgbouncer
ss -H -lnt | grep ':6432'
```

Initial safety requirement:

```
PgBouncer installed
PgBouncer stopped
PgBouncer disabled
No :6432 listener exposed
```

The service remained disabled until TLS, authentication, and configuration validation completed.

Step 2 - TLS deployment

TLS materials were installed separately on each node:

```
/etc/lorawan-pki/pgbouncer/  
├── ca.crt  
├── server.crt  
└── server.key
```

Permissions:

```
Directory:  
750 root:postgres  
  
Files:  
640 root:postgres
```

Validation performed:

```
openssl verify \  
-CAfile /etc/lorawan-pki/pgbouncer/ca.crt \  
-verify_hostname pgbouncer.internal.lorawan.com \  
/etc/lorawan-pki/pgbouncer/server.crt
```

Required result:

```
server.crt: OK
```

Certificate and private key matching was verified by comparing public key hashes.

Step 3 - SCRAM authentication installation

PgBouncer uses PostgreSQL SCRAM verifiers.

Plaintext database passwords are not stored by PgBouncer.

The userlist contains:

```
chirpstack  
fabric_adapter  
telemetry_reader  
telemetry_writer
```

Generated file:

```
/etc/pgbouncer/userlist.txt
```

Permissions:

```
640 root:postgres
```

Validation:

```
sudo wc -l /etc/pgbouncer/userlist.txt
```

Expected:

```
4
```

Required access model:

```
postgres user    -> read allowed
opsadmin user    -> read denied
```

Step 4 - PgBouncer configuration

Each node uses its own private IP.

Configuration pattern:

```
listen_addr = <NODE_PRIVATE_IP>
listen_port = 6432

pool_mode = session

max_client_conn = 50
default_pool_size = 3
max_db_connections = 8

server_tls_sslmode = verify-full
server_tls_ca_file = /etc/lorawan-pki/pgbouncer/ca.crt

client_tls_sslmode = require
client_tls_cert_file = /etc/lorawan-pki/pgbouncer/server.crt
client_tls_key_file = /etc/lorawan-pki/pgbouncer/server.key

auth_type = scram-sha-256
auth_file = /etc/pgbouncer/userlist.txt
```

Databases exposed:

```
chirpstack
lorawan_telemetry
```

Step 5 - Runtime TLS validation

Every node passed:

```
openssl s_client \
-starttls postgres \
-connect <NODE_IP>:6432 \
-CAfile /etc/lorawan-pki/pgbouncer/ca.crt \
-verify_hostname pgbouncer.internal.lorawan.com
```

Verified:

```
TLSv1.3
TLS_AES_256_GCM_SHA384
Certificate verification OK
```

Step 6 - Application database validation

Validated users:

```
chirpstack
fabric_adapter
telemetry_reader
telemetry_writer
```

Validation query:

```
SELECT
  current_user,
  inet_server_addr(),
  pg_is_in_recovery();
```

Expected:

```
pg_is_in_recovery = false
```

Successful routing path:

```
PgBouncer
|
v
HAProxy :15432
|
v
Patroni leader PostgreSQL
```

Phase 7 Failover Validation

Controlled Patroni switchover

Initial state:

```
ulc-02
10.104.0.4
Leader
```

Switchover command:

```
Primary: ulc-02
Candidate: ulc-01
Time: now
```

Result:

```
ulc-01
10.104.0.2
Leader

ulc-02
10.104.0.4
Replica

ulc-03
10.104.0.8
Replica
```

Validation:

```
curl -s http://10.104.0.2:8008/
curl -s http://10.104.0.4:8008/
curl -s http://10.104.0.8:8008/
```

Confirmed:

```
Exactly one primary exists.
Replication lag = 0.
```

PgBouncer Failover Routing Test

After Patroni promotion:

Client connection through PgBouncer returned:

```
current_user = chirpstack
inet_server_addr = 10.104.0.2
pg_is_in_recovery = false
```

Meaning:

```
Application
|
v
PgBouncer
|
v
HAProxy
|
v
New Patroni leader
|
v
ulc-01 PostgreSQL
```

No application connection string changes were required.

No PgBouncer restart was required.

No PostgreSQL restart was required.

Final Phase 7 State

Node	IP	PgBouncer	TLS	SCRAM	HA Routing
ULC-01	10.104.0.2	Active + Enabled	PASS	PASS	PASS
ULC-02	10.104.0.4	Active + Enabled	PASS	PASS	PASS
ULC-03	10.104.0.8	Active + Enabled	PASS	PASS	PASS

Final verified capabilities:

- PgBouncer deployed on all PostgreSQL HA nodes.
- TLS encryption enabled between clients and PgBouncer.
- TLS encryption enabled between PgBouncer and PostgreSQL.
- SCRAM authentication enabled.
- HAProxy PostgreSQL routing preserved.
- Patroni leader changes handled successfully.
- Database clients continued operating after failover.

Phase 7 status:

COMPLETE

Next: [08-mqtt-and-valkey.md](#)

Supporting runbook: 13-backup-restore-and-disaster-recovery.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

13. Minimal POC Backup and Recovery

Status: 13A FAST-PATH PASS / 13S SERVER-ONLY SNAPSHOT COMPLETE + LOCALLY VERIFIED / 13B FINAL LATER. Phase 13A protects the pre-cutover cloud state. Interim **13S server-only snapshot/export** has already captured and locally verified the commissioned server baseline, including the protected PostgreSQL/etcd/OpenBao and reconstruction artifacts described below. **13S is not the final Phase 13B backup set.** Phase 13B must still be recreated after every required pre-test service and real application path are commissioned. Off-host copy and isolated restore proof remain deferred to the pre-Phase-15 destructive/recovery boundary; intentional destructive recovery tests belong to Phase 15.

This HA POC is **not** trying to prove a production disaster-recovery platform. It still needs a real rollback boundary before we deliberately kill leaders, members, and hosts.

HA-preserving scope boundary: Phase 13 does not authorize removing or bypassing any commissioned HA component. Patroni/Spilo, etcd, HAProxy, PgBouncer, Valkey/Sentinel, MQTT redundancy, and the two-node ChirpStack design remain part of the active system. For normal functional commissioning, use the already-passed live HA health/routing evidence plus the fresh validated local logical dumps created on 2026-08-27. The more rigorous off-Droplet copy and isolated-restore rehearsal may be deferred until the dedicated destructive/failure-injection boundary unless a new failure signal, migration risk, or operator requirement makes them immediately necessary. Do not confuse "defer rigorous DR proof" with "remove HA".

The rule is simple:

Do not inject a destructive failure until the fully commissioned state needed to rebuild the POC exists outside the three target Droplets and the required isolated restore checks have passed.

13.0 Two mandatory backup checkpoints

Phase 13A - before gateway/device cutover

Normally complete after Phase 11 and before Phase 12. If the physical gateway is temporarily unavailable, the **cloud-only 13A checkpoint may be executed and closed first** because it does not mutate gateway state: protect the commissioned cloud databases/configuration, copy the backups off the target Droplets, validate catalogs/checksums, and complete an isolated `lorawan_telemetry` restore with Timescale objects intact. When a legacy authoritative source exists, protect that source independently as part of the migration plan. Phase 12 still requires both Phase 11 normal-path commissioning and Phase 13A PASS; finishing 13A early does not waive the gateway prerequisite.

13A PASS permits Phase 12 cutover/provisioning. It does not permit Phase 15 fault injection.

Phase 13S - interim server-only snapshot/export while external dependencies are deferred

13S exists only to remove avoidable server-side work from the final commissioning day. It may run now against the commissioned server components and should capture non-secret reconstruction material for etcd, PostgreSQL/Patroni/TimescaleDB, HAProxy/PgBouncer, Mosquitto, Valkey/Sentinel, ChirpStack, Node-RED, Grafana, OpenBao, and the Fabric outbox database layer. It may also record current image digests, configuration hashes, service/listener state, and a fresh logical database backup if the operator deliberately wants a newer rollback point.

Do **not** fabricate missing artifacts. The original 13S pre-staging pass predated later commissioning of the physical gateway normal path, real EMU-01 rows, the HRC Task 37 handoff, ULC-01 Fabric production writes, adapter images/configuration, and gateway-integrity runtimes; those artifacts now exist and must be included in the next refreshed recovery checkpoint from their verified current locations. The genuinely open infrastructure gates are provider Cloud Firewall/control-plane evidence and Reserved-IP reassignment/failover acceptance, plus ULC-02 Fabric ownership/fencing acceptance and any explicitly deferred destructive restore/failover tests. Do not place plaintext passwords, private keys, AppKeys, OpenBao root/recovery material, Fabric private credentials, or bearer tokens in the general 13S archive.

For normal server preparation, a successful 13S checkpoint means the locally verified server-only package is intact and reproducible. Once `PHASE13S_LOCAL_PACKAGE=PASS` has been reached, treat `SERVER_ONLY_SNAPSHOT_EXPORT=PASS` as satisfied for the current non-destructive commissioning scope and stop repeating backup/export work. Off-host copying, isolated restore rehearsal, and stronger disaster-recovery proof are deferred to the dedicated destructive/failure-testing boundary before Phase 15. This still does **not** satisfy final Phase 13B, which must be refreshed after the remaining provider, hardware, Fabric, and final acceptance work is commissioned.

Phase 13S-1 initial three-host preflight harness stop - 2026-08-29

The first 13S-1 wrapper proved the `ulc-03` control-host/free-space gate and noninteractive SSH + `sudo -n` access to both `ulc-01` and `ulc-02`, then returned cleanly to the `ulc-03` prompt before building or running the host probe. No server/service/database mutation occurred. The stop is a shell-transport harness defect: the wrapper itself was being supplied to `sudo bash` through a heredoc, while the Step 2 `ssh` commands inherited that same stdin. SSH is allowed to consume stdin, so it consumed the remaining wrapper text after the access checks. This is not evidence of a server, SSH, `sudo`, backup-tool, or service failure.

Do not repeat the already-passed access gates merely to diagnose this. Continue from probe construction. When an SSH command inside a heredoc-driven wrapper does **not** intentionally transport a script or payload over stdin, use `ssh -n` or redirect stdin from `/dev/null`. When stdin transport is intentional, redirect from the exact file being sent (for example `ssh ... 'sudo -n bash -s -- ulc-01' < "$PROBE"`) so SSH cannot consume the parent wrapper.

Phase 13S-1 resumed inventory partial PASS / listener-filter harness defect - 2026-08-29

The corrected resume reached all three hosts and proved the useful inventory through OpenBao tooling. `ulc-01`, `ulc-02`, and `ulc-03` all reported Ubuntu 24.04.4 LTS, zero failed systemd units, healthy commissioned service/container baselines, `etcdctl 3.4.30`, PostgreSQL 18.6 `pg_dump/pg_restore`, and OpenBao CLI 2.6.2. Container restart counts were zero and no OOM state was reported. `ulc-03` additionally showed Grafana and active Node-RED running as expected. Patroni remained exactly one leader on `10.104.0.2` plus two replicas on `.4` and `.8`.

The probe then hit `awk`: unexpected newline or end of string in the multi-line listener filter on every host. Because the outer resume wrapper did not use fail-fast handling for each remote probe, it continued to Patroni and printed a final `PHASE13S_SERVER_INVENTORY_PREFLIGHT=PASS` banner even though the per-host probe had exited before listener output, non-secret configuration metadata, and `HOST_PREFLIGHT_*` markers. Therefore that final

banner was not authoritative until the missing tail was rerun. Preserve all successful inventory/tool/resource/Patroni evidence and do not repeat OS/resource/tool discovery.

Phase 13S-1 three-host inventory preflight COMPLETE / PASS - 2026-08-29

The corrected listener/configuration tail passed on all three servers. Every host retained the required etcd, PostgreSQL, PgBouncer, Patroni, OpenBao, and HAProxy database listeners. u1c-01 and u1c-02 also retained the commissioned MQTT, ChirpStack, and OpenBao HAProxy listeners; u1c-03 retained Node-RED on 127.0.0.1:1880, Grafana on 127.0.0.1:3000, and the local Node-RED MQTT frontend on 10.104.0.8:18884. Non-secret reconstruction configuration metadata and SHA-256 values were captured without displaying contents, while PgBouncer userlists and Node-RED/Grafana protected environment files were represented by metadata only.

Combined with the immediately preceding inventory, all three hosts have zero failed units, available backup tooling (etcdctl 3.4.30, PostgreSQL 18.6 pg_dump/pg_restore, OpenBao CLI 2.6.2), current containers with restart count zero/OOM false, and the expected commissioned service layout. Patroni remains one leader on 10.104.0.2 with replicas on .4 and .8. HOST_PREFLIGHT_u1c-01=PASS, HOST_PREFLIGHT_u1c-02=PASS, HOST_PREFLIGHT_u1c-03=PASS, PATRONI_BASELINE=PASS, and PHASE13S_SERVER_INVENTORY_PREFLIGHT=PASS are authoritative. No service or database state changed.

Next: create a protected timestamped 13S snapshot set containing fresh custom-format chirpstack and lorawan_telemetry logical dumps plus a validated etcd v3 snapshot/member record. OpenBao Raft snapshotting is a separate privileged sub-boundary; never use the Fabric adapter identity for operator snapshot privileges.

Phase 13S-2 PostgreSQL + etcd snapshot COMPLETE / PASS - 2026-08-29

The protected server-only snapshot set at /home/opsadmin/backups/phase13s-20260829T022146Z is valid. Fresh custom-format dumps were created from the healthy u1c-03 Spilo replica: chirpstack.dump is 101218 bytes and lorawan_telemetry.dump is 72384 bytes; both passed pg_restore --list. The repeated en_US.utf-8 Perl warnings remain non-blocking locale noise, and the TimescaleDB continuous_agg circular-foreign-key warning is the same known full custom-dump warning already recorded in Phase 13A.

The first etcd health-count attempt stopped only because etcdctl endpoint health emitted the three successful endpoint results on stderr while the wrapper redirected stdout only. No etcd failure occurred and the PostgreSQL dumps were preserved rather than recreated. The corrected resume captured stdout+stderr for each endpoint and proved all three endpoints healthy. etcdctl is 3.4.30; the live etcd servers are 3.5.15. The authoritative member/status record showed three started non-learner members, etcd-02 on 10.104.0.4 as current etcd leader, and a common Raft term 2 / index 1387 at capture time.

A v3 snapshot was then created from 10.104.0.8:2379 as etcd.snapshot, size 491552 bytes. Snapshot status reported hash c0b2b3a5, revision 1326, 1336 total keys, and about 492 kB. The snapshot set contains nine mode-0600 opsadmin:opsadmin files under a mode-0700 opsadmin:opsadmin directory: source metadata, both database dumps, etcd health/status/member evidence, etcd snapshot/status, and SHA256SUMS. sha256sum -c passed before and after ownership/permission normalization. Patroni still reported exactly one leader on 10.104.0.2 and replicas on .4/.8 after snapshot creation. PHASE13S_POSTGRES_ETCD_SNAPSHOT=PASS is authoritative. Do not recreate these exact dumps/snapshot unless the source state changes; continue by adding secret-free reconstruction/application exports in a separate 13S sub-boundary.

Phase 13S-2 PostgreSQL dumps PASS / etcd health-capture harness stop - 2026-08-29

The protected snapshot directory /home/opsadmin/backups/phase13s-20260829T022146Z was created successfully on u1c-03. The pre-snapshot gate proved u1c-03 is still a Patroni replica with replay caught up (true|true), etcdctl is 3.4.30, and sufficient backup space exists. SOURCE-METADATA.txt was created, a fresh chirpstack.dump of 101218 bytes passed pg_restore --list, and a fresh lorawan_telemetry.dump of 72384 bytes passed pg_restore --list. The repeated en_US.utf-8 locale warning remains non-blocking. The TimescaleDB

continuous_agg circular-foreign-key warning is the already-known full custom-format dump warning and did not invalidate the telemetry catalog check.

Step 6 then stopped on a **health-output capture defect, not an etcd health failure**. etcdctl endpoint health visibly returned successful committed-proposal health for all three endpoints (10.104.0.2, .4, .8), but this etcdctl build emitted those human-readable health lines on stderr. The wrapper redirected only stdout to etcd-endpoint-health.txt, leaving that file empty; the subsequent grep -c 'is healthy' therefore returned zero and intentionally stopped before snapshot creation. Do not recreate the two valid PostgreSQL dumps. Reuse this exact partial directory, capture each etcd endpoint health with stderr included (or rely on the command exit status), then continue with endpoint/member records, etcd snapshot, SHA-256 manifest, ownership/mode normalization, and the final Patroni regression gate.

Phase 13S-3 secret-free reconstruction export COMPLETE / PASS - 2026-08-29

The reconstruction bundle under /home/opsadmin/backups/phase13s-20260829T022146Z/reconstruction is complete for the currently commissioned server-only state. Three host manifests capture OS/service/container/listener state plus safe configuration hashes and protected-file metadata without copying environment files, private keys, PgBouncer userlists, or credential databases. Node-RED B was re-proved fenced with no :1880 listener and no flows_cred.json, then the exact reviewed shared runtime (compose.yml, settings.js, package.json, package-lock.json, flows.json) was exported from the stopped standby and matched all authoritative hashes. Grafana compose.yml, datasource provisioning, dashboard provisioning, and lorawan-overview.json passed literal-secret checks before export; grafana.env remained excluded. The reconstruction tree contains 16 mode-0600 opsadmin:opsadmin files under mode-0700 directories and its own SHA-256 manifest passed.

The original Phase 13S-2 PostgreSQL/etcd SHA256SUMS was rechecked afterward and remained unchanged. Node-RED A remained running|0|healthy, Grafana running|0|false, and Node-RED B remained fenced.

PHASE13S_RECONSTRUCTION_EXPORT=PASS is authoritative. Provider-owned public ingress, physical gateway/EMU-01 acceptance, Fabric adapter/handoff, gateway-integrity runtime, final Phase 13B, Phase 14, and Phase 15 remain explicitly deferred or blocked as recorded in DEFERRED-AND-SEPARATE.txt.

The next server-only backup boundary is OpenBao. Cloud production currently has no dedicated snapshot-only backup AppRole. Create and verify a least-privilege openbao-raft-snapshot-reader policy and openbao-backup AppRole before taking the Raft snapshot. Do not use the fabric-adapter role and do not take the snapshot with a long-lived root token. Prefer short-lived one-use SecretIDs that are generated only for the snapshot operation and never stored in the general reconstruction archive.

Phase 13S-4A OpenBao snapshot-only backup identity COMPLETE / PASS - 2026-08-29

The three-node OpenBao cluster was healthy and unsealed before mutation. The existing lorawan-evidence Transit key remained ecdsa-p256, non-exportable, plaintext backup disabled, deletion disabled, and version 1; the fabric-adapter AppRole retained only the fabric-evidence-signer policy and had zero SecretID accessors.

A new policy openbao-raft-snapshot-reader was created with exactly one path: sys/storage/raft/snapshot, capabilities read and sudo. A new openbao-backup AppRole was then created with only that policy, token TTL 900 seconds, token max TTL 1800 seconds, SecretID TTL 900 seconds, secret_id_num_uses=1, and bind_secret_id=true. The RoleID exists but was not printed. No backup SecretID was issued during this step and the Fabric adapter still had zero SecretID accessors afterward. All three OpenBao nodes remained initialized and unsealed after the mutation. PHASE13S_OPENBAO_BACKUP_IDENTITY=PASS is authoritative.

Next: issue exactly one ephemeral backup SecretID in memory, authenticate once, prove a second login with the same SecretID fails, verify the resulting token has read,sudo only on sys/storage/raft/snapshot and deny on Transit sign/verify and Raft restore/force paths, save one OpenBao Raft snapshot, transfer it into the protected Phase 13S set, revoke the short-lived token, and confirm no reusable backup SecretID remains. Do not use the root token for the snapshot operation itself.

Phase 13S-4B first snapshot attempt stopped after SecretID issuance - 2026-08-29

The protected OpenBao snapshot workflow passed the existing 13S integrity check, loaded the protected root token without printing it, reverified the openbao-backup role, proved zero backup SecretID accessors before issuance, and confirmed a three-voter Raft configuration with 10.104.0.2 as leader at that checkpoint. The root administrative identity then successfully issued one new one-use backup SecretID. Immediately afterward, the wrapper attempted to count SecretID accessors using a Python parser that assumed the JSON document was an object containing data.keys. OpenBao 2.6.2 returned the list result as a JSON list in this invocation, so Python raised `AttributeError: 'list' object has no attribute 'get'` and the remote shell exited before AppRole login or snapshot creation.

Treat this as a harness/schema-shape defect, not an OpenBao or Raft failure. The just-issued SecretID value existed only in the failed shell and is no longer recoverable, but its server-side accessor may remain until its 15-minute TTL expires because it was never consumed. Before issuing another SecretID, list accessors with a parser that accepts both top-level JSON lists and wrapped data.keys forms. Require at most one accessor because the role had zero immediately before the failed issuance; if one orphan accessor remains, destroy that exact accessor administratively and re-prove zero. Preserve any partial local openbao-raft-evidence.txt transcript separately. Do not rerun Phase 13S-4A, do not recreate the backup role/policy, and do not claim an OpenBao Raft snapshot exists from this failed run.

Phase 13S-4B OpenBao Raft snapshot COMPLETE / PASS - 2026-08-29

The corrected resume found exactly one orphan openbao-backup SecretID accessor from the failed parser run, destroyed that accessor administratively, and re-proved a zero-accessor clean baseline before minting a fresh credential. The fresh SecretID was issued under the commissioned 15m / one-use backup-role constraints, authenticated successfully exactly once, and a second login with the same SecretID was rejected. The consumed accessor disappeared immediately.

The resulting backup token was verified to have exactly read + sudo on sys/storage/raft/snapshot, while Transit sign, Transit verify, and Raft restore/force capabilities were all denied. Using this restricted token against the current Raft leader 10.104.0.2, OpenBao created /tmp/phase13s-openbao.snap; the protected host copy was 30236 bytes with SHA-256 9965f9e904b83d62b07ebb9321af1b4a45a7f2b15fbfd50d0c7238fac249e68. The token was then explicitly revoked. Final backup SecretID accessors = 0; Fabric adapter SecretID accessors = 0; all three OpenBao members remained initialized/unsealed.

The snapshot was transferred into /home/opsadmin/backups/phase13s-20260829T022146Z/openbao.snap, and its destination SHA-256 exactly matched the source. The remote staging copy was removed. openbao-raft-evidence.txt, the preserved failed-run transcript openbao-raft-evidence.partial-accessor-parser.txt, and OPENBAO-SHA256SUMS were protected as mode 0600 opsadmin:opsadmin; the OpenBao manifest verified all three artifacts. The original PostgreSQL/etcd snapshot manifest and the reconstruction manifest still pass unchanged. PHASE13S_OPENBAO_RAFT_SNAPSHOT=PASS is authoritative.

Phase 13S-5 final local server-only package COMPLETE / PASS - 2026-08-29

The complete interim server-only snapshot set under /home/opsadmin/backups/phase13s-20260829T022146Z was revalidated before packaging. The existing PostgreSQL/etcd, OpenBao, and reconstruction manifests all passed. Required database dumps, etcd snapshot, OpenBao Raft snapshot, Node-RED reviewed runtime export, Grafana provisioning/dashboard export, and deferred/blocker record were present. A filename-level secret gate found no .env, private-key, password/userlist, SecretID, root-token, recovery-share, or unseal-share files in the general archive. All source files remained mode 0600 opsadmin:opsadmin and directories mode 0700 opsadmin:opsadmin.

SERVER-ONLY-STATUS.txt records the server-only/interim scope and explicitly leaves provider ingress, physical gateway/real EMU-01 acceptance, Fabric runtime/handoff/transaction, gateway-integrity runtime, final Phase 13B, final Phase 14, Phase 14B, and Phase 15 unclaimed. PHASE13S-FULL-SHA256SUMS covers 30 payload files and passed. The transport archive /home/opsadmin/backups/phase13s-20260829T022146Z.tar.gz was created outside

the source directory, is 108819 bytes, passed `gzip -t`, contained all required top-level artifacts, and passed a disposable extraction followed by `sha256sum -c PHASE13S-FULL-SHA256SUMS`. Its transport SHA-256 is 19f2072fd4ef41e34442c7cc3949decd62a0cb2cee155bcf734f24121397cc; the sidecar is /home/opsadmin/backups/phase13s-20260829T022146Z.tar.gz.sha256. PHASE13S_LOCAL_PACKAGE=PASS is authoritative.

Streamlined commissioning decision - 2026-08-29: the locally verified package is sufficient for the current server-first preparation scope. The archive has already passed its full 30-file SHA-256 manifest, gzip structure test, disposable extraction, and extracted-manifest verification, so `SERVER_ONLY_SNAPSHOT_EXPORT=PASS` is accepted now. Do not copy it off-host merely to continue normal-path setup, and do not rebuild or rehash it again unless the protected source state materially changes. The `SERVER-ONLY-STATUS.txt` inside this already-built archive still contains the earlier stricter `NOT_YET_OFFHOST_VERIFIED` wording; preserve that file as historical evidence rather than rebuilding the archive solely to change a label. Off-host copy and isolated restore proof remain deferred to the pre-Phase-15 destructive/recovery boundary. This is still an interim Phase 13S package, **not** final Phase 13B.

Phase 13B - after every pre-test service is commissioned

Re-run the backup collection after Phase 12A, Phase 14A, and Phase 20. Include Node-RED flow/config, Grafana dashboard/data-source exports, OpenBao recovery/Raft snapshot material, Fabric adapter image/config references, public-ingress scripts/units, and the final database state.

13B PASS is one prerequisite of Phase 14B. The backup identifier used by Phase 15 must refer to this fully commissioned snapshot, not an earlier 13S/13A partial build.

13.1 What to protect

Before destructive tests keep copies of:

```
chirpstack logical database dump
lorawan_telemetry logical database dump
etcd cluster snapshot + member/config record
OpenBao recovery/unseal material outside the runtime hosts
OpenBao Raft snapshot before destructive KMS recovery tests
HAProxy/PgBouncer configs
public-ingress failover scripts + systemd units/timer
Reserved IPv4 + ha-01/ha-02 Droplet IDs + anchor IPv4 record
ChirpStack configs/region files
Mosquitto configs/ACLs
Valkey/Sentinel configs
Node-RED flow export
Grafana dashboard export
Fabric handoff metadata
Fabric adapter config/image reference when an implementation exists
```

Do not create paid object storage only because a future production design normally would. For this POC, the administration workstation or another protected system outside ha-01/02/03 is sufficient as the off-host copy.

Current Phase 13A preflight result - 2026-08-27

The cloud-only read-only preflight is **PASS**. The initial checker falsely treated missing ulc-03 :16379 and :18883 listeners as failures. Those frontends are intentionally application-node-only: Valkey writable-primary HAProxy is commissioned on ulc-01/02, and the ChirpStack workload MQTT HAProxy frontends are commissioned on ulc-01/02. Do not deploy either frontend on ulc-03. Patroni, etcd, all required database routes, both ChirpStack application nodes, PostgreSQL/TimescaleDB state, telemetry hypertables/schema state, backup tools, and local backup capacity passed. Continue with fresh Phase 13A dumps.

The first fresh-dump wrapper then stopped safely before creating either dump because it required `host(inet_server_addr()) = 10.104.0.8` while `psql` was using the local Unix-domain socket inside the Spilo container. On a Unix-socket PostgreSQL session `inet_server_addr()` is `NULL`, so the observed `|t` means only `pg_is_in_recovery() = true`; it is not evidence of a wrong node. For local-container backup work, verify the host identity outside SQL and require only replica state from PostgreSQL, or explicitly connect by TCP if the server address itself is part of the assertion. Reuse the existing timestamped directory only if it contains no dump artifacts; otherwise create a new timestamped directory.

13.2 Create one protected backup directory

Run on the host from which backups are being collected:

```
umask 077
BACKUP_DIR="$HOME/backups/lorawan-ha-poc-$(date -u +%Y%m%dT%H%M%SZ)"
mkdir -p "$BACKUP_DIR"
printf '%s\n' "$BACKUP_DIR"
```

Store the test/run identifier beside the backup so later evidence shows which failure rehearsal it protected.

Do not put database passwords, private keys, or OpenBao recovery material in the general command transcript.

13.3 PostgreSQL logical dumps through the stable HA path

Both logical databases live on the same Patroni cluster:

```
chirpstack
lorawan_telemetry [TimescaleDB enabled]
```

Use a protected backup/admin role that can read the required database objects. Connect through the local PgBouncer/HAProxy route or an explicitly verified current-primary admin route.

If using PgBouncer, first confirm `<BACKUP_ROLE>` is actually admitted by the configured `auth_file/auth_query` and can connect to **both** logical databases. Do not weaken PgBouncer authentication just to make a backup work. If the backup role is intentionally excluded from PgBouncer, use a separately approved TLS-verified current-primary administration path and record that exception.

Example using the stable pool path:

```
pg_dump \
--host=pgbouncer.internal.lorawan.com \
--port=6432 \
--username=<BACKUP_ROLE> \
--dbname=chirpstack \
--format=custom \
--file="$BACKUP_DIR/chirpstack.dump"

pg_dump \
--host=pgbouncer.internal.lorawan.com \
--port=6432 \
--username=<BACKUP_ROLE> \
--dbname=lorawan_telemetry \
--format=custom \
--file="$BACKUP_DIR/lorawan_telemetry.dump"
```

Provide authentication through the approved protected mechanism such as `.pgpass`, a short-lived credential source, or an interactive prompt. Do not put the live password in the command line.

Immediately validate both catalogs:

```
pg_restore --list "$BACKUP_DIR/chirpstack.dump" >/dev/null
pg_restore --list "$BACKUP_DIR/lorawan_telemetry.dump" >/dev/null
sha256sum "$BACKUP_DIR/chirpstack.dump" "$BACKUP_DIR/lorawan_telemetry.dump" \
| tee "$BACKUP_DIR/SHA256SUMS"
```

Stop here if either `pg_restore --list` fails or either dump is zero bytes.

Why both dumps matter: `lorawan_telemetry` contains the Timescale hypertables and the current application schema. `telemetry.fabric_outbox` is included **only after it has actually been commissioned** by the later Node-RED/Fabric setup. At the current pre-Phase-12A 13A checkpoint, preserve the live source state exactly; do not create an outbox merely to make the backup match the future target architecture.

Current Phase 13A fresh logical backup result - 2026-08-27

The corrected fresh-backup retry is **PASS** on `ulc-03`. Host identity was verified outside PostgreSQL and the local Spilo member returned `pg_is_in_recovery() = true`. Both `chirpstack` and `lorawan_telemetry` exist. Source metadata recorded PostgreSQL 18.6, ChirpStack counts `tenant=1, application=0, device_profile=0, device=0, gateway=0`, telemetry counts `uplinks=0, measurements=0, device_registry=0`, TimescaleDB 2.29.2, hypertables `measurements,uplinks, and fabric_outbox_count=0`. Fresh custom-format dumps were created under `/home/opsadmin/backups/phase13a-20260827T032756Z`: `chirpstack.dump` about 99 KiB and `lorawan_telemetry.dump` about 58 KiB. Both archives passed `pg_restore --list`; `SOURCE-METADATA.txt`, both dumps, and `SHA256SUMS` are mode `0600`; `sha256sum -c SHA256SUMS` passed for all three hashed files. The TimescaleDB `continuous_agg` circular-foreign-key warning is expected for this full custom-format dump and does not invalidate the archive; the isolated restore rehearsal remains the authoritative restore proof. The database-backup sub-gate is therefore complete locally, but Phase 13A remains open until this directory is copied off the target Droplets and destination hashes are verified.

Streamlined Phase 13A path for normal-path commissioning - 2026-08-28

For the current POC, use the minimum rollback boundary needed to continue **non-destructive** Phase 12 / normal-path testing quickly. The fresh `chirpstack` and `lorawan_telemetry` dumps have already passed local `pg_restore --list` and SHA-256 verification on `ulc-03`; do not recreate or re-parse them unless the source state changes.

The fast Phase 13A checkpoint is:

```
existing verified logical dumps on ulc-03
-> package/copy once to the administration workstation
-> verify one transport SHA-256
-> retain the off-host copy
-> continue normal-path commissioning
```

Do **not** require an isolated database restore, etcd snapshot restore rehearsal, destructive membership exercise, or failure injection merely to continue normal-path commissioning. Those stronger DR proofs become mandatory before Phase 15 destructive/failover testing. This streamlining does not remove Patroni, etcd, HAProxy, PgBouncer, Valkey/Sentinel, MQTT HA, or any other HA component.

For the current backup `/home/opsadmin/backups/phase13a-20260827T032756Z`, one compressed archive plus one SHA-256 sidecar is sufficient for the transport step. The internal dump catalogs and hashes are already proven at source.

13.4 Copy PostgreSQL backups off the three Droplets

If the backup was created on ha-01, ha-02, or ha-03, copy the entire protected backup directory to the administration workstation or another approved off-host location before the destructive test.

Then recalculate the checksums at the destination and compare with SHA256SUMS.

A dump that exists only on a Droplet scheduled for failure is not a useful rollback copy.

Phase 13A off-host transport PASS - 2026-08-28

The compressed transport archive `phase13a-20260827T032756Z.tar.gz` was copied from `u1c-03` to the Windows administration workstation using the proven `id_ed25519_home_ops` SSH identity. The source SHA-256 was `e97d50c31252ede1fe55b734b6686f270e92ebecb69a36d637b04fbf726cda1c`; Windows `Get-FileHash -Algorithm SHA256` returned the same value and the operator obtained `PHASE13A_OFFHOST_COPY=PASS`. Treat the streamlined Phase 13A transport checkpoint as PASS. For normal-path commissioning, do not repeat the database dumps, dump-catalog checks, isolated restore rehearsal, or destructive DR tests now; the stronger recovery proof remains deferred to the later destructive/failover test boundary.

13.5 etcd snapshot before destructive coordination tests

For a normal single-member loss, the preferred recovery is to restore/rejoin the missing member using the normal etcd member procedure. A cluster snapshot is still mandatory safety evidence before deliberately changing membership or performing a destructive quorum-recovery test.

Using a compatible `etcdctl` against the currently tested east-west endpoints:

```
ETCDCTL_API=3 etcdctl \
  --endpoints=http://10.104.0.2:2379,http://10.104.0.4:2379,http://10.104.0.8:2379 \
  endpoint health

ETCDCTL_API=3 etcdctl \
  --endpoints=http://10.104.0.2:2379 \
  snapshot save "$BACKUP_DIR/etcd.snapshot"
```

These commands match the current HTTP-only etcd deployment on `10.104.0.0/20`. If etcd transport TLS is deployed later, update this procedure and use the tested CA/client credentials; do not mix the two transport modes.

Inspect the snapshot using the status command supported by the pinned etcd release (`etcdctl snapshot status` on releases that use `etcdctl`). Record:

```
etcd member list
cluster ID
member IDs / names / peer URLs
snapshot hash/revision/status
```

The etcd snapshot contains coordination state, not PostgreSQL rows.

13.6 OpenBao recovery material and Raft snapshot

Keep Shamir/recovery/unseal material **outside** `ha-01/02/03` and outside the general configuration archive.

Before an OpenBao failure rehearsal record:

```
member list
active node
sealed/unsealed state
Raft health
Transit mount
lorawan-evidence key metadata
```

Before a test that can damage Raft state, use a privileged backup identity—not the Fabric adapter—to create the approved snapshot, for example with the pinned CLI:

```
bao operator raft snapshot save "$BACKUP_DIR/openbao.snap"
```

Then copy that snapshot off the three Droplets and retain the protected recovery material needed to restore/unseal the cluster.

The Fabric adapter must not receive root, recovery, or snapshot privileges.

13.7 Configuration snapshot

Capture the non-secret configuration that would be needed to reconstruct the scale model:

```
HAProxy
PgBouncer
public-ingress scripts and systemd service/timer units
Reserved IPv4, ha-01/ha-02 Droplet IDs, anchor IPv4s, and expected initial/current owner
public-ingress health/takeover/evaluator script hashes
Patroni/Spilo non-secret config and image digest
etcd member/config and pinned version
Mosquitto config + ACL + image digest
Valkey/Sentinel config + image digest
ChirpStack region/config files + image digest
Node-RED flow export + palette versions
Grafana dashboard/data-source export
OpenBao server config + version
Fabric handoff metadata
Fabric adapter non-secret config/image digest when available
```

Keep private keys/passwords in their designated protected recovery location, not in a general tar archive. In particular, do **not** copy the live DIGITALOCEAN_TOKEN from /etc/lorawan-cloud/public-ingress.env into the ordinary configuration bundle. Record its secret-store reference and required permission scope, then restore the token separately through the approved secret workflow.

13.8 Isolated PostgreSQL restore rehearsal

Do this before the first destructive HA test and after any PostgreSQL-major/TimescaleDB change.

Use a temporary target that has:

```
same compatible PostgreSQL major
same required TimescaleDB extension build available
no production data
no connection from live ChirpStack/Node-RED/Fabric adapters
```

Procedure:

1. create a temporary empty PostgreSQL instance/database outside the live three-node cluster when possible;
2. verify the TimescaleDB extension files are available for the selected PostgreSQL major;

3. create/enable the extension as required by the restore procedure;
4. run `pg_restore --list lorawan_telemetry.dump` again on the restore host;
5. restore with `--exit-on-error`;
6. query `pg_extension` and confirm `timescaledb` is present;
7. query `timescaledb_information.hypertables` and confirm `telemetry.uplinks` and `telemetry.measurements` are hypertables;
8. compare `telemetry.fabric_outbox` with the recorded source state: if the source backup contains it, confirm it restores as an ordinary table; if the source records it as `not-yet-commissioned`, confirm it remains absent rather than creating it during restore;
9. compare the small POC row counts with the source evidence;
10. verify the `telemetry_reader/writer` ownership/grants needed by the restored schema;
11. capture the sanitized restore result;
12. destroy the temporary restore target after the evidence is retained.

Do not call a backup tested merely because `pg_dump` exited zero. The restore rehearsal is what proves the dump is usable.

13.9 Outbox recovery rule

After restoring `lorawan_telemetry`:

1. keep both Fabric adapters stopped during database restore
2. validate the restored outbox and current ownership/fencing state
3. start only ULC-01 adapter-1 as the commissioned writer
4. inspect pending / processing / reconciling / confirmed / failed / needs_attention rows; treat submitted_unknown as legacy compatibility state
5. release or wait out only leases proven stale
6. reconcile any row with durable Fabric transaction material by commit status plus source-bound query/digest verification
7. verify no row is falsely marked confirmed
8. keep ULC-02 adapter-2 write-disabled unless the separate HA takeover gate is deliberately being executed

Why: a Fabric transaction may have committed even when the client timed out. Blind replay can create duplicate/conflicting work.

The adapter implementation and ULC-01 production writer are commissioned. If they are unavailable during a disaster recovery event, preserve the outbox and prepared transaction material; do not replace them with an improvised writer or enable ULC-02 outside the approved HA takeover procedure.

13.10 What is deliberately deferred

Not required to pass the **structural HA POC**:

WAL-G
continuous WAL archiving
PITR
paid object storage
multi-region backup replication
long production retention schedules
production RPO/RT0 certification

These are future deployment hardening, not reasons to remove Patroni, TimescaleDB, OpenBao, or the outbox from the POC.

13.11 Phase 13A pass condition

For the current **non-destructive normal-path commissioning**, Phase 13A passes when:

- the already-validated current chirpstack and lorawan_telemetry logical dumps exist outside the target Droplets;
- the off-host transport archive/hash matches the source transport hash;
- current non-secret HAProxy/PgBouncer/Mosquitto/Valkey/ChirpStack configuration references remain documented;
- when a legacy authoritative source exists, its final migration backup/off-host destination is ready.

Do not repeat source `pg_restore --list` or dump SHA-256 checks when those exact dump files have already passed and have not changed.

Before **Phase 15 destructive/failover testing**, complete the stronger DR gate that was intentionally deferred for speed: isolated `lorawan_telemetry` restore with Timescale objects intact, required `etcd` snapshot/member/config evidence, and any other destructive-recovery prerequisite relevant to the planned fault.

After the streamlined **13A PASS**, continue to [12-gateway-and-device-migration.md](#) only when the Phase 11 normal path is also ready. Do not start fault injection until the stronger pre-Phase-15 DR gate passes.

13.12 Phase 13B final full-stack snapshot

After Node-RED, Grafana, OpenBao, and the Fabric adapters have passed their normal-path setup, repeat the backup collection and add:

```
Node-RED flow export + image/palette/config references
Grafana dashboard/data-source export without credentials
OpenBao recovery material protected off-host
OpenBao Raft snapshot + version/member record
Fabric adapter immutable image/source reference + worker IDs/config reference
Fabric external handoff metadata + last normal confirmed tx reference
public-ingress Reserved IPv4/Droplet/anchor worksheet
public-ingress script/systemd hashes and protected token reference
final schema/migration/outbox state
final service image/config hashes needed to recreate the tested baseline
```

Re-check the database dumps and isolated restore against the final schema. Phase 13B must not rely only on the earlier 13A archives because the application/outbox state has changed.

Phase 13B PASS does not itself authorize fault injection. Next run the healthy evidence-harness dry run in [14-observability-alerting-and-logging.md](#), then the hard gate in [14b-pre-test-commissioning-gate.md](#).

Phase 13A transport archive created PASS - 2026-08-28

The streamlined off-host transport archive was created successfully on `u1c-03` from the already-validated source directory `/home/opsadmin/backups/phase13a-20260827T032756Z`. The resulting archive is `/home/opsadmin/backups/phase13a-20260827T032756Z.tar.gz`, size about 28 KiB, with SHA-256 `e97d50c31252ede1fe55b734b6686f270e92ebecb69a36d637b04fbf726cda1c`. The sidecar `phase13a-20260827T032756Z.tar.gz.sha256` was created successfully. Treat archive creation as PASS. The only remaining fast-path transport gate is to copy the archive to the administration workstation and confirm the destination SHA-256 matches exactly; do not recreate the database dumps or rerun their source validation.

Supporting runbook: 00-README.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

PostgreSQL and TimescaleDB Telemetry Storage

Use these guides to build the TimescaleDB-backed telemetry schema. There are **two deployment shapes**:

- local/single-host lab: a separate `telemetry-db` PostgreSQL + TimescaleDB service;
- cloud HA POC: TimescaleDB is installed as an extension on all three Patroni/PostgreSQL members and enabled in the separate logical database `lorawan_telemetry` inside that HA cluster.

Before applying changes, identify which profile you are using and verify PostgreSQL/Timescale versions, database roles, retention policy, and backup state.

Unless a step explicitly opens a `psql` prompt or names another host, run shell commands on the LoRaWAN application server from `/opt/lorawan-lab`. This is the Compose directory created by the server lab procedure; use the actual active Compose directory when adapting the integration to another deployment.

The **logical telemetry database and permissions** remain separate from the ChirpStack database. In the local lab they may also be separate PostgreSQL services; in the cloud HA POC they share one Patroni cluster:

```
LoRaWAN sensor
-> RAK5146
-> ChirpStack
-> MQTT JSON uplink event
-> Node-RED
-> telemetry-db
-> generic uplink hypertable
-> measurements hypertable
```

Why keep a separate telemetry database boundary?

The documented ChirpStack deployment uses PostgreSQL for network-server state and enables MQTT integration. Its operational database stores tenants, applications, device profiles, keys, sessions, and frame-counter state. It should not be treated as a stable public telemetry schema for dashboards.

The separate `lorawan_telemetry` database boundary provides:

- an explicit application-owned telemetry schema;
- time-series compression and retention options;
- read-only access for Grafana;
- telemetry write access only for Node-RED;
- a separate `gateway_evidence` schema for the least-privilege verifier/index state when the gateway-integrity integration is enabled;
- simpler backups and restores; and
- protection for ChirpStack when analytical queries become expensive.

The gateway_evidence schema is not a per-sensor schema and does not replace telemetry.uplinks or telemetry.measurements. Large journal segments and captured raw gateway-event objects remain in the protected evidence store; PostgreSQL keeps their hashes, references, lineage, and status.

Multi-sensor design

Do not create a PostgreSQL schema for every sensor model. This design uses:

- one telemetry SQL schema;
- one generic uplinks hypertable with one row per ChirpStack event;
- one measurements hypertable with one row per decoded metric;
- a device registry for model, decoder, site, zone, and asset assignment;
- payload_json for fields that have not yet been promoted to reporting columns.

This means the database can be prepared before any sensor arrives. An empty table is a valid and expected state.

Values used in this guide

Component	Value
Host	Application server running the Compose stack
Compose directory	/opt/lorawan-lab on the application server
MQTT broker hostname	mosquitto
MQTT port inside Docker	1883
Telemetry database hostname	telemetry-db
Telemetry database port inside Docker	5432
Database	lorawan_telemetry
Telemetry writer	Node-RED
Evidence verifier	Separate least-privilege role; updates only gateway-evidence result/state
Reader	Grafana
Sensor data status	No real sensor rows required during installation
Timezone policy	Store timestamps in UTC; display local time in Grafana

Guide map

File	Purpose
01-deploy-timescaledb.md	Local-lab only: add a separate TimescaleDB service; cloud HA uses Timescale inside Patroni instead
02-create-telemetry-schema.md	Create roles, generic multi-sensor tables, hypertables, indexes, views, and retention
03-connect-and-verify.md	Verify connections, permissions, inserts, and query performance

File	Purpose
04-backup-security-and-maintenance.md	Backups, restores, credentials, upgrades, retention, and hardening
05-troubleshooting.md	Diagnose database, extension, permission, and performance failures

Verify the completed database

- TimescaleDB starts automatically with the Compose stack.
- The telemetry database has the TimescaleDB extension enabled.
- The schema accepts new sensor metrics without a new PostgreSQL schema.
- Node-RED can atomically insert one uplink event and its approved normalized measurements as the telemetry writer role.
- When gateway integrity is enabled, the independent verifier can write only the `gateway_evidence` lineage/result fields it owns and cannot rewrite telemetry values.
- Duplicate event and metric retries do not create extra rows.
- A v2 Fabric-selected event can be joined uniquely to one `gateway_evidence.event_verification` result by source event key and observation time.
- Grafana can query telemetry as a read-only role.
- Both time-series tables are partitioned by UTC event time.
- Retention is deliberate and documented.
- A tested backup exists before sensor deployment.

Important rule

Never point Grafana or Node-RED at the ChirpStack core database or give either service the telemetry administrator role. Use the dedicated telemetry database, keep port 5432 internal to the Compose network, and use the least-privilege roles described here.

Do not give Node-RED permission to mark gateway evidence `verified`, and do not give the gateway-evidence verifier permission to sign through OpenBao or submit to Fabric. See [Gateway Integrity](#).

Next: [01-deploy-timescaledb.md](#)

Canonical source: `deployment/technology-guides/07-postgresql-patroni-timescaledb.md`. For any mutable production-status statement, check the dated baseline and verified live state.